

AegisFA: AI-Powered Forensic Assistant for Security Operation Centers

by

Joaquin Baltasar Villegas

A Portfolio submitted in partial fulfillment

of the requirements for the degree of

Bachelor of Science

in

Computer Information Systems

Department of Applied Computing, College of Business and Technology

Lander University

CIS 499 Project Implementation and Management

Spring 2026

Faculty Advisor: Bahar Mahmud Ph.D.

Table of Contents

Chapter 1 – Introduction	13
1.1 Project Background	14
1.2 Problem Identification	16
1.3 Scope Statement.....	19
1.3.1 Log Ingestion and Normalization	19
1.3.2 Threat Analysis and Classification	19
1.3.3 Correlation and Framework Mapping	20
1.3.4 AI-Generated Intelligence	20
1.3.5 Incident Management and Collaboration	20
1.3.6 Multi-Tenancy and Access Control	21
1.3.7 Platform Architecture	21
1.3.8 Out of Scope:	21
1.3.9 Target Users:.....	22
1.4 Project Objectives	22
1.4.1 Objective 1: Automate multi-format log ingestion and normalization.....	22
1.4.2 Objective 2: Classify and detect threats using machine learning and large language models.....	22
1.4.3 Objective 3: Correlate events and map threats to the MITRE ATT&CK framework..	23
1.4.4 Objective 4: Generate actionable intelligence through AI.	23
1.4.5 Objective 5: Enable collaborative incident response.....	23
1.4.6 Objective 6: Deliver an accessible, secure, multi-tenant platform.	23

1.5 Significance	25
1.5.1 Operational: Reducing detection-to-response time.	26
1.5.2 Educational: Lowering the barrier to security analysis.	26
1.5.3 Technical: Demonstrating integrated AI in security operations.	27
1.5.4 Economic: Democratizing enterprise capabilities.	27
1.6 Chapter Summary	28
<i>Chapter 2 – Requirements & Feasibility</i>	<i>32</i>
2.1 Methodology	33
2.2 Functional Requirements	34
2.2.1 Log Ingestion & Normalization (Objective 1)	35
2.2.2 Threat Analysis & Classification (Objective 2)	36
2.2.3 Correlation & MITRE ATT&CK Mapping (Objective 3)	37
2.2.5 Incident Management & Collaboration (Objective 5)	38
2.2.6 Multi-Tenancy & Access Control (Objective 6)	39
2.3 Non-Functional Requirements	40
2.3.1 Performance	40
2.3.4 Scalability & Maintainability	42
2.3.5 Usability	43
2.4.1 Technical Feasibility	44
2.4.2 Scheduling Feasibility	45
2.5 Success Criteria	46
2.6 Team Limitations	47

Chapter 3 – System Design	49
3.1 System Architecture Overview	50
3.2 Database Design.....	52
3.2.1 Schema Overview	52
3.2.2 Core Tables	53
3.2.3 pgvector Integration & MITRE Technique Storage	54
3.2.4 Row-Level Security & Multi-Tenancy	56
3.3 Backend Design	56
3.3.1 Application Structure.....	56
3.3.2 Request Middleware	57
3.3.3 Log Ingestion & Parsing Pipeline.....	58
3.3.4 Normalization Engine.....	58
3.3.5 Threat Analysis Pipeline.....	59
3.3.6 Correlation Engine.....	61
3.3.7 ML Classification (RandomForest)	61
3.3.8 Insights & Natural Language Generation.	63
3.3.9 Background Job Processing.....	64
3.3.10 Timeline Service.....	65
3.4 Frontend Design	66
3.4.1 Application Structure.....	66
3.4.2 Authentication Flow	66
3.4.3 Dashboard.....	67
3.4.4 Admin Panel	71

3.4.5 Workspace	72
3.4.6 AI feedback.....	73
3.5 API Design	81
3.6 Security Architecture	82
3.7 Chapter Summary	83
<i>Chapter 4 – Implementation</i>	<i>86</i>
4.1 Development Environment & Tools	87
4.1.1 Core toolchain:	87
4.1.2 Environment configuration:.....	87
4.2 Database Implementation	88
4.2.1 Migration Strategy	88
4.2.2 pgvector Setup	89
4.3 Backend Implementation.....	89
4.3.1 Application Factory & Middleware.....	89
4.3.2 File Ingestion Implementation	90
4.3.3 Correlation Engine Implementation	91
4.3.4 RandomForest Classifier Implementation	92
4.3.5 RAG Pipeline Implementation	92
4.3.6 Insights Generation Implementation	93
4.3.7 Chunked Upload Implementation.....	94
4.3.8 Storage Service Implementations	94
4.4 Frontend Implementation.....	95

4.4.2 File Upload Implementation	95
4.4.3 Real-Time Progress	95
4.4.4 Persistent State.....	96
4.5 Deployment Implementation	96
4.5.1 Docker Configuration	96
4.5.2 Docker Compose Orchestration.....	96
4.6 Implementation Challenges & Decisions.....	97
4.7 Chapter Summary.....	99
<i>Chapter 5 – Testing & Evaluation.....</i>	<i>101</i>
5.1 Testing Methodology	102
5.2 Functional Testing.....	103
5.2.1 Log Ingestion & Normalization (Objective 1)	103
5.2.2 Threat Analysis & Classification (Objective 2)	107
5.2.3 Correlation & MITRE ATT&CK Mapping (Objective 3)	109
5.2.4 AI-Generated Intelligence (Objective 4)	110
5.2.5 Incident Management & Collaboration (Objective 5).....	112
5.2.6 Multi-Tenancy & Access Control (Objective 6)	112
5.3 Security Testing	114
5.4 Performance Evaluation	115
5.5 Machine Learning Model Evaluation.....	117
5.6 User Acceptance & Usability.....	119

5.7 Results & Discussion	121
5.8 Chapter Summary	124
<i>Chapter 6 – Conclusion</i>	<i>126</i>
6.1 Summary of Achievements	127
6.2 Limitations	129
6.3 Future Work	130
6.4 Chapter Summary	132
6.4 User Manual & Installation/Setup.....	134
6.4.1 System Requirements	134
6.4.2 Installation & Setup	135
6.4.3 Manual Setup (Development).....	136
6.4.4 Troubleshooting.....	140
6.4.5 Support & Logs	141
6.4.6 Security Notes.....	141
<i>References</i>	<i>142</i>

List of Tables

Table 2.1: Structure of Functional Requirements for Log Ingestion & Normalization.	36
Table 2.2: Structure of Functional Requirements for Threat Analysis and Classification.	37
Table 2.3: Structure of Functional Requirements for Correlation and MITRE ATT&CK Mapping	38
Table 2.4: Structure of Functional Requirements for AI-Generated Intelligence.....	38
Table 2.5: Structure of Functional Requirements for Incident Management and Collaboration..	39
Table 2.6: Structure of Functional Requirements for Multi-Tenancy and Access Control.	40
Table 2.7: Structure of Non-Functional Requirements for Performance.....	41
Table 2.8: Structure of Non-Functional Requirements for Security.	42
Table 2.9: Structure of Non-Functional Requirements for Reliability.	42
Table 2.10: Structure of Non-Functional Requirements for Scalability & Maintainability.	43
Table 2.11: Structure of Non-Functional Requirements for Usability.	44
Table 2.12: Structure of Project's Technical Feasibility.	45
Table 2.13: Structure of Project's Scheduling Feasibility.	46
Table 2.14: Structure of Project's Success Criteria.....	47
Table 3.1: Supabase Database Migration Schema for the Proposed System.....	52
Table 4.1: Structure of the Core Toolchain Breakdown.....	87
Table 4.2: Supabase Database Migration Sequence	89
Table 5.1: Structure of the Functional Requirements for Log Ingestion and Normalization (Tested)	105
Table 5.2: Structure of the Functional Requirements for Threat Analysis and Classification (Tested)	109

Table 5.3: Structure of the Functional Requirements for Correlation and MITRE ATT&CK Mapping (Tested)	110
Table 5.4: Structure of the Functional Requirements for AI-Generated Threat Intelligence (Tested)	111
Table 5.5: Structure of the Functional Requirements for Incident Management and Collaboration (Tested)	112
Table 5.6: Structure of the Functional Requirements for Multi-Tenancy and Access Control (Tested)	113
Table 5.7: Machine Learning Model Evaluation Metrics	118
Table 5.8: Structure of the Project's Success Criteria (Tested)	122
Table 6.1: User Manual and System Requirements	134
Table 6.2: User Manual and Services	137
Table 6.3: User Manual and Troubleshooting	141

List of Figures

Figure 3.1: Architecture Diagram of Proposed System	50
Figure 3.2: Full Supabase Database Tables	52
Figure 3.3: Supabase Organizations Database Table.....	53
Figure 3.4: Supabase Users Database Table	53
Figure 3.5: Supabase Log Files Database Table	53
Figure 3.6: Supabase Raw Logs Database Table.....	54
Figure 3.7: Supabase Analysis Results Database Table	54
Figure 3.8: Supabase MITRE Techniques Database Table	55
Figure 3.9: Supabase Hierarchical Navigable Small World (HNSW) SQL query	55
Figure 3.10: Supabase Cosine Similarity SQL query	55
Figure 3.11: Backend File Structure in Environment	57
Figure 3.12: Confidence Scoring Breakdown.....	60
Figure 3.13: RandomForest Classification Pipeline from Top to Bottom	62
Figure 3.14: Frontend File Structure in Environment.....	66
Figure 3.15: User Interface File Ingestion	68
Figure 3.16: User Interface Analysis Result Cards.....	68
Figure 3.17: User Interface Analysis Result AI Narrative & Confidence Breakdown.....	69
Figure 3.18: User Interface Analysis Result Top Findings & Technique Matchings.....	69
Figure 3.19: User Interface Analysis Results Alert Review	70
Figure 3.20: User Interface Analysis Results Recent Jobs with Timestamps and Status	70
Figure 3.21: User Interface Analysis Result Remediation Steps	71
Figure 3.22: User Interface Admin Analysis Results Timeline Preview of Events.....	72

Figure 3.23: User Interface Admin Analysis Results Relationship Graph (Full Attack Chain)...	72
Figure 3.24: User Interface Collaborative Workspace with Incident Management	73
Figure 3.25: User Interface AI-Feedback with Comments and Response Rating	74
Figure 3.26: User Interface Homepage	74
Figure 3.27: User Interface Homepage Security Operations Lifecycle	75
Figure 3.28: User Interface Homepage Target Audience and Benchmarks	75
Figure 3.29: User Interface Homepage System Capabilities Explanation.....	76
Figure 3.30: User Interface About Page	76
Figure 3.31: User Interface About Page Workflow Breakdown.....	77
Figure 3.32: User Interface About Page Step-by-Step.....	77
Figure 3.33: User Interface Support Page	78
Figure 3.34: User Interface Support Page Quick Start Validation.....	78
Figure 3.35: User Interface Support Page Common Issues and Fix Paths	78
Figure 3.36: User Interface Support Page API Diagnostics	79
Figure 3.37: User Interface Support Page Reporting Formatting	79
Figure 3.38: User Interface Support Page "How To Run Diagnostics"	79
Figure 3.39: User Interface Contact Page	80
Figure 3.40: User Interface Contact Paths and Context.....	80
Figure 3.41 User Interface Contact Page Response Expectations	81
Figure 3.42: User Interface Login Page	81
Figure 4.1: Docker Dashboard (backend-1 and frontend-1).....	97
Figure 5.1: Error Code 409 for Duplicate Files	106
Figure 5.2: Supabase Log calls During Training	107

Figure 5.3: Docker CPU Usage TF-IDF Vectorization Spikes.....	116
Figure 5.4: Docker Container No Memory Leaks.	117
Figure 5.5: Supabase Storage Buckets with Pickle Files	119
Figure 6.1: User Manual Setting up the .env	135
Figure 6.2: User Manual Backend Setup with Flask	136
Figure 6.3: User Manual Frontend Setup.....	137
Figure 6.4: User Manual Onboarding and Role Setup.....	137

Chapter 1 – Introduction

1.1 Project Background

The cybersecurity threat landscape has grown increasingly complex. The World Economic Forum's Global Cybersecurity Outlook 2025 report highlights that cyberattack sophistication continues to outpace organizational defenses, with supply chain vulnerabilities, geopolitical tensions, and the rapid adoption of emerging technologies compounding risk across sectors (WEF, 2025). Organizations face not only more frequent attacks, but more coordinated ones, ransomware campaigns, advanced persistent threats (APTs), and state-sponsored intrusions now operate at scales that overwhelm traditional defensive postures.

Central to any defensive strategy is the ability to detect, analyze, and respond to threats captured in security logs. Logs generated by firewalls, authentication systems, operating systems, and network devices constitute the primary evidence trail for identifying malicious activity. However, the sheer volume of log data produced by modern IT infrastructure makes manual analysis impractical. Security Operations Center (SOC) analysts face alert fatigue, a well-documented phenomenon where the volume of alerts degrades analyst effectiveness, leading to missed detections and delayed response (Rahman et al., 2021). The National Institute of Standards and Technology (NIST) Cybersecurity Framework (CSF) 2.0 emphasizes structured incident response processes across Identify, Protect, Detect, Respond, and Recover functions, yet implementing these processes requires tooling that many small and mid-sized organizations lack (NIST, 2024).

Cyberthreat intelligence (CTI) has emerged as a critical discipline for keeping pace with evolving attacker tactics. McMillan (2013) defines CTI as “evidence-based knowledge, including context, mechanisms, indicators, implications, and actionable advice about existing or emerging menace or hazard to assets”. CTI spans four subdomains: strategic intelligence for decision–

makers, operational intelligence about specific impending attacks, tactical intelligence describing attacker TTPs (tactics, techniques, and procedures), and technical intelligence such as indicators of compromise (IoCs) consumed by automated systems (Tounsi & Rais, 2018; Rahman et al., 2021). The MITRE ATT&CK framework has become the de facto standard for cataloging adversary behaviors, providing a structured taxonomy of techniques that enables organizations to communicate about threats using a common language.

Despite the maturity of these frameworks, significant gaps remain in making CTI actionable for resource-constrained teams. Rahman et al. (2021) surveyed 64 studies on automated CTI extraction from text and found that while NLP and ML techniques, including classification, named entity recognition, dependency parsing, and topic modeling, have been applied to extract threat indicators, attack patterns, and security events from unstructured text, most existing approaches focus on data collection rather than analysis. The survey also identifies key challenges: the lack of clean labeled datasets, class imbalance in security data, and insufficient focus on the actionability of extracted intelligence. Critically, the authors advocate for future work that bridges extracted CTI with proactive decision-making, including threat prioritization and automated threat modeling - precisely the gap AegisFA aims to address.

The convergence of several enabling technologies makes a platform like AegisFA feasible. Large language models (LLMs) such as OpenAI's GPT-4o-mini can generate contextual threat narratives, investigation guides, and remediation recommendations from analyzed log data - capabilities that did not exist at practical cost even five years ago. Machine learning classifiers trained on labeled network traffic datasets (e.g., CICIDS2017, CICIDS2019) can distinguish benign from malicious activity across multiple attack categories. Vector embedding techniques (pgvector) combined with retrieval-augmented generation (RAG) enable

semantic matching of detected threats against the MITRE ATT&CK knowledge base without requiring exact keyword matches. Together, these technologies enable a new class of security analysis tool: one that ingests raw logs, classifies and correlates events, maps findings to standardized threat frameworks, and generates human-readable intelligence, all within a single, accessible web platform.

AegisFA (Aegis Forensic Assistant) was developed in this context. Named after the mythological shield of Zeus, AegisFA is a web-based security log analysis platform designed to automate the pipeline from raw log ingestion through threat detection to actionable intelligence output. The platform targets security analysts, small SOC teams, and cybersecurity researchers who need enterprise-grade analysis capabilities without enterprise-grade complexity or cost. AegisFA combines multi-format log parsing, ML-based classification, rule-based event correlation, MITRE ATT&CK technique mapping via vector similarity search, LLM-powered insight generation, and collaborative incident management into a unified, multi-tenant architecture built on React, Flask, and Supabase (PostgreSQL)

1.2 Problem Identification

Despite advances in cybersecurity tooling, security log analysis remains a significant operational bottleneck for organizations of all sizes, but disproportionately so for small and mid-sized teams. Several interconnected problems define this gap:

Initially the volume and heterogeneity of log data, where in modern IT environments the logs get generated from diverse sources - firewalls, Windows event logs, authentication systems, syslog daemons, and application-level monitors - each with its own format, schema, and verbosity. A single enterprise endpoint can produce thousands of log entries per hour. Without automated normalization, analysts must manually interpret disparate formats before any

meaningful analysis can begin. This fragmentation delays detection and introduces inconsistency in how events are classified across sources. Moreover, alert fatigue and analyst overload is a real issue linked to this, because SOC analysts are expected to triage, investigate, and respond to continuous streams of alerts. The WEF Global Cybersecurity Outlook 2025 reports that increasing complexity in the cyber landscape has widened the gap between cyber-resilient and cyber-vulnerable organizations, with workforce shortages identified as a top challenge (WEF, 2025). Rahman et al. (2021) note that security analysts “waste a significant amount of time and effort” collecting and processing threat data manually, time that could otherwise be spent on investigation and response. When alert volume exceeds analyst capacity, critical threats are missed - not because they were undetected, but because they were buried.

Furthermore, the disconnection between detection and action is critical. Traditional SIEM platforms and log management tools focus on detection: they identify anomalies, flag rule matches, and surface alerts. However, they rarely explain why an event is significant, what attacker behavior maps to, or how to respond. The NIST CSF 2.0 Incident Response Community Profile outlines comprehensive response processes spanning preparation, detection, containment, eradication, recovery, and post-incident activity (NIST, 2024), yet most affordable tools cover only the detection phase. Analysts must independently research threats, cross-reference frameworks like MITRE ATT&CK, and draft remediation steps, a process that is slow, knowledge-dependent, and inconsistent across team members.

Another critical contributing issue is the underutilization of CTI frameworks. The MITRE ATT&CK framework provides a comprehensive taxonomy of adversary tactics and techniques, widely adopted as a standard for threat communication and gap analysis. However, mapping detected events to specific ATT&CK techniques typically requires manual lookup or

rigid keyword-based matching, both of which are error-prone and incomplete. Rahman et al. (2021) identify that while automated CTI extraction techniques exist - including NLP-based named entity recognition and ML classification - most research stops at data extraction rather than progressing to threat contextualization and actionable mapping. The opportunity to use semantic similarity (vector embeddings) for flexible, context-aware technique matching remains largely unexploited in accessible tooling.

When it comes to tooling and usage, we encounter extremely high costs and complexity barriers. Enterprise-grade platforms such as Splunk, IBM QRadar, and Microsoft Sentinel offer comprehensive log analysis and incident response capabilities, but at price points and operational complexity levels that exclude smaller organizations, academic institutions, and independent security researchers. These tools require dedicated infrastructure, licensing budgets, and specialized personnel to configure and maintain. No widely available, affordable alternative combines the full pipeline of ingestion, normalization, classification, correlation, framework mapping, AI-generated insights, and incident management in a single deployable platform.

Also, the emerging but unintegrated AI capabilities are a massive opportunity for systems like this where large language models and ML classifiers have demonstrated strong performance in security-relevant tasks: log classification, anomaly detection, threat narrative generation, and recommendation synthesis. However, these capabilities are typically available as isolated tools or API endpoints, not as integrated components of an analysis workflow. The practical challenge is not whether AI can assist in log analysis, but whether it can be embedded into a coherent pipeline that takes raw logs as input and produces prioritized, contextualized, actionable intelligence as output.

AegisFA addresses these problems by unifying six capabilities into a single platform: (1) multi-format log ingestion and normalization, (2) ML-based event classification, (3) LLM-powered threat analysis with severity scoring, (4) rule-based event correlation, (5) MITRE ATT&CK mapping via vector similarity search, and (6) AI-generated investigation guides and remediation plans, all within a multi-tenant, role-based web application accessible to analysts without enterprise budgets or infrastructure.

1.3 Scope Statement

AegisFA is a web-based security log analysis platform that automates the pipeline from raw log ingestion through threat detection to actionable intelligence. The scope of this project encompasses the design, development, and evaluation of the following capabilities:

In Scope:

1.3.1 Log Ingestion and Normalization

1. File-based upload supporting four formats: CSV, JSON, NDJSON, and plain text.
2. Chunked streaming upload for files up to 110MB (4MB chunks with session management).
3. Auto-detection of log source types: Windows Event, Firewall, Authentication, Syslog, and Custom.
4. Normalization of raw log entries into standardized event structures regardless of source format.

1.3.2 Threat Analysis and Classification

1. ML-based log classification using RandomForest model trained on the CICIDS2017 and

CICIDS2019 network traffic dataset, supporting 25+ attack categories.

2. LLM-powered threat analysis using OpenAI GPT-4o-mini for event-level threat detection.
3. Severity scoring across four levels: critical, high, medium, low.
4. Classification of events into 10 categories: authentication, authorization, network, system, application, database, security, performance, error, and anomaly.

1.3.3 Correlation and Framework Mapping

1. Rule-based correlation engine with configurable detection rules (regex matching, numeric comparisons, event grouping).
2. MITRE ATT&CK technique mapping via RAG pipeline using pgvector embeddings (1536-dimensional) with HNSW indexing for semantic similarity search.
3. Confidence scoring for technique matches.

1.3.4 AI-Generated Intelligence

1. Threat insight generation from analyzed log data.
2. Incident narrative summarization.
3. Investigation guides per detected threat.
4. Remediation plan generation with actionable steps.
5. User feedback mechanism for rating AI output quality.

1.3.5 Incident Management and Collaboration

1. Incident creation, tracking, and status management.
2. Task assignment to team members.

3. Collaborative workspace interface.
4. Event-to-incident linking

1.3.6 Multi-Tenancy and Access Control

1. Organization-based data isolation via PostgreSQL Row-Level Security (RLS)
2. Role-based access control with three roles: admin, analysts, viewer.
3. GitHub OAuth authentication through Supabase Auth.
4. Protected routes with role-based UI rendering.

1.3.7 Platform Architecture

1. React 18 frontend with Vite build tooling.
2. Flask backend with RESTful API.
3. Supabase (PostgreSQL) database with pgvector extension.
4. Docker-containerized deployment (frontend, backend, database).
5. Background job processing for long-running analyses.

1.3.8 Out of Scope:

1. Real-time log streaming or agent-based collection - AegisFA processes uploaded files, not live data feeds.
2. Full Security Orchestration, Automation, and Response (SOAR) - the platform generates recommendations but does not execute automated containment or remediation actions.
3. Endpoint detection and response (EDR) agent deployment.
4. Horizontal scaling or load balancing for production-scale multi-node deployment.
5. Integration with third-party SIEM platforms or ticketing systems (e.g., ServiceNow, Jira).

6. Mobile application development.
7. Compliance-specific reporting (PCI DSS, HIPPA, SOC 2) - through MITRE ATT&CK mapping supports compliance-adjacent workflows.

1.3.9 Target Users:

1. Security analysts in small to mid-sized SOC teams.
2. Cybersecurity researchers and students analyzing log datasets.
3. IT administrators responsible for security monitoring without dedicated SOC staff.
4. Organizations seeking affordable alternatives to enterprise SIEM platforms.

1.4 Project Objectives

The primary objective of AegisFA is to design and develop a web-based platform that automates security log analysis and delivers actionable threat intelligence to resource constrained security teams. This objective decomposes into six core objectives, each addressing a specific problem identified in Section 1.2:

1.4.1 Objective 1: Automate multi-format log ingestion and normalization

Eliminate manual log preprocessing by building an ingestion pipeline that accepts CSV, JSON, NDJSON, and plain text formats, auto-detects log source types, and normalizes entries into a consistent event schema, enabling analysis regardless of source heterogeneity.

1.4.2 Objective 2: Classify and detect threats using machine learning and large language models.

Reduce analyst's workload by automating event classification across 10 security categories and 25+ attack types using a RandomForest classifier (trained on CICIDS2017 and

CICIDS2019 and LLM-based threat detection (GPT-4o-mini), with severity scoring at four levels.

1.4.3 Objective 3: Correlate events and map threats to the MITRE ATT&CK framework

Bridge the gap between raw detections and standardized threat language by implementing a rule-based correlation engine and a RAG pipeline that semantically matches detected threats to ATT&CK techniques via vector similarity search.

1.4.4 Objective 4: Generate actionable intelligence through AI.

Address the detection-action disconnect by producing LLM-generated threat insights, incident narratives, investigation guides, and remediation plans, transforming raw analysis output into guidance analysts can act on immediately.

1.4.5 Objective 5: Enable collaborative incident response.

Support team-based security operations by providing incident tracking, task assignment, event-to-incident linking, and a shared workspace interface aligned with NIST CSF 2.0 response process expectations.

1.4.6 Objective 6: Deliver an accessible, secure, multi-tenant platform.

Lower the cost and complexity barrier by packaging all capabilities into a Docker-deployable web application with organization-based data isolation (RLS), role-based access control, and GitHub OAuth authentication, requiring no enterprise infrastructure.

1.4.7 Subgoals:

Each objective breaks into measurable deliverables used to evaluate project completion in

Chapter 5:

Under Objective 1 - Ingestion & Normalization:

1. Functional file upload endpoint supporting four formats with validation.
2. Chunked streaming upload with session management for files up to 110MB/2Gb.
3. Auto-detection of five log source types (Windows, Firewall, Auth, Syslog, Custom).
4. Batch insertion of normalized events into PostgreSQL.

Under Objective 2 - Classification & Detection:

1. Trained RandomForest model with tracked metrics (accuracy, precision, recall, F-1 score).
2. Model versioning and artifact storage in Supabase.
3. LLM-based threat analysis with provider support (OpenAI).
4. Event classification into 10 categories with four-tier severity scoring (critical, high, medium, low).
5. Background job processing for long-running analysis tasks.

Under Objective 3 - Correlation & MITRE Mapping:

1. Configurable correlation rules with regex and numeric comparison operators.
2. Detection creation from correlated event groups.
3. 1536-dimensional vector embeddings for MITRE ATT&CK techniques.
4. HNSW-indexed similarity search with confidence scoring.
5. Junction table linking analysis results to matched techniques.

Under Objective 4 - AI Intelligence:

1. Threat insight generation per analyzed file.
2. Incident narrative investigation guide output.

3. Per-threat investigation guide output.
4. Remediation plan generation with actionable steps.
5. Feedback collection mechanism (rating + suggestion) for output quality tracking.

Under Objective 5 - Incident Response:

1. CRUD operations for incidents with status tracking.
2. Task creation and assignment to team members.
3. Event-to-incident linking via junction table.
4. Workspace interface for collaborative triage.

Under Objective 6 - Platform & Security:

1. React frontend with protected routes and role-based rendering (admin, analyst, viewer).
2. Flask REST API with structured logging, request ID tracking, and retry logic.
3. RLS policies enforcing organization-scoped data access on all critical tables.
4. GitHub OAuth flow via Supabase Auth.
5. Docker containers for frontend, backend, and database services.
6. Environment-based configuration for deployment flexibility.

The reason for the structure of the subgoals is to allow reviewers to trace the Problem (1.2), the Objective (1.4), the Subgoal (1.5), the Implementation (Ch4), and the Test (Ch5). This way each subgoal becomes a testable acceptance criterion in Chapter 5 and nothing floats unanchored.

1.5 Significance

AegisFA's significance spans four dimensions: operational, educational, technical, and economic.

1.5.1 Operational: Reducing detection-to-response time.

Speed determines cost. IBM's Cost of a Data Breach Report 2025 found that the global average cost of a data breach reached \$4.88 million, with breaches taking an average of 258 days to identify and contain (IBM Security, 2025a). Organizations that extensively deployed security AI and automation reduced breach costs by an average of \$2.22 million compared to those that did not and shortened the breach lifecycle by approximately 100 days (IBM Security, 2025b).

These figures quantify the operational value of automation in security workflows. AegisFA compresses what traditionally requires multiple tools and manual steps, such as log collection, normalization, classification, threat lookup, and report writing into a single automated pipeline. By generating investigation guides and remediation plans alongside detections, the platform reduces cognitive load during triage. Threats arrive pre-contextualized with MITRE ATT&CK mappings, severity scores, and actionable next steps. For small SOC teams where one analyst may handle the entire response lifecycle, this compression directly targets the detection-to-containment window as well as mean time to detect (MTTD) and mean time to respond (MTTR), which IBM identifies as the primary cost driver.

1.5.2 Educational: Lowering the barrier to security analysis.

Cybersecurity education often suffers from a gap between theoretical knowledge (frameworks, taxonomies, attack models) and practical application (analyzing real logs, triaging, real alerts). The WEF Global Cybersecurity Outlook 2025 identifies workforce shortages as a top challenge compounding cyber risk globally (WEF, 2025). AegisFA bridges the theory-practice gap by providing a hands-on environment where students and researchers can upload log datasets, observe how events are classified, see how correlation rules fire, and examine how threats map to ATT&CK techniques, including AI-generated explanations at each stage. The

NIST CSF 2.0 Incident Response Community Profile outlines structured response processes across preparation, detection, containment, eradication, recovery, and post-incident activity (NIST, 2024); AegisFA makes these stages tangible rather than abstract, functioning as both a working tool and a teaching instrument.

1.5.3 Technical: Demonstrating integrated AI in security operations.

While individual AI capabilities, such as LLM text generation, ML classification, vector similarity search, are already well-established, their integration into a cohesive security analysis pipeline remains underexplored in accessible tooling. IBM's AI-focused analysis found that organizations deploying AI across prevention, detection, investigation, and response workflows, rather than in isolated stages, have achieved the greatest cost reductions (IBM Security, 2025b). AegisFA demonstrates this integrated approach: RandomForest classifiers handle structured pattern recognition on network traffic features, LLMs handle unstructured threat analysis and natural language generation, and RAG with pgvector embeddings handles semantic framework mapping. This layered architecture, which involves using each technology where it performs best, offers a replicable pattern for future security tool development. Rahman et al. (2021) explicitly call for research that moves beyond CTI extraction toward actionable threat intelligence; AegisFA provides a working implementation of that progression.

1.5.4 Economic: Democratizing enterprise capabilities.

The cost disparity in cybersecurity tooling creates a two-tier system. IBM reports that organizations with lower security staffing levels and less mature tooling face breach costs well above the global average, while those with automated security workflows absorb significantly less financial impact (IBM Security, 2025a). Enterprise SIEM platforms (Splunk, QRadar,

Sentinel) charge per data volume ingested, require dedicated infrastructure, and demand specialized personnel for configuration and maintenance. Organizations that cannot absorb these costs, such as small businesses, nonprofits, academic labs, and startups are left with manual processes or fragmented open-source tools covering only portions of the analysis pipeline. AegisFA packages ingestion, analysis, correlation, MITRE mapping, AI insights, and incident management into a Docker-deployable application using open-source and low-cost components (React, Flask, PostgreSQL, Supabase free tier, OpenAI API pay-per-use). The platform does not replace enterprise SIEMs for large-scale deployments, but it fills the gap for organizations where the alternative is no automated analysis at all, which is precisely the population IBM's data shows is most vulnerable.

1.6 Chapter Summary

This chapter established the foundation for AegisFA, a web-based security log analysis platform designed to automate threat detection and deliver actionable intelligence to resource-constrained security teams.

The cybersecurity landscape demands faster, smarter defensive operations. Attack sophistication continues to outpace organizational defenses (WEF, 2025), while the average data breach costs \$4.88 million and takes 258 days to identify and contain (IBM Security, 2025a). Organizations that deploy AI and automation across their security workflows reduce breach costs by \$2.22 million and shorten the breach lifecycle by approximately 100 days (IBM Security, 2025b), yet most affordable tools offer only partial coverage of the analysis pipeline, and the majority of automated CTI research stops at data extraction rather than progressing to actionable intelligence (Rahman et al., 2021).

Six problems define the gap AegisFA addresses: (1) the volume and format heterogeneity

of security logs from diverse sources, (2) analysts alert fatigue driven by workforce shortages and data overload, (3) the disconnect between threat detection and actionable response guidance, (4) underutilization of standardized frameworks like MITRE ATT&CK due to rigid matching approaches, (5) prohibitive cost and complexity of enterprise SIEM platforms for small organizations, and (6) the availability of powerful AI capabilities, such as LLMs, ML classifiers, vector search, that remain unintegrated into cohesive security workflows.

AegisFA responds with six corresponding capabilities unified in a single platform: multi-format log ingestion and normalization across CSV, JSON, NDJSON, and plain text from Windows, Firewall, Authentication, Syslog, and Custom sources; ML-based event classification using a RandomForest model trained on the CICIDS2017 and CICIDS2019 dataset, and LLM-powered threat analysis via OpenAI GPT-4o-mini; rule-based event correlation with configurable detection rules; MITRE ATT&CK technique mapping through a RAG pipeline using pgvector embeddings with HNSW-indexed similarity search; AI-generated threat insights, incident narratives, investigation guides, and remediation plans; and collaborative incident management with task assignment, integrated all within a multi-tenant architecture enforcing organization-scoped data isolation through PostgreSQL Row-Level Security and role-based access control across admin, analyst, and viewer roles.

The project scope bounds AegisFA to file-based batch analysis rather than real-time streaming, excludes automated remediation execution (SOAR), and targets deployment via Docker containers on the React/Flask/Supabase stack. Target users include SOC analysts in small to mid-sized teams, cybersecurity researchers and students, and IT administrators responsible for security monitoring without dedicated SOC staff.

The significance of this work is fourfold. Operationally, AegisFA compresses the

detection-to-response timeline that IBM identifies as the primary breach cost driver.

Educationally, it provides a hands-on environment that makes the incident response lifecycle defined by NIST CSF 2.0 tangible for learners. Technically, it demonstrates an integrated AI architecture, as well as combining ML classification, LLM generation, and vector-based semantic search, aligned with capabilities previously gated behind enterprise pricing, serving the under-tooled organizations that IBM's data shows are most financially vulnerable to breaches.

The remainder of this thesis is organized as follows:

- Chapter 2 - Requirements & Feasibility defines functional and non-functional requirements, evaluates technical and scheduling feasibility, identifies beneficiaries, and establishes success criteria against which the project is measured.
- Chapter 3 - System Design presents the three-tier architecture (React frontend, Flask backend pipeline design (ingestion, normalization, classification, correlation, RAG, NLG), frontend design (authentication flows, dashboard, workspace, admin panel), API structure, and security architecture.
- Chapter 4 - Implementation details the development of each system component: log ingestion and parsing, threat analysis and classification, correlation and detection, MITRE ATT&CK mapping, AI-powered insight generation, ML training pipeline, frontend interfaces, and Docker-containerized deployment.
- Chapter 5 - Testing & Evaluation covers functional testing against requirements, security validation of RLS and authentication flows, performance evaluation, ML model metrics (accuracy, precision, recall, F1-score), and user acceptance assessment.
- Chapter 6 - Conclusion summarizes achievements against stated objectives, acknowledges limitations, and proposes future work including real-time streaming

ingestion, SOAR integration, horizontal scaling, and third-party platform connectors.

Chapter 2 – Requirements & Feasibility

2.1 Methodology

AegisFA followed an iterative, incremental development methodology adapted from Agile principles. The project was developed by a single developer, making formal Scrum ceremonies impractical; instead, development was organized into feature-driven iterations, each targeting a vertical slice of functionality; from database schema through backend service to frontend interface.

Each iteration followed a consistent cycle:

1. Requirements definition - identify the next capability to implement, derive functional requirements from the problem analysis (Section 1.2) and NIST CSF 2.0 incident response process expectations (NIST, 2024).
2. Design - define database tables, API endpoints, and UI flows needed to support the capability.
3. Implementation - build backend services (Flask), database migrations (PostgreSQL/Supabase), and frontend components (React) in parallel where possible.
4. Testing - validate functionality against requirements, verify RLS enforcement, and confirm API behavior.
5. Integration - merge the capability into the running platform, resolve conflicts with existing features, and regression-test.

This approach was chosen for three reasons. First, AegisFA's pipeline architecture (ingestion → normalization → classification → correlation → mapping → insights) naturally decomposes into sequential stages, each buildable and testable independently. Second, iterative delivery allowed early validation of architectural decisions (e.g., Supabase RLS for multi-tenancy, pgvector for MITRE mapping) before committing downstream features to those

foundations. Third, as a solo project, iterative development provided natural checkpoints for reassessing scope and priorities without the overhead of formal sprint planning.

Key development iterations included:

- Iteration 1: Database schema design, Supabase project setup, RLS policy implementation.
- Iteration 2: Log ingestion pipeline (file upload, parsing, normalization, batch insertion).
- Iteration 3: Threat analysis engine (LLM integration, severity scoring, event classification).
- Iteration 4: Correlation engine and detection rule framework.
- Iteration 5: MITRE ATT&CK RAG pipeline (vector embeddings, similarity search).
- Iteration 6: ML classifier (RandomForest training on CICIDS2017 and CICIDS2019, model versioning).
- Iteration 7: AI-generated insights (narratives, investigation guides, remediation plans).
- Iteration 8: Incident management, task assignment, collaborative workspace.
- Iteration 9: Frontend interfaces (dashboard, admin panel, workspace, authentication flows).
- Iteration 10: Chunked upload system, background job processing, performance optimization.

2.2 Functional Requirements

Functional requirements are organized by subsystem, each traceable to a project objective from Section 1.4. The functional requirements were adjusted and expanded, and they may differ from their initial form.

Requirements are labeled FR-XX for reference in testing (Chapter 5).

2.2.1 Log Ingestion & Normalization (Objective 1)

ID	Requirement	Implementation Detail
FR-01	System shall accept file uploads in CSV, JSON, NDJSON, and plain text formats.	<i>file_parser.py</i> – format-specific parsing with auto-detection.
FR-02	System shall support chunked streaming upload for large files.	Upload session management: <i>init</i> → <i>upload-part</i> → <i>complete</i> endpoints; 16MB per part, 2GB total assembly limit.
FR-03	System shall auto-detect log source type from uploaded content.	Five source types recognized: Windows, Firewall, Auth, Syslog, Custom.
FR-04	System shall normalize raw log entries into a standardized event schema.	<i>normalization.py</i> – rule-based normalization with optional AI-enhanced normalization (<i>USE_AI_NORMALIZATION</i> flag)
FR-05	System shall batch-insert normalized events into the database.	<i>RAW_LOG_INSERT_BATCH_SIZE</i> = 200 rows per insert operation.
FR-06	System shall enforce file size limits.	Flask <i>MAX_CONTENT_LENGTH</i> = 120MB; per-part limit configurable via environment variable.
FR-07	System shall track upload session lifecycle.	Sessions expire after 3600 seconds TTL; maximum 100 concurrent sessions; HTTP

		429 returned when limit reached.
--	--	----------------------------------

Table 2.1: Structure of Functional Requirements for Log Ingestion & Normalization.

2.2.2 Threat Analysis & Classification (Objective 2)

ID	Requirement	Implementation Detail
FR–08	System shall classify log events into security categories.	10 categories: authentication, authorization, network, system, application, database, security, performance, error, anomaly.
FR–09	System shall assign severity levels to classified events.	Four-tier scoring: critical, high, medium, low.
FR–10	System shall support ML-based classification using a trained RandomForest model.	<i>log_classifier.py</i> – 150 estimators, max depth 25, TF-IDF features (800 max, bigram), sigmoid calibration.
FR–11	System shall support LLM-based threat analysis	OpenAI GPT-4o-mini via <i>INSIGHTS_LLM_PROVIDER</i> config.
FR–12	System shall train ML models on labeled datasets with tracked metrics.	CICIDS2017/CICIDS2019 datasets; 70/15/15 stratified split; metrics: accuracy, precision, recall, F1 (macro + weighted), confusion matrix.
FR–	System shall version and persist trained	Pickle artifacts stored locally and in Supabase

13	models.	<i>ml-models</i> bucket; model activation gated on ≥ 0.80 weighted precision.
FR– 14	System shall process analysis as background jobs.	<i>analysis_jobs</i> table with status tracking; pollable via <i>GET /analysis-jobs/<job_id></i>
FR– 15	System shall provide confidence scores for classifications.	Top 3 predictions with probabilities; high-confidence threshold at 0.7

Table 2.2: Structure of Functional Requirements for Threat Analysis and Classification.

2.2.3 Correlation & MITRE ATT&CK Mapping (Objective 3)

ID	Requirement	Implementation Detail
FR– 16	System shall evaluate configurable correlation rules against normalized events.	<i>correlation_engine.py</i> – regex matching, numeric comparisons (>, <, >=, <=, eq, neq, in, contains), event grouping.
FR– 17	System shall create detections when correlation rules match.	Detection records linked to triggering events and source rules.
FR– 18	System shall map detected threats to MITRE ATT&CK techniques via semantic similarity.	<i>rag_service.py</i> - 1536-dimensional embeddings, HNSW-indexed pgvector search on <i>mitre_techniques</i> table.
FR– 19	System shall provide confidence scores for MITRE techniques matches.	Cosine similarity scoring: results linked via <i>analysis_result_mitre_links</i> junction table.

Table 2.3: Structure of Functional Requirements for Correlation and MITRE ATT&CK Mapping

2.2.4 AI-Generated Intelligence (Objective 4)

ID	Requirement	Implementation Detail
FR– 20	System shall generate threat insights from analyzed log data.	<i>insights_generator.py</i> – LLM-powered insight synthesis per analyzed file.
FR– 21	System shall generate incident narrative summaries.	Stored in <i>summaries</i> table, linked to incidents.
FR– 22	System shall generate investigation guides per detected threat.	Step-by-step investigation procedures output by LLM.
FR– 23	System shall generate remediation plans with actionable steps.	Viewable at <i>/remediation-view/:remediationId</i>
FR– 24	System shall collect user feedback on AI-generated output	<i>POST /feedback</i> - type, content, rating; <i>GET /feedback</i> filtered by <i>org</i> .

Table 2.4: Structure of Functional Requirements for AI-Generated Intelligence.

2.2.5 Incident Management & Collaboration (Objective 5)

ID	Requirement	Implementation Detail
FR– 25	System shall support incident creation, update, and tracking	CRUD via <i>/incidents</i> endpoints; fields: title, description, severity, status.

FR– 26	System shall support task creation and assignment to team members	CRUD via <i>/tasks</i> endpoints; assignable with status and priority tracking.
FR– 27	System shall link events to incidents	<i>incident_events</i> junction table
FR– 28	System shall provide a collaborative workspace interface	<i>/workspace</i> route – protected, accessible to all authenticated roles.

Table 2.5: Structure of Functional Requirements for Incident Management and Collaboration.

2.2.6 Multi-Tenancy & Access Control (Objective 6)

ID	Requirements	Implementation Details
FR– 29	System shall authenticate users via GitHub OAuth.	Supabase Auth; Bearer token validation on all non-public endpoints.
FR– 30	System shall enforce role-based access control.	Three roles: admin, analyst, viewer; <i>_require_roles()</i> decorator; admin-only routes: <i>/rf/train</i> , <i>/rf/load-latest</i> , <i>/admin</i>
FR– 31	System shall isolate data by organization.	RLS policies on 17 tables; <i>_enforce_org_scope()</i> and <i>_enforce_file_scope()</i> validation at API layer.
FR– 32	System shall auto-provision new users on first login.	User record created from auth metadata; default role: viewer; <i>org_id</i> claimed from

		metadata or request context.
FR– 33	System shall redirect unauthenticated users to login.	<i>ProtectedRoute</i> component checks session; redirects to <i>/login</i> ; <i>authenticatedFetch</i> handles 401 with redirect to <i>/login?reason=expired</i> .

Table 2.6: Structure of Functional Requirements for Multi-Tenancy and Access Control.

2.3 Non-Functional Requirements

Non-functional requirements define quality attributes. The non-functional requirements were adjusted and expanded, and they may differ from their initial form. Labeled NFR-XX for traceability.

2.3.1 Performance

ID	Requirement	Target	Implementation Evidence
NFR– 01	API response time for non-analysis endpoints	< 2 seconds	Request timing logged via <i>perf_counter()</i> ; <i>elapsed_ms</i> in all response logs
NFR– 02	File parsing throughput	Up to 200,000 rows per background job.	<i>BACKGROUND_PARSE_MAX_ROWS = 200000</i>
NFR–	Database operation	Retry with	4 attempts, 0.8s base delay, capped at 4.0s

03	resilience	exponential backoff.	per retry
NFR– 04	Upload session concurrency	100 simultaneous sessions	<i>MAX_UPLOAD_SESSIONS = 100;</i> HTTP 429 on overflow
NFR– 05	ML inference latency	Batch classification with confidence scoring.	TF-IDF vectorization + RandomForest predict in-memory; top 3 predictions returned.

Table 2.7: Structure of Non-Functional Requirements for Performance.

2.3.2 Security

ID	Requirement	Implementation Evidence
NFR– 06	All sensitive endpoints require authentication.	<i>@main.before_request</i> middleware validates Bearer token on every request except <i>GET /</i>
NFR– 07	Cross-organization data access prevented	Dual enforcement: PostgreSQL RLS policies + application-layer <i>_enforce_org_scope()</i>
NFR– 08	Role escalation prevented	Role validated from database record, not client-supplied; only {"admin", "analyst", "viewer"} accepted
NFR– 09	Error responses must not leak internal state	Structured <i>_error_response()</i> with controlled fields: code, message, retrieable, request_id; no

		stack traces in production
NFR– 10	Request traceability	<i>X-Request-ID</i> header propagated or generated; included in all log entries and error responses

Table 2.8: Structure of Non-Functional Requirements for Security.

2.3.2 Reliability

ID	Requirement	Implementation Evidence
NFR– 11	Storage operations must retry on transient failure.	<i>storage.py</i> – 3 attempts, 0.6s base delay, 3.0s cap
NFR– 12	Database operations must retry on transient failure.	<i>routes.py</i> – 4 attempts, 0.8s base delay, 4.0s cap
NFR– 13	ML model activation gated on quality threshold	Model auto-archived if weighted precision < 0.80; only active models served
NFR– 14	Chunked uploads must validate part contiguity	Assembly verifies parts 1 through N exists before concatenation; HTTP 409 on gaps

Table 2.9: Structure of Non-Functional Requirements for Reliability.

2.3.4 Scalability & Maintainability

ID	Requirement	Implementation Evidence
NFR–	Configuration via environment	All operational constants (retry count, file

15	variables	limits, model providers, log levels) overridable via env vars
NFR– 16	Structured logging for operational visibility	<i>structlog</i> with JSON rendering, ISO timestamps, log level filtering, request context binding
NFR– 17	Containerized deployment	Docker images for frontend (Node 20-alpine), backend (Python 3.11-bullseye), with docker-compose orchestration
NFR– 18	Modular backend architecture	Blueprint-based Flask app; separated services: parser, classifier, normalizer, correlation, RAG, insights, timeline, storage

Table 2.10: Structure of Non-Functional Requirements for Scalability & Maintainability.

2.3.5 Usability

ID	Requirement	Implementation Evidence
NFR– 19	Intuitive navigation with role-appropriate interfaces	Admin panel restricted to admin role; dashboard and workspace accessible to all authenticated users
NFR– 20	Real-time feedback during long operations	SSE streaming for <i>/upload/stream</i> ; pollable job status for background analysis

NFR– 21	Persistent client state across sessions	<i>usePersistentState</i> hook using <code>localStorage</code>
------------	---	--

Table 2.11: Structure of Non-Functional Requirements for Usability.

2.4 Project Feasibility

2.4.1 Technical Feasibility

AegisFA’s architecture relies on mature, well-documented technologies. Each component was evaluated for technical risk before adoption:

Component	Technology	Feasibility Rationale
Frontend	React 18, Vite 5	Widely adopted; extensive ecosystem; fast development cycle with HMR
Backend	Flask 3.1, Python 3.11	Strong ML/NLP library support (scikit-learn, transformers, pandas); lightweight framework suitable for API-first design
Database	Supabase (PostgreSQL)	Managed service eliminates DBA overhead; native RLS for multi-tenancy; pgvector extension for vector search without separate infrastructure
Authentication	Supabase Auth (GitHub OAuth)	Eliminates custom auth implementation; JWT-based tokens; built-in session management
ML	scikit-learn	Well-understood algorithm; interpretable; trainable on

Classification	RandomForest	CICIDS2017 & CICIDS2019 data size without GPU
LLM Integration	OpenAI GPT-4o-mini	API-based; no local GPU required for primary provider
Vector Search	pgvector + HNSW	Runs inside existing PostgreSQL instance; no separate vector database infrastructure
Deployment	Docker	Reproducible environments; dev-to-production parity; devcontainer support for onboarding

Table 2.12: Structure of Project's Technical Feasibility.

The primary technical risk was integrating multiple AI/ML systems (RandomForest, LLM threat analysis, RAG pipeline) into a single coherent pipeline. This was mitigated by designing the analysis pipeline (*analysis_pipeline.py*) as an orchestrator that sequences independent services such as classification, detection, and insight generation, all of which are callable and testable in isolation.

2.4.2 Scheduling Feasibility

Beneficiary	Benefit
Small SOC teams	Enterprise-grade analysis pipeline without enterprise cost; automated triage reduces MTTD/MTTR – IBM reports organizations using security AI save \$2.22M per breach on average (IBM Security, 2025b)

Cybersecurity students and researchers	Hands-on environment for learning log analysis workflows, MITRE ATT&CK mapping, and ML-based classification; AI-generated explanations serve as educational scaffolding
IT administrators without dedicated security staff	Unified platform replaces manual multi-tool workflows; actionable remediation plans reduce dependency on specialized security knowledge
Organizations with compliance-adjacent needs	MITRE ATT&CK mapping provides standardized threat documentation; incident tracking creates audit trail; structured logging supports forensic review

Table 2.13: Structure of Project's Scheduling Feasibility.

2.5 Success Criteria

Success is evaluated across four dimensions, each measurable against specific deliverables:

Criterion	Measure	Threshold
Functional completeness	All 33 functional requirements (FR-01 through FR-33) implemented and testable	100% of FRs demonstrable
ML model performance	RandomForest classifier trained on CICIDS2017 & CICIDS2019 with tracked evaluation metrics	Weighted precision ≥ 0.80 (model activation threshold)

Security enforcement	RLS policies active on all 17 critical tables; role-based access control enforced on all protected endpoints	Zero cross-org data leakage in testing; unauthorized role access returns HTTP 403
Pipeline Integration	End-to-end flow from file upload through analysis to AI-generated insights completes without manual intervention	Automated pipeline produces classification, detections, MITRE mappings, and insights for a test log file
Deployment readiness	Platform deployable via Docker containers with environment-based configuration	<i>docker-compose up</i> produces running frontend, backend, and database services

Table 2.14: Structure of Project's Success Criteria

2.6 Team Limitations

AegisFA was developed as a solo capstone project. This constraint shaped several architectural and scope decisions:

- No dedicated QA role. Testing was performed by the developer, limiting the diversity of test scenarios and reducing coverage of edge cases. Automated testing was prioritized where feasible, but comprehensive end-to-end test suites were not developed.
- No dedicated DevOps role. Deployment was containerized via Docker to minimize infrastructure management overhead. Production-grade concerns (horizontal scaling, load balancing, zero-downtime deployment) were scoped out.
- No UX design specialist. Frontend interfaces prioritized functionality over visual polish.

Component layouts follow React conventions and Supabase Auth UI defaults rather than custom design systems.

- No security auditor. While RLS policies and RBAC were implemented and tested, no independent penetration testing or formal security audit was performed. Security measures reflect best-effort implementation by a single developer.
- Time-bound to one semester. Features were prioritized by pipeline criticality: ingestion and analysis received the most development time; collaborative features (workspace, task management) received less refinement.

These limitations are acknowledged as constraints on the evaluation (Chapter 5) and inform future work recommendations (Chapter 6).

Chapter 3 – System Design

3.1 System Architecture Overview

AegisFA follows a three-tier web application architecture: a React-based frontend, a Flask-based backend API, and a Supabase-managed PostgreSQL database. The three tiers communicate over HTTP/HTTPS and are containerized independently, enabling environment parity between development and deployment.

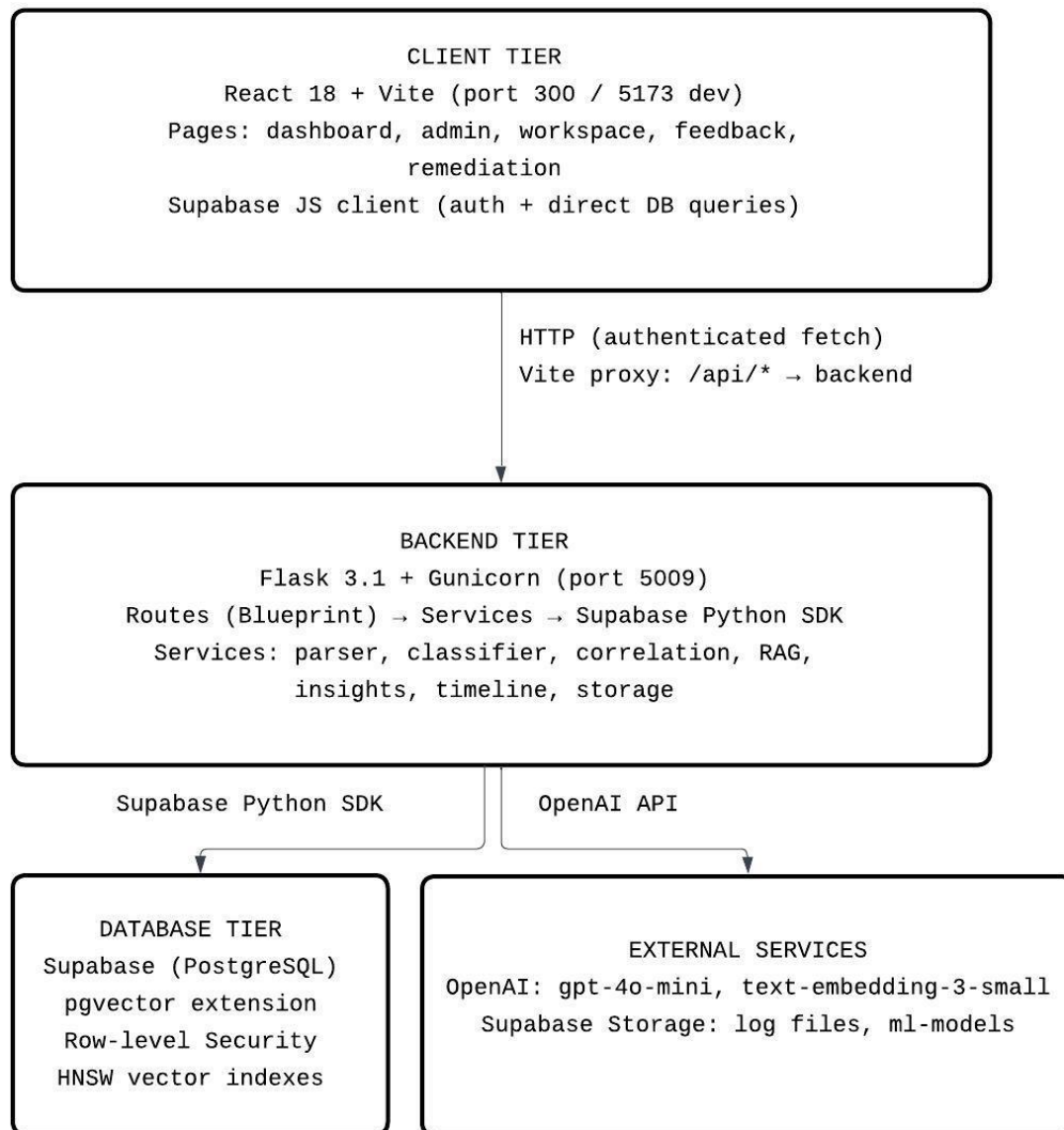


Figure 3.1: Architecture Diagram of Proposed System

Communication protocols:

- Frontend to Backend: HTTP REST via Vite dev proxy (/api/* → <http://localhost:5009>, prefix stripped). In production containers, VITE_API_BASE_URL configures the target directly.
- Frontend to Supabase: Supabase JS client over HTTPS for authentication session management and direct read queries where RLS enforcement is sufficient.
- Backend to Supabase: Supabase Python SDK using the service role key, bypassing RLS for privileged write operations.
- Backend to OpenAI: HTTPS REST via *openai* SDK for LLM completions and text embeddings.
- Backend to Supabase Storage: S3-Compatible API for binary file uploads and model artifact retrieval.

Container architecture (.devcontainer/docker-compose.yml):

- *frontend*: Node 20-alpine3.21, Vite build, exposed on port 3000, depends on backend.
- *backend*: Python 3.11-bullseye, Gunicorn WSGI server, exposed on port 5001 (mapped from internal 5009).
- Services communicate over a shared *app-network* bridge network.
- All credential injected via environment variables are runtime.

This architecture separates concerns cleanly: the frontend handles presentation and user interaction; the backend owns all analysis logic, authentication validation, and database orchestration; the database tier enforces data isolation through RLS regardless of how queries arrive.

3.2 Database Design

3.2.1 Schema Overview

The AegisFA database comprises 22 tables across 8 migration files (*000_db_implementation.sql* through *007_rf_training.sql*). Tables are grouped by functional domain:

Domain	Tables
Multi-tenancy & Users	<i>organizations, users, user_permissions</i>
Log Ingestion	<i>log_sources, raw_logs, normalized_events, log_files</i>
Threat Analysis	<i>analysis_results, mitre_techniques,</i> <i>analysis_result_mitre_links</i>
Correlation & Detection	<i>correlation_rules, detections, correlation_errors</i>
Background Processing	<i>analysis_jobs, analysis_job_items</i>
Incident Response	<i>incidents, incident_events, tasks, summaries, feedback</i>
ML Training	<i>model_versions, training_runs</i>

Table 3.1: Supabase Database Migration Schema for the Proposed System

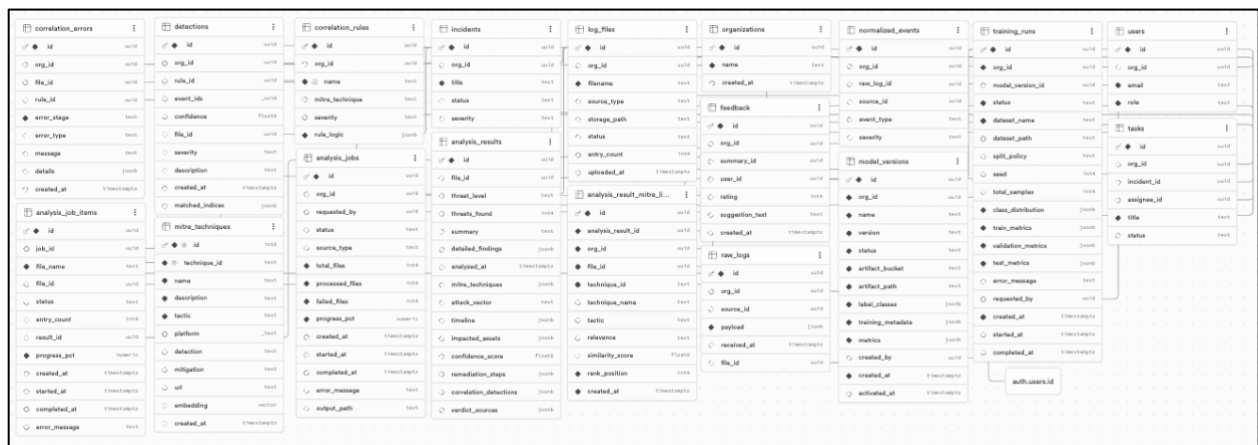


Figure 3.2: Full Supabase Database Tables

3.2.2 Core Tables

organizations – root multi-tenancy anchor:

1	id	UUID	PRIMARY KEY	DEFAULT	uuid_generate_v4()
2	name	TEXT			
3	created_at	TIMESTAMPTZ	DEFAULT	now()	

Figure 3.3: Supabase Organizations Database Table

users – extends Supabase Auth:

1	id	UUID	PRIMARY KEY	REFERENCES	auth.users(id)	ON DELETE CASCADE
2	org_id	UUID	REFERENCES	organizations(id)	ON DELETE CASCADE	
3	email	TEXT				
4	role	TEXT	CHECK	(role IN ('admin', 'analyst', 'viewer'))	DEFAULT	'viewer'

Figure 3.4: Supabase Users Database Table

Every table containing user data carries an *org_id* foreign key to *organizations*, establishing a consistent ownership chain that RLS policies enforce.

log_files – upload lifecycle tracking:

1	id	UUID	PRIMARY KEY			
2	org_id	UUID	REFERENCES	organizations(id)		
3	filename	TEXT				
4	source_type	TEXT	CHECK	(source_type IN ('windows', 'firewall', 'auth', 'syslog', 'custom'))		
5	storage_path	TEXT				
6	status	TEXT	CHECK	(status IN ('uploaded', 'analyzing', 'completed', 'failed'))		
7	entry_count	INT				
8	created_at	TIMESTAMPTZ				

Figure 3.5: Supabase Log Files Database Table

raw_logs – raw ingest storage:

1	id	UUID PRIMARY KEY
2	org_id	UUID REFERENCES organizations(id)
3	source_id	UUID REFERENCES log_sources(id) ON DELETE SET NULL
4	payload	JSONB
5	received_at	TIMESTAMPTZ DEFAULT now()

Figure 3.6: Supabase Raw Logs Database Table

The *payload* column stores the original parsed entry as JSONB, preserving full fidelity regardless of source format. Indexed on *org_id*, *received_at*, and *file_id* for efficient per-org and per-file queries.

analysis_results – analysis output:

1	id	UUID PRIMARY KEY
2	file_id	UUID REFERENCES log_files(id) ON DELETE CASCADE
3	threat_level	TEXT CHECK (threat_level IN ('none', 'low', 'medium', 'high', 'critical'))
4	threats_found	INT
5	summary	TEXT
6	detailed_findings	JSONB
7	mitre_techniques	JSONB
8	attack_vector	TEXT
9	timeline	JSONB
10	impacted_assets	JSONB
11	confidence_score	FLOAT
12	remediation_steps	JSONB
13	correlation_detections	JSONB
14	verdict_sources	JSONB
15	created_at	TIMESTAMPTZ

Figure 3.7: Supabase Analysis Results Database Table

The *verdict_sources* records a breakdown of contributing signals: *llm_findings* count, *rf_risk_score*, *llm_confidence*, *retrieval_strength*, *correlation_strength*, and *evidence_consistency_bonus* – enabling auditability of the confidence computation.

3.2.3 pgvector Integration & MITRE Technique Storage

MITRE ATT&CK technique matching uses semantic vector similarity rather than

keyword lookup. The *mitre_techniques* table stores pre-computed OpenAI embeddings alongside technique metadata:

1	id	BIGINT PRIMARY KEY
2	technique_id	TEXT UNIQUE -- e.g., T1110
3	name	TEXT
4	description	TEXT
5	tactic	TEXT
6	platform	TEXT[]
7	detection	TEXT
8	mitigation	TEXT
9	url	TEXT
10	embedding	vector(1536) -- OpenAI text-embedding-3-small output
11	created_at	TIMESTAMPTZ

Figure 3.8: Supabase MITRE Techniques Database Table

A Hierarchical Navigable Small World (HNSW) index accelerates approximate nearest-neighbor search:

```
1 CREATE INDEX mitre_techniques_embedding_idx
2 ON mitre_techniques USING hnsw (embedding vector_cosine_ops);
```

Figure 3.9: Supabase Hierarchical Navigable Small World (HNSW) SQL query

HNSW was chosen over IVFFlat for its better query-time performance at the scale of the MITRE ATT&CK dataset (~700 techniques) and its support for dynamic inserts without requiring periodic index rebuilding (Malkov & Yashunin, 2018). The cosine distance operator ($\Leftrightarrow$) computes $1 - \text{cosine_similarity}$, enabling threshold-based filtering via:

```
1 SELECT * FROM match_mitre_techniques(query_embedding, match_threshold := 0.45,
   match_count := 5);
```

Figure 3.10: Supabase Cosine Similarity SQL query

Matched techniques persisted to *analysis_result_mitre_links* with *similarity_score* and *rank_position*, creating a normalized junction table for querying which techniques were matched

for any given analysis result.

3.2.4 Row-Level Security & Multi-Tenancy

RLS policies are enabled on all 17 critical tables (*003_rls_implementation.sql*). Every policy follows the same pattern – org-scoping via the authenticated user’s *org_id*, and for tables without a direct *org_id* column (e.g., *analysis_result_mitre_links*), the policy joins through parent tables:

The backend uses the Supabase service role key, which bypasses RLS for all write operations – ensuring the analysis pipeline can insert data on behalf of any organization. The frontend uses the anon key, which is fully subject to RLS, and users can only read their own organization’s data regardless of query construction.

This dual-key design provides defense in depth: even if an API request bypasses application-layer org scope checks, the database still enforces isolation at the query level.

3.3 Backend Design

3.3.1 Application Structure

The backend is a Flask 3.1 application organized as a Blueprint (*main*) with a service layer beneath it. Route handlers delegate to service modules, keeping business logic out of the routing layer.

backend/ingestion/app/	
— <code>__init__.py</code>	Flask app factory, <code>MAX_CONTENT_LENGTH=120MB</code>
— <code>routes.py</code>	All endpoints (~2800 lines), middleware, auth
— <code>analysis_pipeline.py</code>	Pipeline orchestrator
— <code>file_parser.py</code>	Multi-format log parsing
— <code>normalization.py</code>	Rule-based event normalization
— <code>log_classifier.py</code>	RandomForest classifier service
— <code>threat_analysis.py</code>	LLM threat detection
— <code>rag_service.py</code>	MITRE RAG pipeline (main analysis entry)
— <code>correlation_engine.py</code>	Rule-based event correlation
— <code>insights_generator.py</code>	NLG for threats, incidents, remediation
— <code>timeline_service.py</code>	Event aggregation & timeline generation
— <code>storage.py</code>	Supabase Storage integration
— <code>service.py</code>	Local LLM loader
— <code>rf_training_mapping.py</code>	CICIDS2019 & CICIDS2017 label mapping & severity scoring
— <code>supabase_client.py</code>	Shared Supabase client
— <code>logging_config.py</code>	structlog JSON configuration

Figure 3.11: Backend File Structure in Environment

3.3.2 Request Middleware

Every incoming request passes through `@main.before_request`, which executes two operations before any route handler is invoked:

1. Request ID assignment: Generates or extracts `X-Request-ID` header; bind to logger context for full request traceability across all log entries produced during that request.
2. Authentication (`_authenticate_request()`), this validates the Bearer token via Supabase Auth, resolves `org_id` and `role` from the `user`'s tables, auto-provisions new users on first login, and stores the resolved identity in Flask's `g` context (`g.auth_user_id`, `g.auth_role`).

The only exempt endpoint is `GET /` (health check).

After authentication, route handlers enforce authorization via two decorators:

- `_require_roles (*allowed_roles)`: This validates `g.auth_role` against the allowed set; returns HTTP 403 if unauthorized.
- `_enforce_org_scope (request_org_id)`: This verifies the requested org matches the

authenticated org, and prevents cross-tenant data access at the API layer as a second line of defense behind RLS.

3.3.3 Log Ingestion & Parsing Pipeline

Entry: *POST /upload* (*routes.py*, line 1234)

The ingestion pipeline accepts a multipart form upload containing the log file, source type, and organization ID. Parsing is delegated to *file_parser.py*, which:

1. Detects encoding: Attempts UTF-8, UTF-16, CP1252, Latin-1 in sequence.
2. Detects format: Probes the byte stream to identify JSON array, NDJSON (newline-delimited JSON), CSV (delimiter sniffed), or plain text.
3. Parses entries: Converts each record to a Python dictionary with normalized keys.
4. Returns metadata: entry count, detected format, any parse warnings.

Maximum file size is 110MB per upload (Flask *MAX_CONTENT_LENGTH = 120MB*).

For files exceeding this threshold, the chunked upload system (Section 3.3.9) is used instead.

Parsed entries are inserted into *raw_logs* in batches of 200 (*RAW_LOG_INSERT_BATCH_SIZE = 200*) to balance throughput against connection overhead. A *log_files* record is created with *status = 'analyzing'* before analysis begins and updated to *'completed'* or *'failed'* upon completion.

3.3.4 Normalization Engine

Rule-based normalization (*normalization.py*) maps source-specific field names to a common schema:

Source-specific mappings:

- Windows: *EventID*, *User*, *IpAddress*, *EventType*, *Status*.

- Firewall: *rule_id*, *src_ip*, *action* (status derived: allow → success, deny → failure).
- Auth: *id*, *username*, *source_ip*, action = 'login', *result*.
- Syslog / Custom: Best-effort field matching.

When *USE_AI_NORMALIZATION=true*, an LLM call augments rule-based normalization for non-standard formats, which is useful for custom source types where field names cannot be predetermined.

Normalized entries populate the *normalized_events* table for archival and cross-file querying. The primary analysis pipeline, however, operates on raw parsed entries rather than normalized record, which preserves full field fidelity for the LLM context window.

3.3.5 Threat Analysis Pipeline

The threat analysis pipeline (*rag_service.py:analyze_threats()*) orchestrates four sequential stages:

Stage 1 → LLM Threat Detection (*_detect_threats()*):

Log entries are compacted to 40 representative samples (14 fields, 100-character value truncation) and submitted to OpenAI GPT-4o-mini with a structure prompt requesting a JSON array of detected threats. Each threat object contains *threat_type*, *severity*, *description*, and *indicators*. Temperature is set to 0.2 for focused, reproducible output.

Stage 2 → MITRE ATT&CK Retrieval (*_retrieve_mitre_techniques()*):

A contextual query string is synthesized from LLM findings, correlation detections, RF classification statistics, and source type. This query is embedded using OpenAI *text-embedding-3-small* (1536 dimensions) and matched against the *mitre_techniques* table via the

match_mitre_techniques RPC function, which executes HNSW cosine similarity search with a 0.45 similarity threshold and returns the top 5 matches.

Stage 3 → Incident Summary Generation (*_generate_incident_summary()*):

A second LLM call synthesizes findings, MITRE matches, and detections into a structure incident summary containing: *summary* (narrative), *attack_vector*, *timeline* (chronological event list), *impacted_assets*, and *remediation_steps*.

Stage 4 → Confidence Scoring:

A blended confidence score aggregates four evidence signals with defined weights:

```
confidence = (0.20 × llm_confidence)
             + (0.20 × retrieval_strength)
             + (0.30 × rf_risk_score)
             + (0.30 × correlation_strength)
             + consistency_bonus (up to +0.12)
```

Figure 3.12: Confidence Scoring Breakdown

RF classification and correlation detection are weighted most heavily (30% each) as they present deterministic, rule-based signals. LLM and retrieval signals are weighted less (20% each) to account for generative model uncertainty. The consistency bonus rewards agreement across all four signal types, and the full confidence breakdown is stored in *verdict_sources* for auditability, where reviewers can inspect exactly how each signal contributed to the final threat level classification.

3.3.6 Correlation Engine

The correlation engine (*correlation_engine.py:run_correlation()*) evaluates organization-specific and default detection rules against normalized log entries before LLM analysis begins, providing rule-based signal to complement probabilistic ML and LLM outputs.

Rules are stored in the *correlation_rules* table with a *rule_logic* JSONB field defining the detection logic. Five rule types are supported:

Type	Detection Logic
<i>threshold</i>	Count entries matching filter conditions; fire if count $\geq$ threshold
<i>sequence</i>	Detect ordered event patterns within a time window
<i>distinct_value</i>	Count distinct field values after filtering; fire if count $\geq$ threshold
<i>existence</i>	Fire if any entry matches all filter conditions
<i>time_rate</i>	Fire if event rate per minute exceeds threshold

Table 3.2: Correlation Engine Rule Types

Filter conditions support nine operations: *eq*, *neq*, *in*, *contains*, *regex* (capped at 500 characters to prevent ReDoS), *exists*, *gt*, *gte*, *lt*, *lte*.

For datasets exceeding 50,000 entries, the engine processes in 50,000-row batches, adjusting *matched_indices* by batch offset to maintain accurate position references in the final output. Evaluation errors are recorded to *correlation_errors* with structured fields (*error_stage*, *error_type*, *details*) for debugging without disrupting the overall pipeline.

3.3.7 ML Classification (RandomForest)

The *LogClassifier* class (*log_classifier.py*) implements a RandomForest classifier trained on the CICIDS2019 and CICIDS2017 network traffic dataset. CICIDS2019 and CICIDS2017

were selected due to their comprehensive coverage of modern attack scenarios with 25+ categories including DoS, DDoS, brute force, web attacks, infiltration, and botnet traffic, as well as their widespread use as a benchmark in security ML research (Sharafaldin et al., 2018).

Architecture:

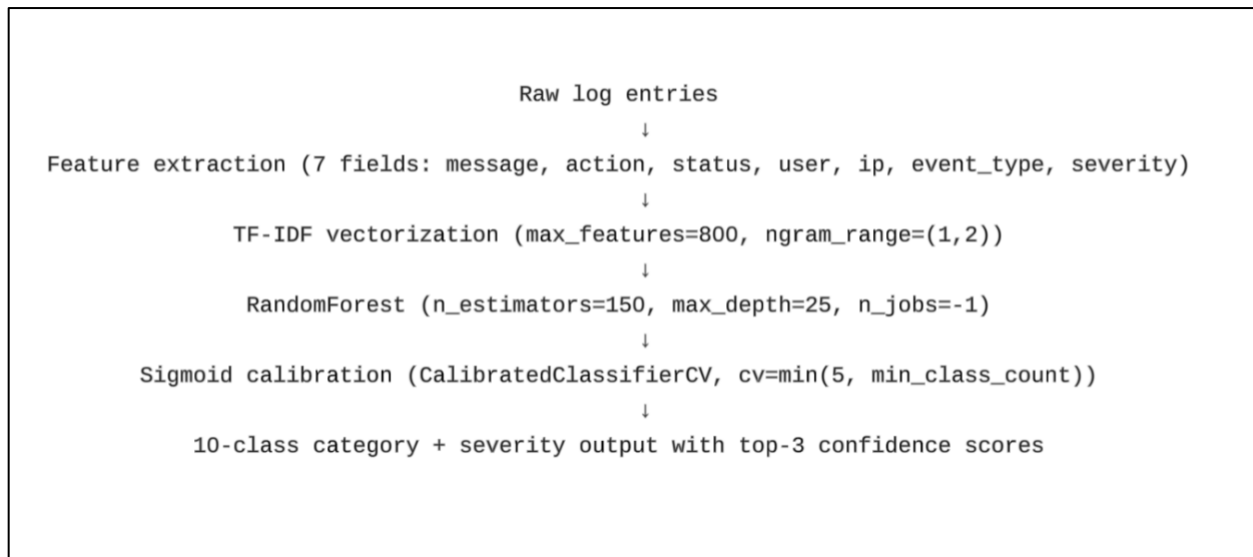


Figure 3.13: Random Forest Classification Pipeline from Top to Bottom

Training configuration (*POST /rf/train*):

- Dataset: CICIDS2019 and CICIDS2017, up to 120,000 rows per training attempt.
- Split: 70% train / 15% validation / 15% test (stratified)
- Random seed: 42 (reproducible)
- Activation threshold: weighted precision ≥ 0.80
- Metrics tracked: accuracy, precision/recall/F1 (macro + weighted), confusion matrix

Models below the activation threshold are archived rather than deployed. Active models persisted as pickle artifacts both locally (`app/.models/rf_classifier.pkl`) are remotely in the *ml-models* Supabase Storage bucket with version-stamped filenames, enabling rollback. The *model_versions* table tracks status, metrics, and artifact paths; *training_runs* records the full

execution history.

At inference time, *classify_batch(entries)* returns per-entry classifications including category, severity, top 3 confidence predictions, and an aggregate *rf_context* dict summarizing category distribution and high-confidence anomaly counts for consumption by the confidence scoring stage.

3.3.8 Insights & Natural Language Generation.

The *InsightsGenerator* class (*insights_generator.py*) converts structured threat analysis output into human-readable intelligence through four generation methods:

Method	Input	Output
<i>generate_threat_insights()</i>	Threat list	Per-threat: summary, concern, immediate actions, long-term mitigation, risk score (1-10), exploitability.
<i>generate_incident_summary()</i>	Threats + log count	Executive summary: what happened, overall risk level, top 3 recommended actions
<i>generate_investigation_guide()</i>	Classification context + threats	Investigation steps, relevant data sources, suggested queries, escalation criteria
<i>generate_remediation_plan ()</i>	Single threat	Phased plan: immediate (0-1 hr), short term (1-24hr), medium term (1-7 days),

		long term (1-3 months)
--	--	------------------------

Table 3.3: Insights Generator Breakdown

All methods submit structured prompts to OpenAI GPT-4o-mini (temperature 0.2, max tokens 512-700) requesting JSON-formatted responses for programmatic parsing. This cloud deployment using GPT-4o-mini provide higher-quality output.

3.3.9 Background Job Processing

File uploads exceeding Flask’s synchronous handling capacity use a three-phase chunked upload system with background job processing:

Phase 1 → Session Initialization (*POST /upload-sessions/init*):

Creates a session manifest in Supabase Storage at *{org_id}/upload_session/{session_id}/manifest.json*, tracking expected parts, filename, source type, and expiry (3600s TTL). Returns *session_id* to the client.

Phase 2 → Part Upload (*POST /upload-sessions/upload-part*):

Each chunk (up to 16MB, configurable via *MAX_UPLOAD_PART_BYTES*) is stored at *{org_id}/upload_sessions/{session_id}/parts/{part_number:08d}.part*. Part number padding ensures lexicographic ordering matches numeric ordering for assembly.

Phase 3 → Assembly and Job Creation (*POST /upload-sessions/complete*):

The backend retrieves all parts, validates contiguity (parts 1 through N must all exist), concatenates them (total size enforced against *MAX_UPLOAD_SESSION_ASSEMBLY_BYTES = 2GB*), uploads the assembled file to the *log-files* bucket, and creates an *analysis_jobs* +

analysis_job_items record pair. The client receives a *job_id* for status polling.

Job processing follows the pipeline described in Sections 3.3.3-3.3.8, updating

analysis_job_items.progress_pct and the parent *analysis_jobs.progress_pct* incrementally. The

frontend polls *GET /analysis-jobs/{job_id}* at 1-second intervals until status reaches *completed* or *failed*.

Job states: queued → running → completed | failed | partial.

3.3.10 Timeline Service

The *TimelineService* (*timeline_service.py*) aggregates events from four sources into a unified chronological view:

1. *raw_logs*: Ingest-time events
2. *detections*: Correlation engine matches
3. *analysis_results.mitre_techniques*: mapped ATT&CK techniques
4. *analysis_results*: summary/findings and LLM-identified threat events

Two timeline scopes are supported: file-level (events from a single upload) and organization-level (events across all uploads for an org). Both support filtering by timestamp range, severity, and event type, with pagination (default 100 items/page, max 500).

For graph visualization (*GET /timeline/file/{file_id}/graph*), the service builds a node-edge structure where nodes represent events and edges represent temporal sequence or causal relationships (e.g., a correlation detection node linking to the raw log nodes that triggered it). The graph is capped at 120 nodes (file scope) and 160 (org scope) for rendering performance.

3.4 Frontend Design

3.4.1 Application Structure

frontend/src/	
├ App.jsx	Route definitions, ProtectedRoute component
├ client.js	Supabase client, authenticatedFetch wrapper
├ pages/	
│ └ homepage.jsx	Public landing page
│ └ login.jsx	GitHub OAuth + Supabase Auth UI
│ └ dashboard.jsx	Main analyst interface (upload, analysis, timeline)
│ └ admin-dashboard.jsx	Admin controls (org, users, models, rules)
│ └ collaborative-workspace.jsx	Incidents, tasks, team collaboration
│ └ ai-feedback.jsx	AI output rating and correction
│ └ remediation-view.jsx	Phased remediation plan display
├ components/	
│ └ navbar.jsx	Navigation with auth state
│ └ footer.jsx	Footer
│ └ search-bar.jsx	Filter component (threat level, source, date)
├ hooks/	
│ └ usePersistentState.js	localStorage-backed state hook
└ css/	Per-page stylesheets

Figure 3.14: Frontend File Structure in Environment

3.4.2 Authentication Flow

Authentication uses GitHub OAuth via Supabase Auth. The flow:

1. User visits `/login` and clicks “Sign in with GitHub”
2. Supabase redirects to GitHub OAuth consent
3. GitHub redirects back with authorization code
4. Supabase exchanges code for session; JWT stored in `localStorage`
5. `ProtectedRoute` reads session via `supabase.auth.getSession()`
6. Role extracted from `session.user.app_metadata.role`
7. Admin-only routes redirect non-admin users to `/dashboard`

The `authenticatedFetch` wrapped (`client.js`) attaches `Authorization: Bearer {token}` to

every API request and handles 401 responses by redirecting to */login?reason=expired*, and 403 responses by throwing a typed *HttpAuthError* for UI-level handling.

3.4.3 Dashboard

The dashboard (*dashboard.jsx*) is the primary analyst interface. Key capabilities:

- File upload: Dual-mode, which is the standard multipart upload for small files, chunked session upload (4MB chunks) for large files.
- Progress tracking: SSE stream consumption for streaming uploads; polling for background jobs.
- Multi-file tracking: Concurrent upload state managed with *usePersistentState* (localStorage-backed to survive page refreshes).
- Analysis results display: Threat level, threats found, MITRE technique cards, confidence score, detailed findings list.
- Timeline visualization: Per-file event timeline with severity filtering.

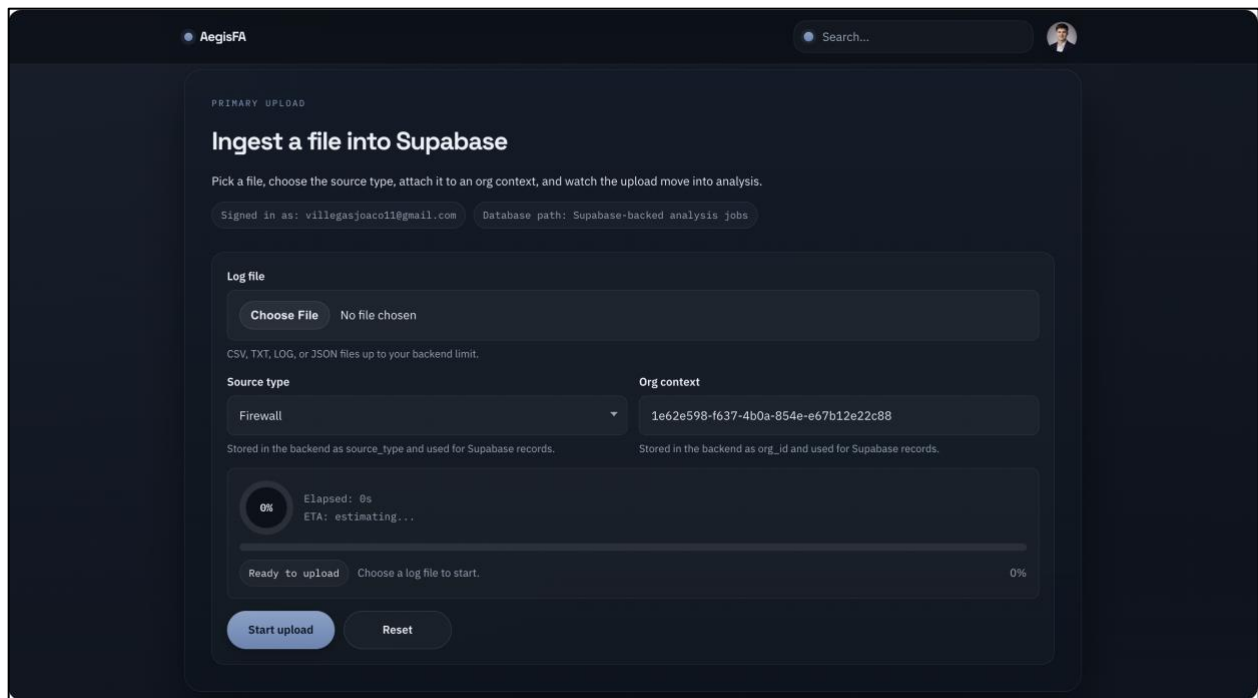


Figure 3.15: User Interface File Ingestion

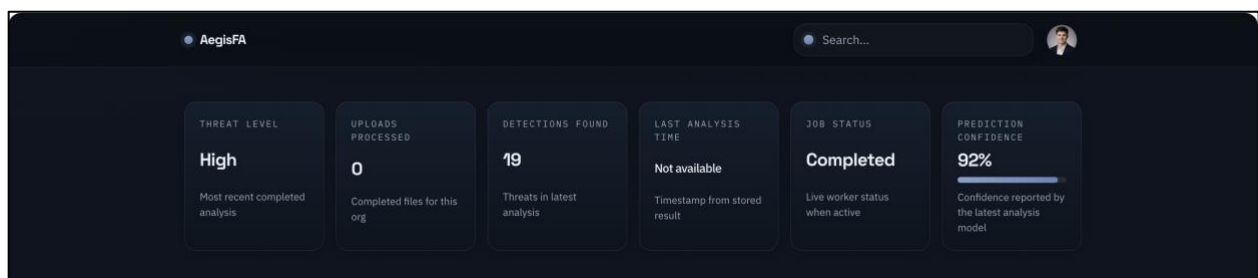


Figure 3.16: User Interface Analysis Result Cards

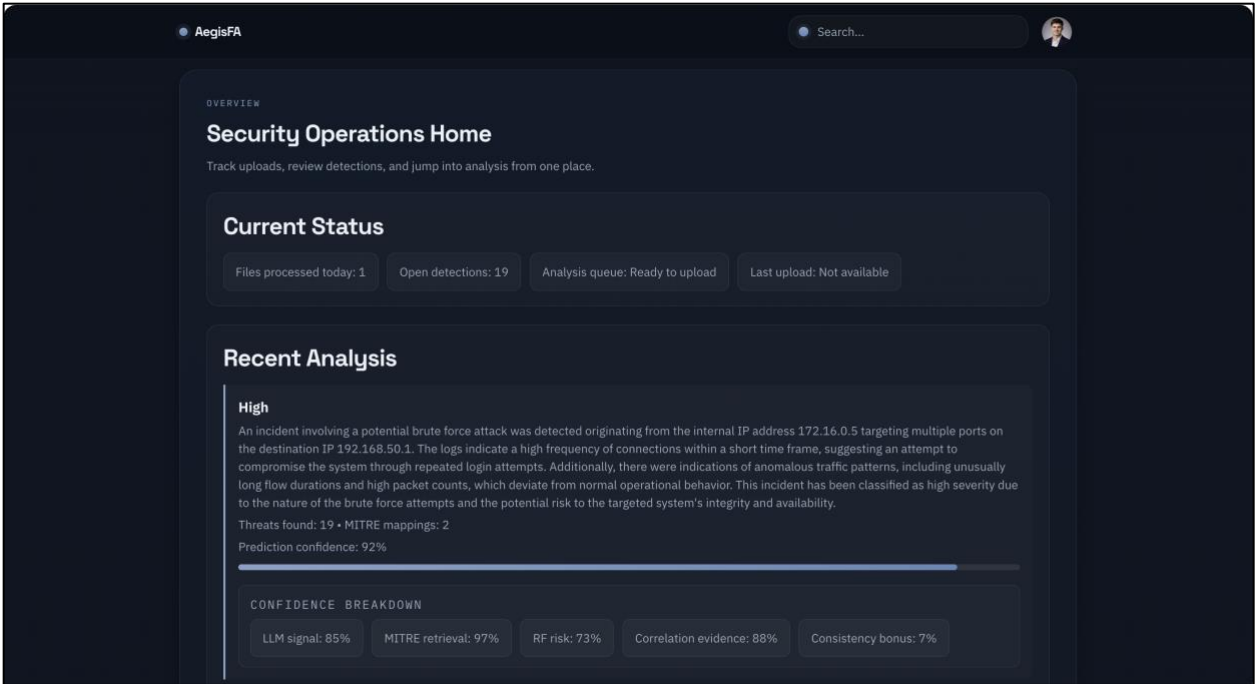


Figure 3.17: User Interface Analysis Result AI Narrative & Confidence Breakdown

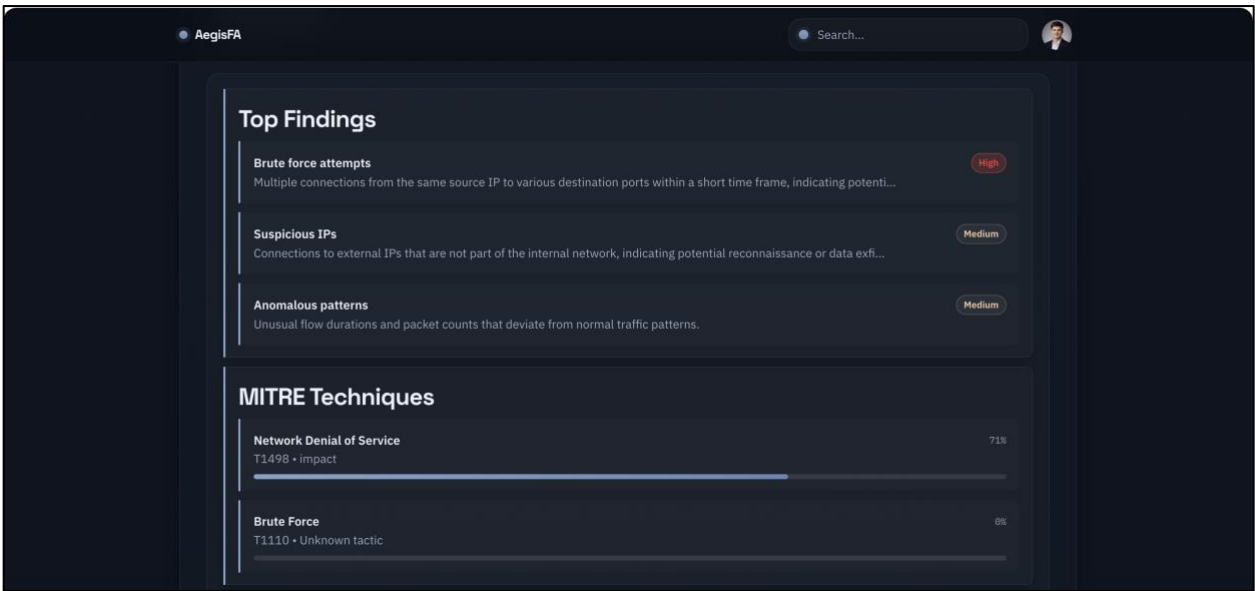


Figure 3.18: User Interface Analysis Result Top Findings & Technique Matchings

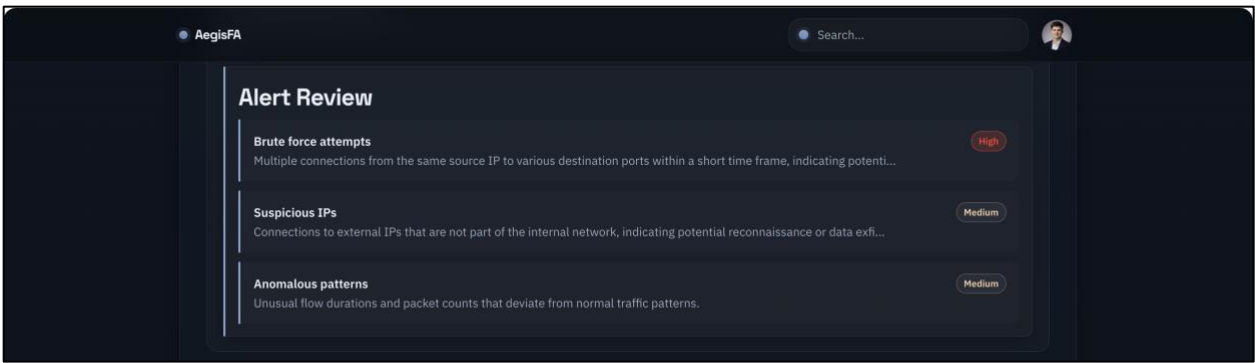


Figure 3.19: User Interface Analysis Results Alert Review

The screenshot shows the 'Recent Jobs' section of the AegisFA interface. It contains a table with the following data:

FILENAME	STATUS	PROGRESS	STARTED	COMPLETED	RESULT
Unknown	Completed	100%	4/15/2026, 3:17:11 PM	4/15/2026, 3:18:17 PM	Pending
Unknown	Completed	100%	4/13/2026, 8:54:35 AM	4/13/2026, 8:54:57 AM	Pending
Unknown	Completed	100%	4/12/2026, 11:50:48 PM	4/12/2026, 11:52:11 PM	Pending
Unknown	Queued	0%	4/12/2026, 11:47:08 PM	Not available	Pending
Unknown	Completed	100%	4/9/2026, 9:46:07 AM	4/9/2026, 9:47:22 AM	Pending
Unknown	Completed	100%	4/9/2026, 9:28:15 AM	4/9/2026, 9:37:10 AM	Pending
Unknown	Failed	100%	4/9/2026, 9:27:01 AM	4/9/2026, 9:27:20 AM	Pending
Unknown	Failed	100%	4/9/2026, 9:24:17 AM	4/9/2026, 9:24:35 AM	Pending

Figure 3.20: User Interface Analysis Results Recent Jobs with Timestamps and Status

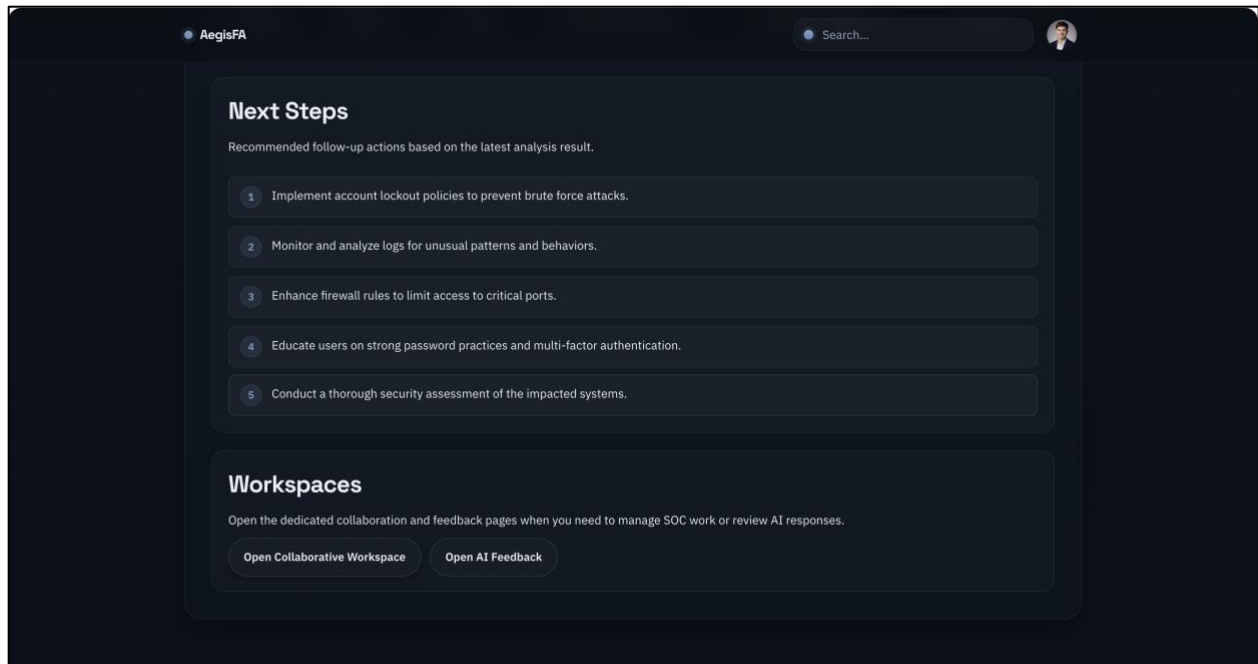


Figure 3.21: User Interface Analysis Result Remediation Steps

3.4.4 Admin Panel

The admin panel (*admin-dashboard.jsx*) is role-gated (admin only). It provides:

- Organization-wide upload and analysis status visibility
- User role management
- Correlation rule creation and management
- ML model training trigger (*POST /rf/train*) with training run status display
- Model version management

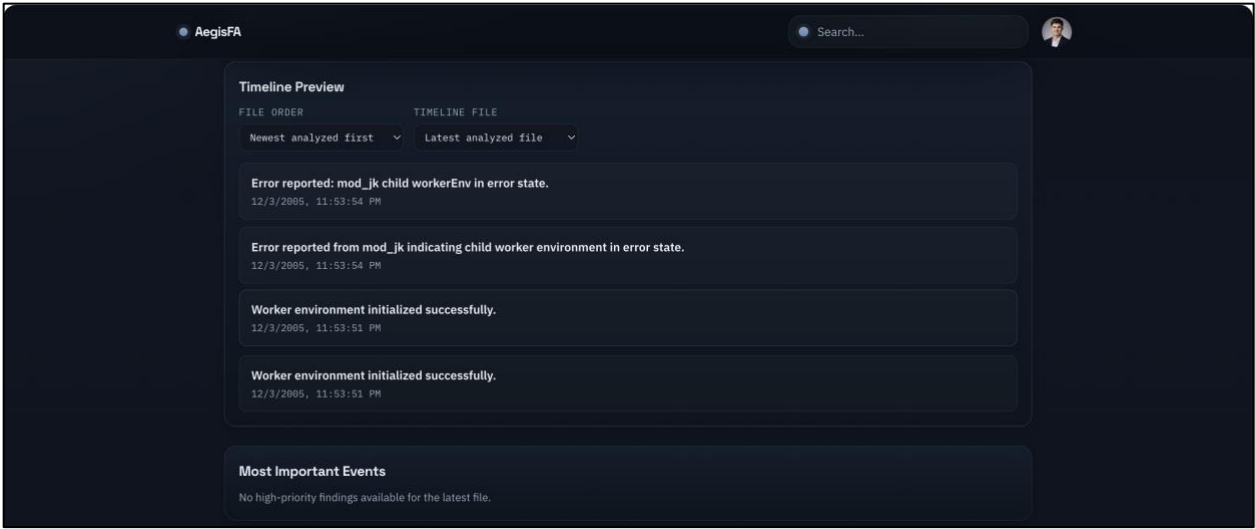


Figure 3.22: User Interface Admin Analysis Results Timeline Preview of Events

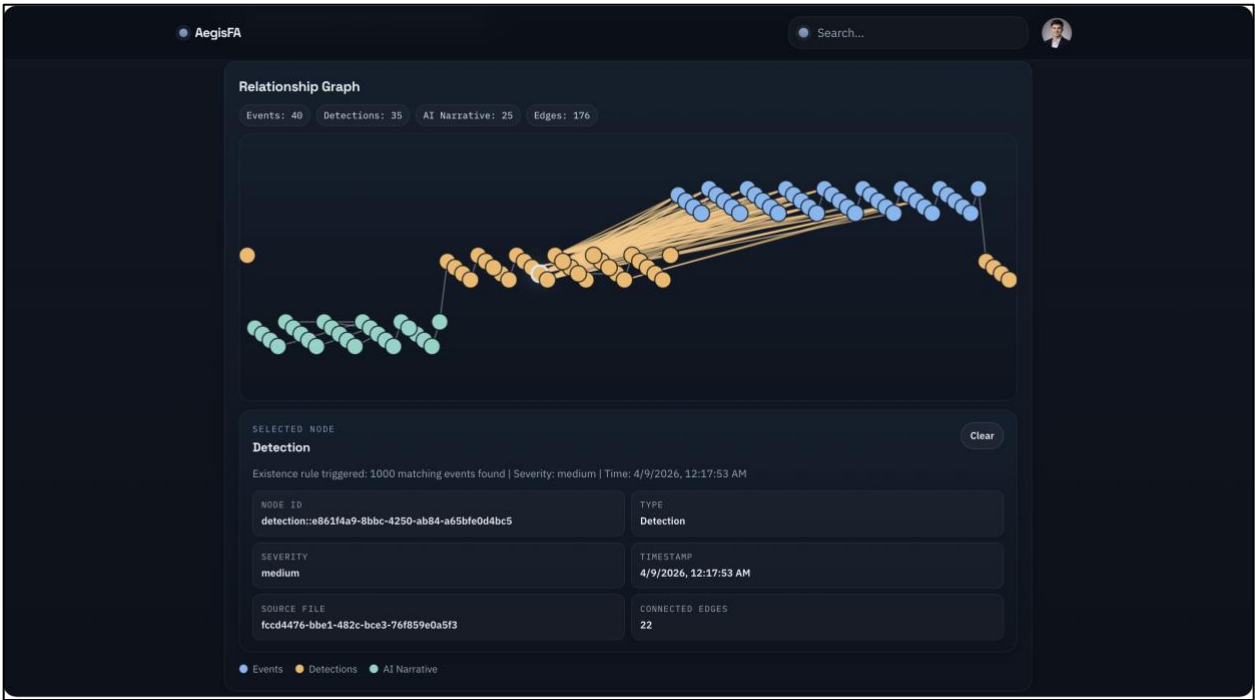


Figure 3.23: User Interface Admin Analysis Results Relationship Graph (Full Attack Chain)

3.4.5 Workspace

The collaborative workspace (*collaborative-workspace.jsx*) supports incident response team workflows:

- Incident list with creation and status update forms
- Task creation, assignment to team members, and status tracking
- Event-to-incident linking
- AI-generated incident narrative display (from *summaries* table)

INCIDENT	STATUS	SEVERITY
brute force attempts	Open	High
suspicious IPs	Open	Medium
suspicious IPs	Open	Medium
suspicious IPs	Open	Medium
suspicious IPs	Open	Medium
suspicious IPs	Open	Medium
anomalous patterns	Resolved	Medium

Figure 3.24: User Interface Collaborative Workspace with Incident Management

3.4.6 AI feedback

The AI-feedback section allows analysts to provide feedback based on the detections and AI-produced output allowing to improve and modify training as well as allowing the generation of more precise responses and analysis.

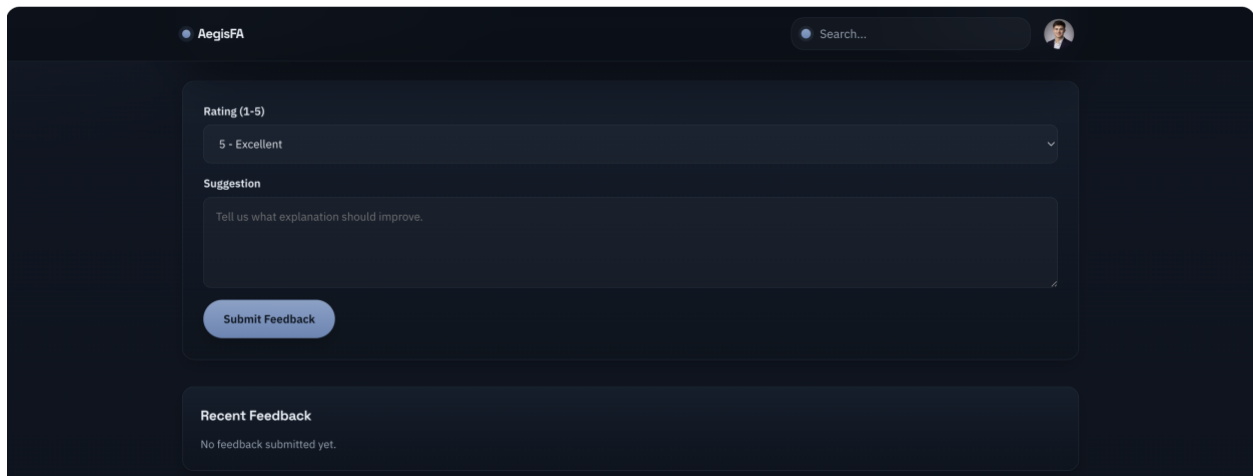


Figure 3.25: User Interface AI-Feedback with Comments and Response Rating

3.4.7 Supplemental Pages (Home, Support, About, Contact, Info)

AegisFA is a fully developed web-platform that contains the standard supplemental pages any website should have: *homepage.jsx*, *contact.jsx*, *login.jsx*, *privacy.jsx*, *support.jsx*, *terms.jsx*, *about.jsx*, and *privacy.jsx*.

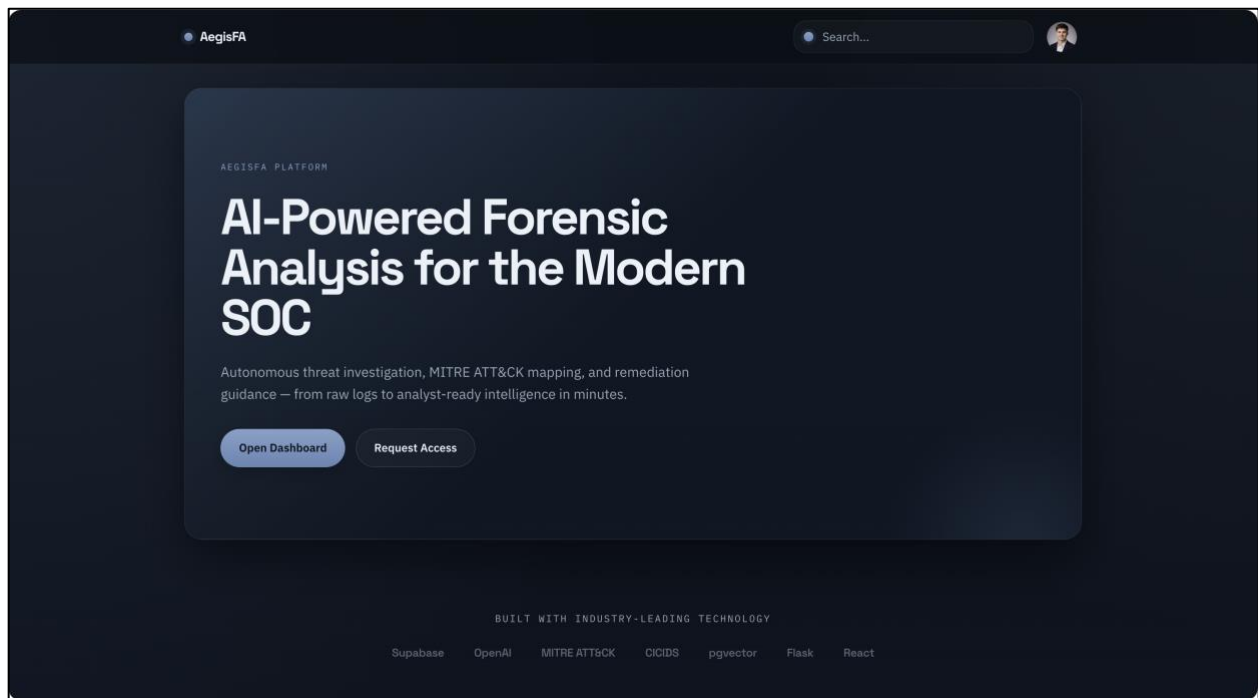


Figure 3.26: User Interface Homepage

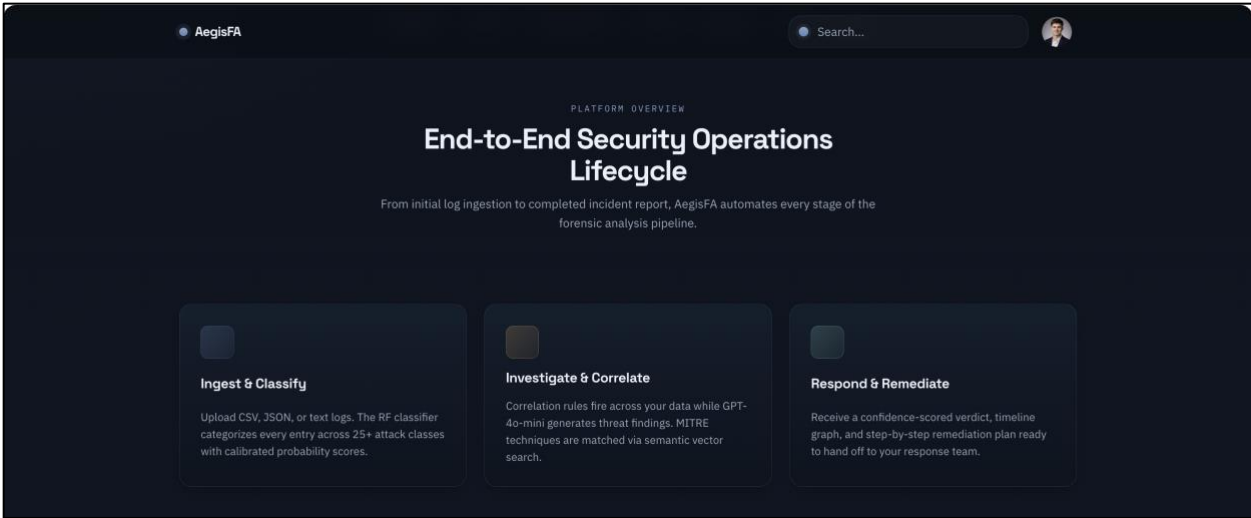


Figure 3.27: User Interface Homepage Security Operations Lifecycle

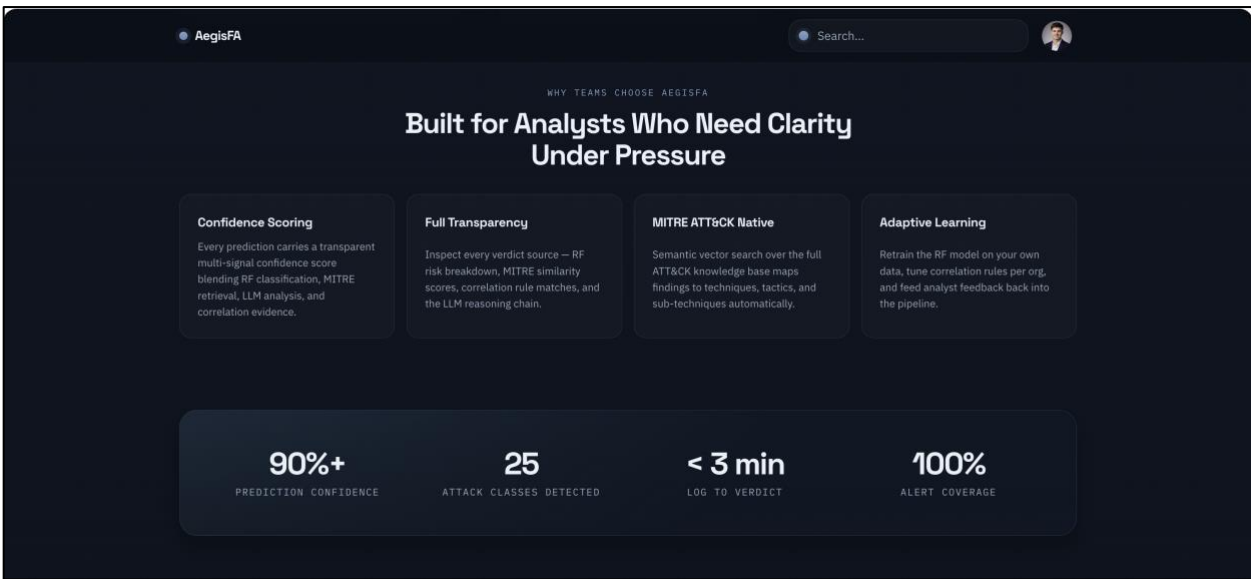


Figure 3.28: User Interface Homepage Target Audience and Benchmarks

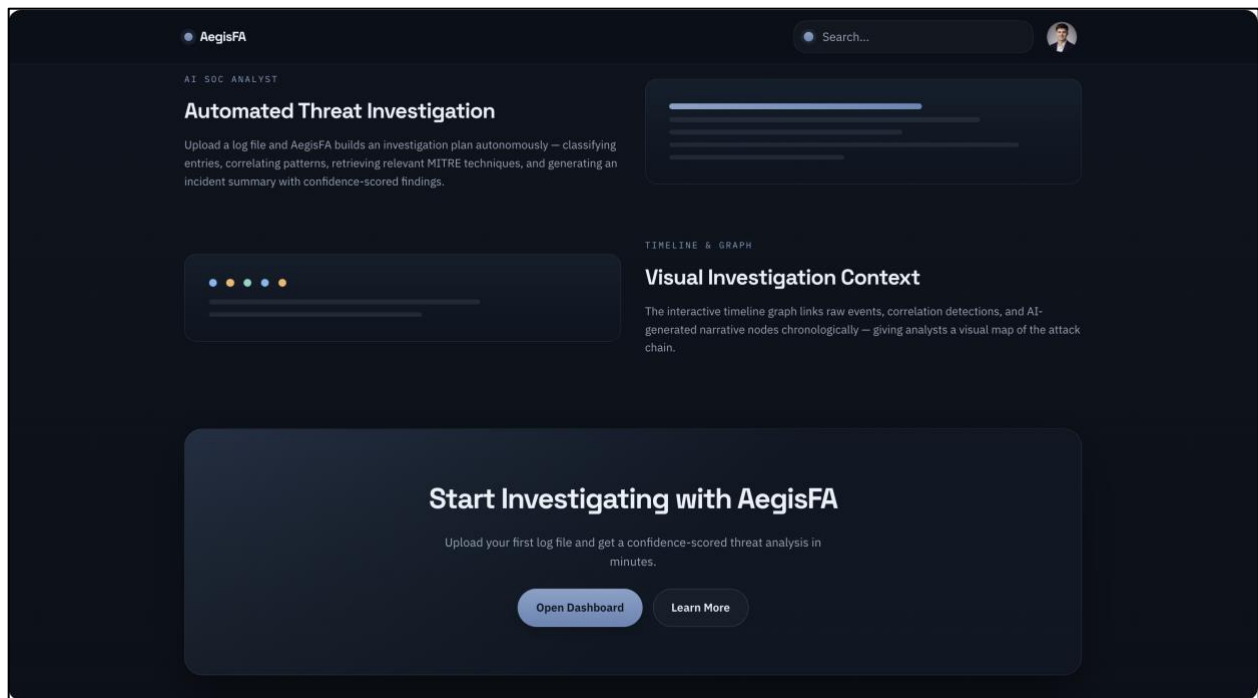


Figure 3.29: User Interface Homepage System Capabilities Explanation

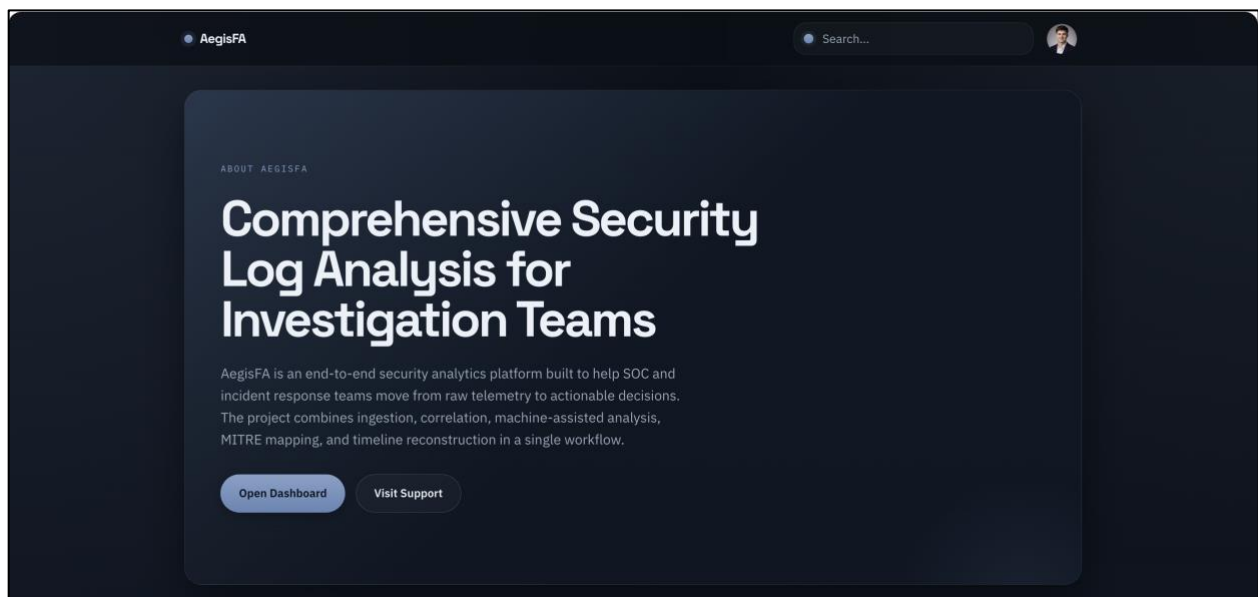


Figure 3.30: User Interface About Page

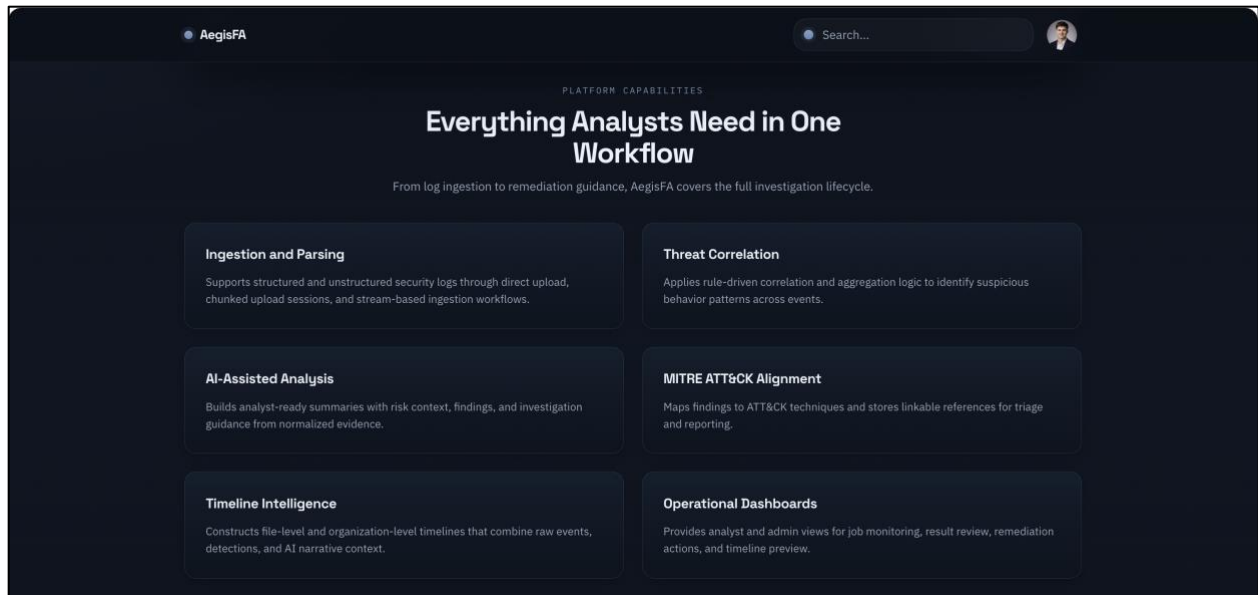


Figure 3.31: User Interface About Page Workflow Breakdown

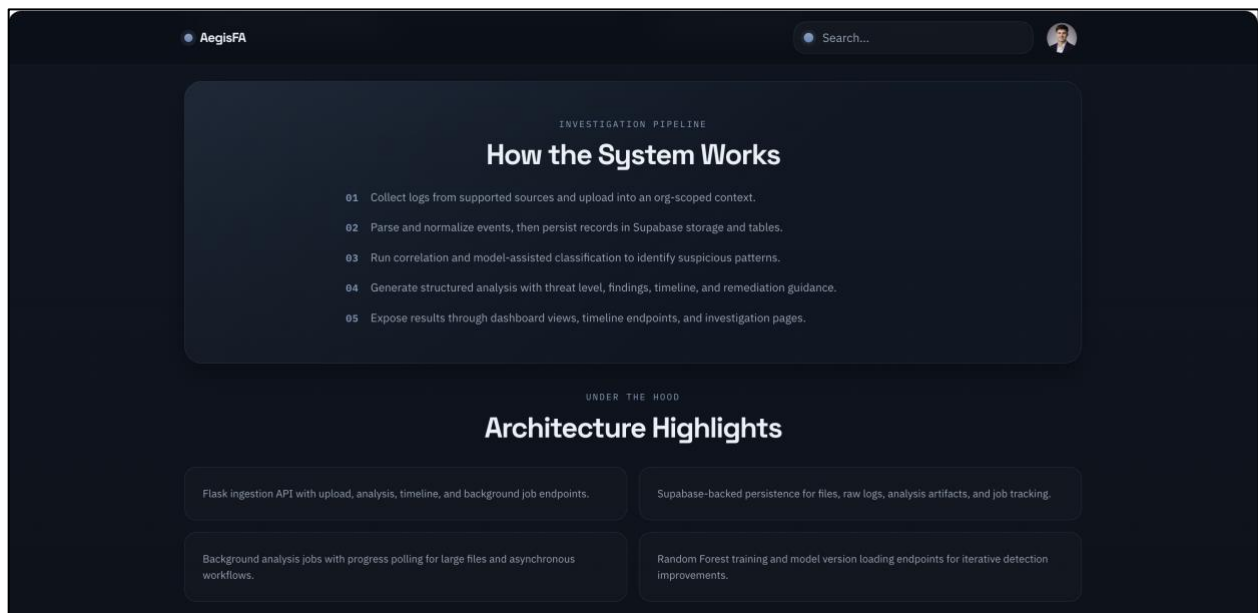


Figure 3.32: User Interface About Page Step-by-Step

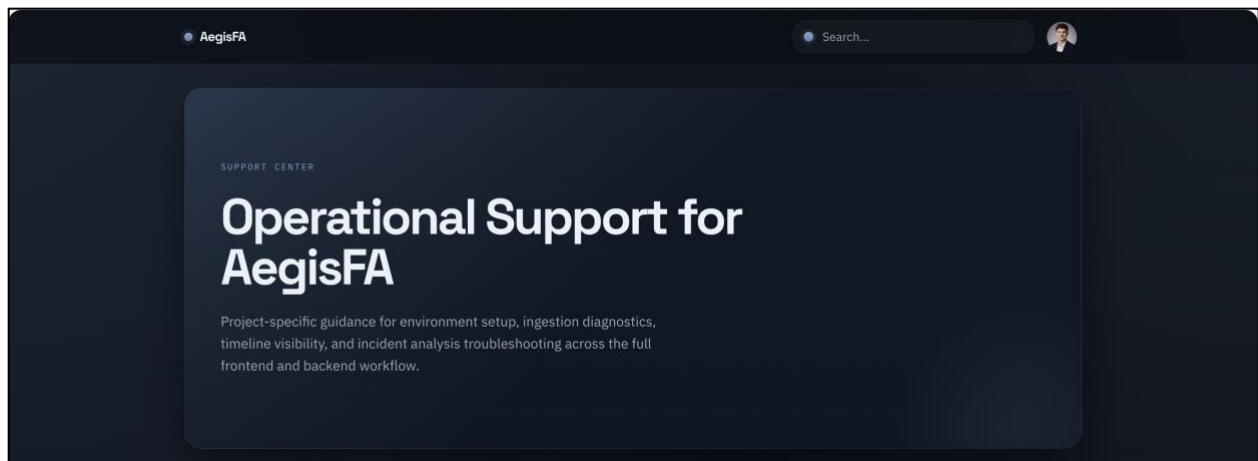


Figure 3.33: User Interface Support Page

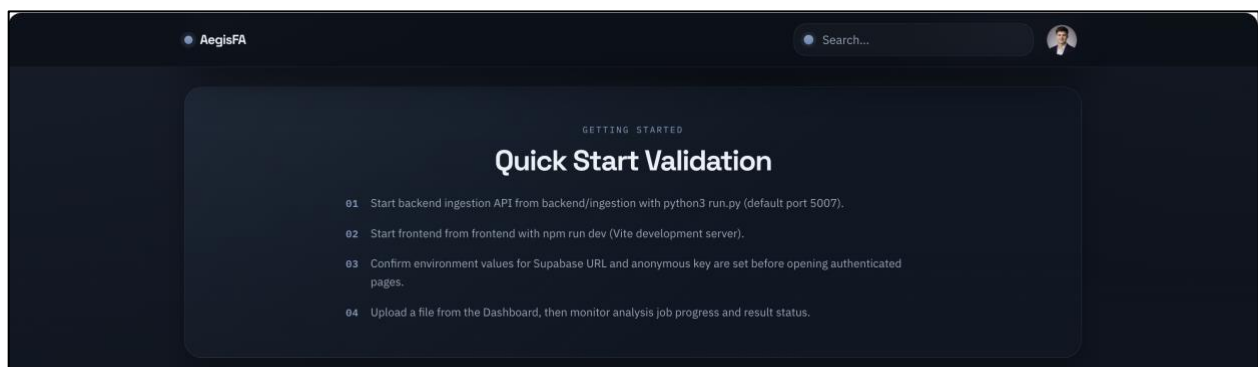


Figure 3.34: User Interface Support Page Quick Start Validation

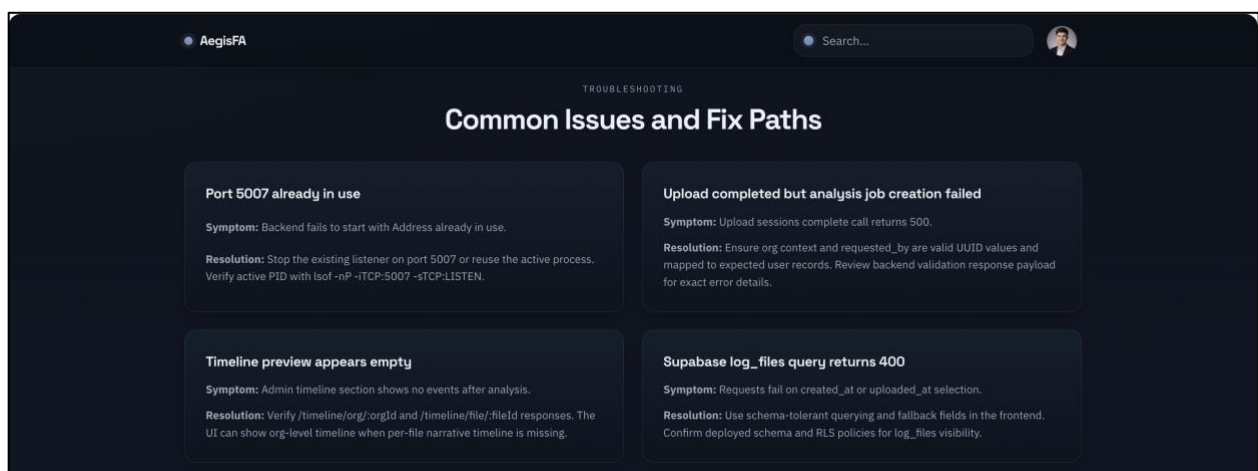


Figure 3.35: User Interface Support Page Common Issues and Fix Paths



Figure 3.36: User Interface Support Page API Diagnostics



Figure 3.37: User Interface Support Page Reporting Formatting

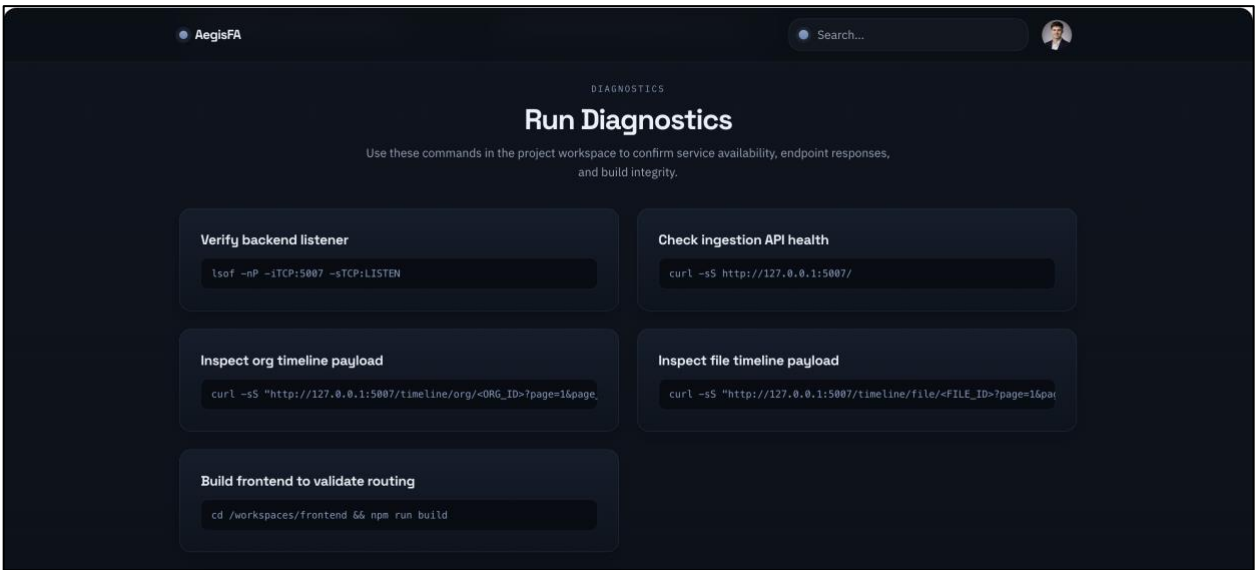


Figure 3.38: User Interface Support Page "How To Run Diagnostics"

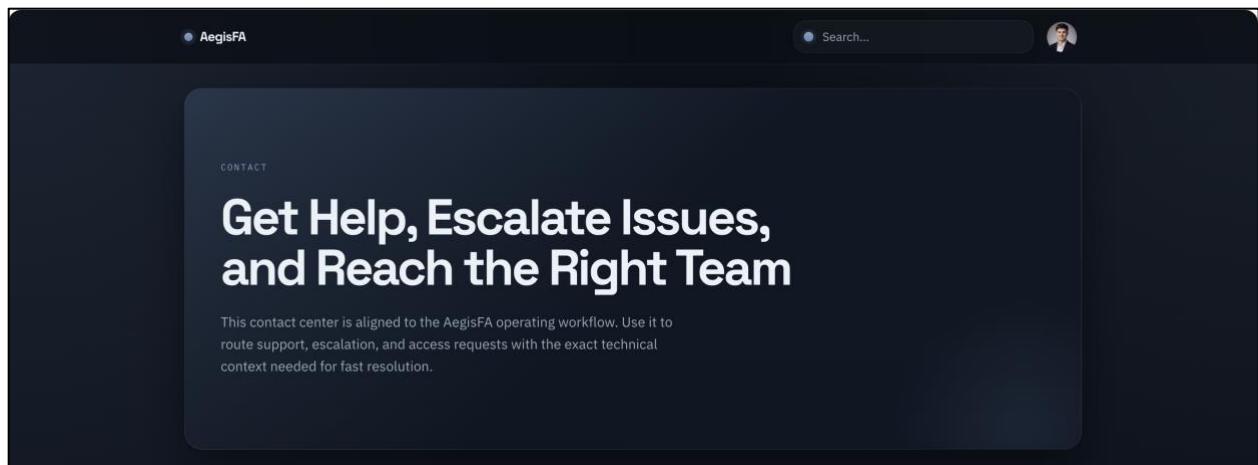


Figure 3.39: User Interface Contact Page

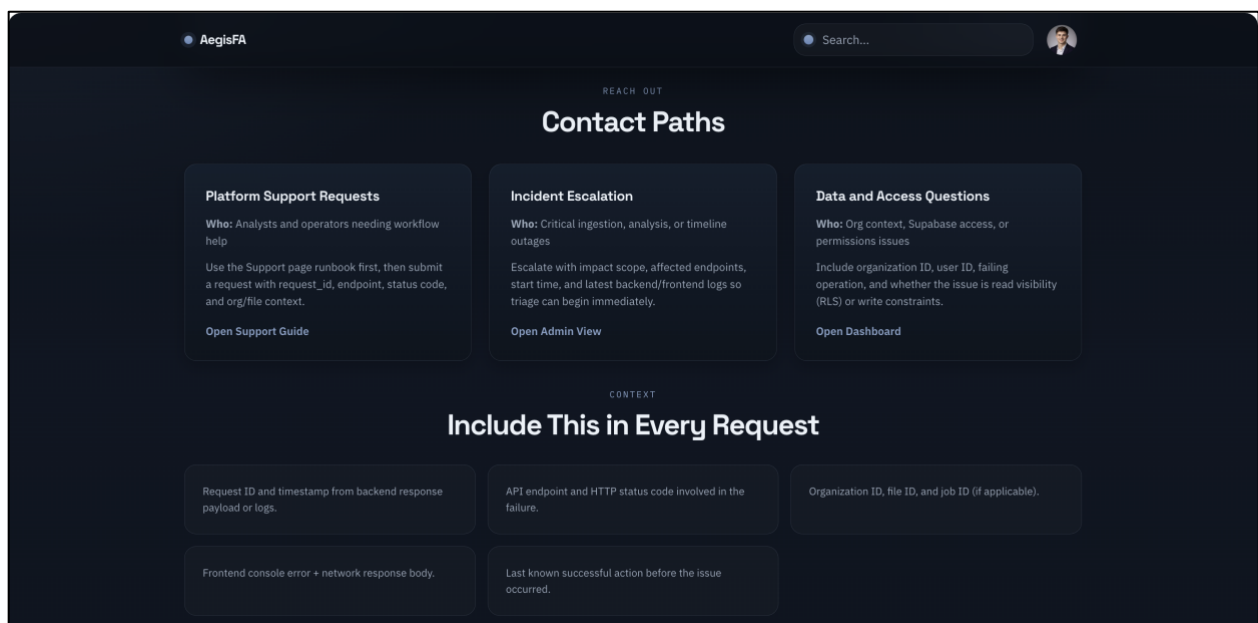


Figure 3.40: User Interface Contact Paths and Context

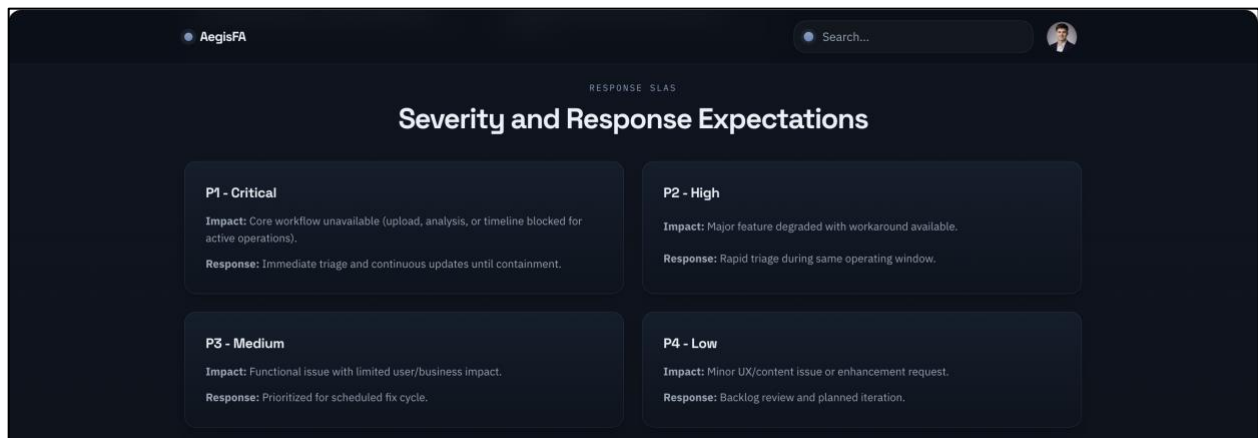


Figure 3.41 User Interface Contact Page Response Expectations

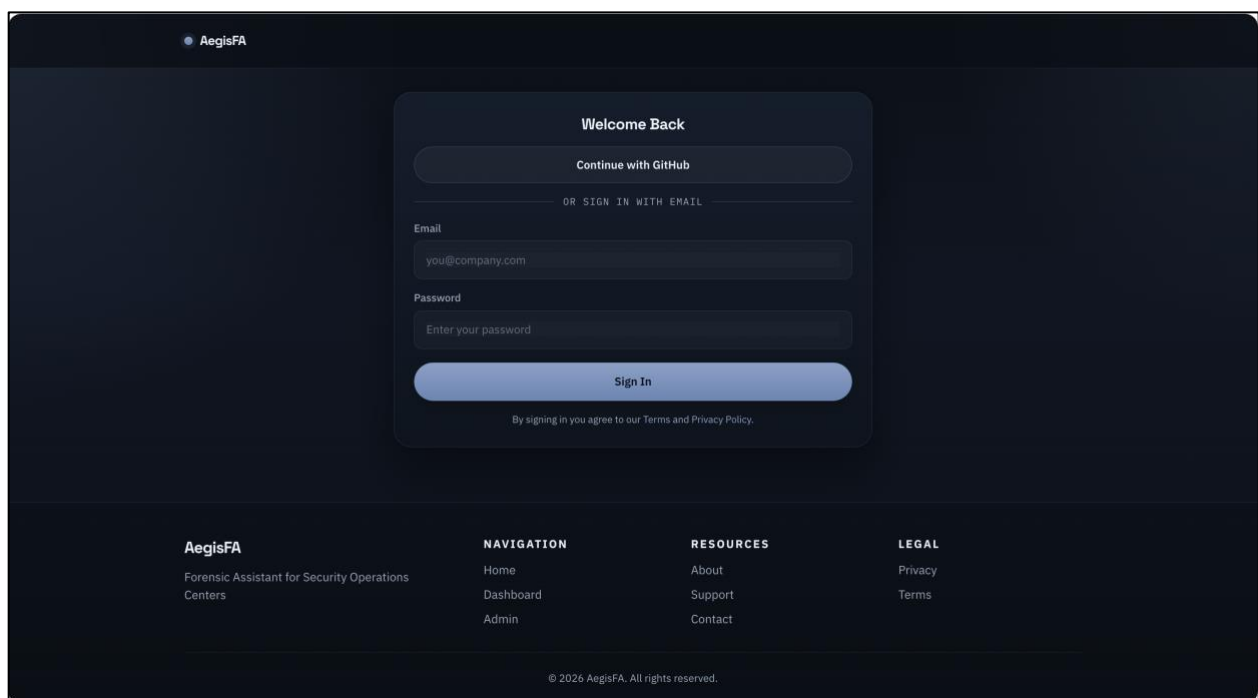


Figure 3.42: User Interface Login Page

3.5 API Design

AegisFA exposes a RESTful API organized into six functional groups. All endpoints except *GET* / require Bearer token authentication. The complete endpoint inventory from Chapter 2 (Section 2.2) applies; the design principles are documented are:

Design decisions:

- Resource-oriented URLs: */incidents/{id}*, */tasks/{id}*, */analysis/{file_id}* follow REST conventions
- Org scoping at API layer: *org_id* required as parameter on all write operations; validated against authenticated identity.
- Structured error responses: All errors return *{error: {code, message, retrieable, request_id}}*, never raw exception text
- Retriable flag: Server-side error
(DATABASE_ERROR, STORAGE_ERROR, ANALYSIS_ERROR, MODEL_LOAD_ERROR, RATE_LIMIT) are flagged *retrieable: true*; client-side errors are not.
- Pagination: Timeline endpoints support *page* and *page_size* parameters (default 100, max 500)
- Job-based async: Long-running analyses return a *job_id* immediately; clients poll *GET /analysis-jobs/{job_id}* for completion

3.6 Security Architecture

AegisFA implements a multi-layered security model addressing authentication, authorization, data isolation, and operational security.

Layer 1 → Authentication: Every API request is validated against Supabase Auth. The Bearer token is verified via *supabase_client.auth.get_user(token)*. Invalid, expired, or missing tokens return HTTP 401. The frontend handles 401 responses by clearing the session and redirecting to */login?reason=expired*.

Layer 2 → Role-Based Access Control: Three roles define capability boundaries:

- *admin*: Full access including ML training, model management, admin dashboard.
- *analyst*: Upload, analyze, create incidents and tasks, submit feedback.
- *viewer*: *Read-only access to analysis results, incidents, tasks, timelines.*

Role is stored in the *users* table and validated server-side on every request, also never trusted from the client token payload alone.

Layer 3 → Organization Scope Enforcement: The `_enforce_org_scope()` and `_enforce_file_scope()` functions verify that requested resources belong to the authenticated organization. This check occurs at the API layer before any database query.

Layer 4 → Row-Level Security: PostgreSQL RLS policies on 17 tables provide a final enforcement layer at the database level. Even if an API-layer check is bypassed, the database will not return or accept data belonging to a different organization. The service role key used by the backend bypasses RLS for write operations, and this is intentional and controlled: only the backend process holds this credential.

Layer 5 → Operational Security: Structured error responses (`_error_response()`) never expose stack traces, internal service names, or database error text in production. Request IDs propagate through all log entries, enabling incident reconstruction without leaking sensitive context. The `LOG_LEVEL` environment variable controls log verbosity for production deployments.

This layered approach aligns with NIST CSF 2.0’s “Protect” function, which emphasizes identity management, access control, and data security as core protective capabilities (NIST, 2024).

3.7 Chapter Summary

This chapter presented the complete system design for AegisFA across six dimensions: architecture, database, backend services, frontend interfaces, API, and security.

AegisFA follows a three-tier architecture composed by React frontend, Flask backend,

Supabase/PostgreSQL database, all containerized via Docker for reproducible deployment. The tiers communicate over HTTP REST with Vite proxy handling in development and *VITE_API_BASE_URL* in production. Supabase provides both the relational database and managed authentication, reducing infrastructure overhead while enabling enterprise features (RLS, pgvector, OAuth) without custom implementation.

The database schema comprises 22 tables across 8 migrations. Multi-tenancy is implemented through *org_id* foreign keys on all data tables, enforced by RLS policies on 17 tables. The *mitre_techniques* table stores 1,536-dimensional OpenAI embeddings indexed via HNSW with cosine similarity, enabling semantic ATT&CK technique matching without keyword dependency. The *analysis_results* and *verdict_sources* tables preserve full auditability of how threat levels and confidence scores were computed.

The backend pipeline orchestrates seven services in sequence: file parsing (four formats, encoding-aware), rule-based normalization (source-specific field mapping), RandomForest classification (CICIDS2019/2017-trained, 10 categories, 17+ attack types), rule-based correlation (five rule types, nine filter operators), LLM threat detection (GPT-4o-mini), MITRE RAG retrieval (HNSW similarity search, 0.45 threshold), and NLG insight generation (threat insights, incident narrative, investigation guide, remediation plan). The confidence score blends RF, correlation, LLM, and retrieval signals with defined weights and a consistency bonus. Background job processing handles large files through a three-phase chunked upload system with session management, assembly validation, and status polling.

The frontend provides six protected routes across three interfaces: the analyst dashboard (upload, analysis, timeline), the admin panel (org management, model training, rule configuration), and the collaborative workspace (incidents, tasks, AI narratives). Authentication

uses GitHub OAuth via Supabase Auth with JWT session management; role-based rendering restricts admin-only interfaces at the component level.

The security architecture implements five enforcement layers, from authentication, RBAC, org scope validation, RLS, and operational security, which is aligned with NIST CSF 2.0 Protect function requirements. No single layer is relied upon exclusively; failure of any one layer does not expose cross-tenant data. Chapter 4 details the implementation of each component described here, including specific code-level decisions, integration challenges, and deployment configuration.

Chapter 4 – Implementation

4.1 Development Environment & Tools

AegisFA was develop inside a VS Code devcontainer defined in `.devcontainer/docker-compose.yml`, providing a consistent, reproducible environment from day one. This eliminated “works on my machine” problems across different development contexts and ensured the development environment matched the production container configuration.

4.1.1 Core toolchain:

Layer	Tool	Version	Purpose
Frontend runtime	Node.js	20-alpine3.21	JS runtime for Vite and React
Frontend bundler	Vite	5.4.19	HMR dev server, production build, API proxy
Backend runtime	Python	3.11-bullseye	Flask application server
Backend WSGI	Gunicorn	latest	Production HTTP server
Database	Supabase (PostgreSQL)	Managed	Relational DB, Auth, Storage, pgvector
Version Control	Git	-	Source Control
Containerization	Docker + Compose	-	Environment isolation and orchestration
IDE	VS Code	-	Devcontainer-integrated development

Table 4.1: Structure of the Core Toolchain Breakdown

4.1.2 Environment configuration:

All sensitive credentials and operational parameters are injected via environment variables, never

hardcoded. Backend variables include

SUPABASE_URL, *SUPABASE_SERVICE_ROLE_KEY*, *OPENAI_API_KEY*, and all operational constants

(*MAX_UPLOAD_PART_BYTES*, *SUPABASE_RETRY_ATTEMPTS*, *INSIGHTS_LLM_PROVIDER*, etc.). Frontend variables use Vite's *VITE_* prefix convention:

VITE_SUPABASE_URL, *VITE_SUPABASE_ANON_KEY*, *VITE_API_BASE_URL*.

4.2 Database Implementation

4.2.1 Migration Strategy

Database schema was implemented through 8 sequential SQL migration files (*000* through *007*), each scoped to a single functional domain. This incremental approach meant earlier migrations could be deployed and tested before dependent features were built, and the pgvector MITRE table (*001*) was seeded and validated before the RAG service that queries it was written.

Migration sequence:

File	Domain
<i>000_db_implementation.sql</i>	Core tables: organizations, users, log pipeline, incidents
<i>001_pgvector_mitre.sql</i>	pgvector extension, <i>mitre_techniques</i> table, HNSW index
<i>002_correlation_engine.sql</i>	<i>correlation_rules</i> , <i>detections</i> tables
<i>003_rls_implementation.sql</i>	RLS policies for all 17 critical tables
<i>004_log_analysis_results.sql</i>	<i>log_files</i> , <i>analysis_results</i> , <i>analysis_result_mitre_links</i>
<i>005_analysis_job.sql</i>	<i>analysis_jobs</i> , <i>analysis_job_items</i> background processing

<i>006_correlation_errors.sql</i>	<i>correlation_errors</i> audit table
<i>007_rf_training.sql</i>	<i>model_versions</i> , <i>training_runs</i> ML tracking

Table 4.2: Supabase Database Migration Sequence

4.2.2 pgvector Setup

The pgvector extension was enabled on the Supabase PostgreSQL instance and the *mitre_techniques* table populated with pre-computed OpenAI *text-embedding-3-small* embeddings for all MITRE ATT&CK techniques. The 1536-dimensional vector column was defined as: *embedding vector (1536)*. Moreover, the HNSW index was created after data population to avoid incremental index updates during bulk insert.

A Supabase RPC function *match_mitre_techniques* wraps the similarity query, abstracting the vector operator syntax from the Python service layer as shown in A. 1: Supabase RPC Wrapper.

4.2.3 RLS Implementation

Each table received four policies (SELECT, INSERT, UPDATE, DELETE) following the org-scope pattern. Implementation required care around tables with indirect org relationships, such as *analysis_result_mitre_links* carries an explicit *org_id* column (denormalized from the parent *analysis_results*) to avoid policy JOIN overhead on high-frequency queries.

4.3 Backend Implementation

4.3.1 Application Factory & Middleware

The Flask application is initialized in *__init__.py* with *MAX_CONTENT_LENGTH = 120 * 1024 * 1024* (120 MB) and the *main* Blueprint registered. The middleware chain in *routes.py*

executes on every request via *@main.before_request*.

The *_authenticate_request()* function resolves user identity in four steps:

1. Extract Bearer token from *Authorization* header; return 401 if absent or malformed
2. Validate token via *supabase_client.auth.get_user(token)*
3. Query *users* table for *org_id* and role; auto-provision if absent (default role: *viewer*)
4. Store resolved identity in *g.auth_user_id*, *g._auth_org_id*, *g.auth_role*

Auto-provisioning handles first-login scenarios, and Supabase Auth creates the *auth.users*, but the *users* table record (with *org_id* and *role*) is created lazily on first authenticated API call. The *org_id* is claimed from *user_metadata* (set during GitHub OAuth) or from the request payload if available.

Structured logging is configured in *logging_config.py* using *structlog* with JSON rendering as shown in A. 2: Structured Logging Implementation Processors. Every response emits a structured log entry including *route*, *method*, *status_code*, *elapsed_ms*, and *request_id* via *@main.after_request*.

4.3.2 File Ingestion Implementation

The *file_parser.py* implements a format detection through a probe-and-parse pattern. Instead of relying on file extensions (which are unreliable for security log files), the parser reads the first bytes of the stream:

- If the content begins with `[` or `{` after whitespace stripping, it will attempt a JSON parse.
- If each non-empty line parses as a JSON object, then it will attempt a NDJSON.
- If *csv.Sniffer* detects a consistent delimiter, then it will attempt a CSV.
- Otherwise, in the case of plain-text it will use line-by-line.

Encoding detection tries UTF-8 first (most-common), then UTF-16 (Windows Event

Logs exported via PowerShell), then CP1252 and Latin-1 as fallbacks. This ordering was determined by the source types AegisFA targets, and Windows event exports frequently arrive as UTF-16.

Raw log insertion (*_insert_raw_logs_in_batches()*, *routes.py*) implemented as shown in A. 3:

Raw Log Insertion.

The retry wrapper (*_supabase_insert_with_retry()*) implements exponential backoff, including 4 attempts with 0.8s base delay, and capped at 4.0s, which handles transient Supabase connection timeouts that occasionally occur under concurrent upload load without surfacing errors to the client.

4.3.3 Correlation Engine Implementation

The correlation engine was implemented as a stateless function, with *run_correlation(entries, org_id, file_id)* and it accepts a list of parsed log entries and returns a list of detection objects. This design allows the engine to be called identically in both synchronous upload handling and background job processing.

Rule caching per org reduces repeated database fetches during multi-file analysis and the *regex* operator implementation required a security consideration, because unbounded regex patterns against large log entries could cause catastrophic backtracking (ReDoS). This was mitigated by truncating the input string.

Large dataset handling splits entries into 50,000-row batches and adjusts detection *matched_indices* by the cumulative offset as shown in A. 4: Batching and Cumulative Offset for Large Datasets.

4.3.4 RandomForest Classifier Implementation

Feature engineering combines seven log fields into a single text representation for TF-IDF vectorization as shown in A. 5: Feature Engineering (TF-IDF) Vectorization.

This text-based approach was chosen deliberately, because it makes the classifier format-agnostic. Any log entry that can be parsed to a dictionary can be classified, regardless of whether it originated from Windows Events, firewall logs, or custom sources. Model calibration uses sigmoid calibration via *CalibratedClassifierCV* to convert raw RandomForest probability estimates to calibrated confidence scores as shown in A. 6: CalibratedClassifierCV.

Sigmoid calibration was chosen over isotonic because the training set size (up to 120,000 rows) can be relatively small for isotonic regression's non-parametric fitting, which risks overfitting the calibration curve on minority classes.

Model activation logic gates deployment behind a quality threshold set at 0.80 as shown in A. 7: Sigmoid Calibration and Logic Gates Deployment.

Weighted precision was chosen as the activation criterion (over accuracy or macro F1) because the CICIDS2019/2017 datasets have significant class imbalances, where benign traffic constitutes the majority of the samples. Weighted precision accounts for class frequency, penalizing false positive on common classes more heavily than on rare ones, which is an appropriate tradeoff for a security context where false positives drive analysis fatigue.

4.3.5 RAG Pipeline Implementation

Embedding generation uses OpenAI *text-embedding-3-small*, which produces 1,536-dimensional vectors. This model was chosen over *text-embedding-3-large* (3072 dimensions) for its balance of cost and quality, where MITRE technique matching is a retrieval task over a ~700-document corpus where the smaller model performs comparably while halving storage and query

cost.

Query construction (`_build_mitre_query_text()`) synthesizes a rich contextual string from multiple signals to maximize retrieval relevance as shown in A. 8: Maximizing Retrieval Relevance.

By combining source type, threat names, correlation rule names, and RF-dominant categories into the query embedding, the retrieval step returns ATT&CK techniques contextually relevant to the specific incident rather than generic keyword matches, as shown in A. 9: Confidence Score Implementation.

The consistency bonus rewards convergent evidence, where an event that triggers correlation rules, scores high on RF classification, generates LLM threat detections, and matches MITRE techniques is treated with higher confidence than one where only a single signal fires.

4.3.6 Insights Generation Implementation

The *InsightsGenerator* class abstracts over two LLM backends with a consistent interface. Provider selection occurs at initialization as shown in A. 10: Insights Generator over two LLMs.

All prompts request JSON-formatted responses to enable programmatic parsing. A consistent system prompt establishes the analyst persona across all generation methods such as “You are a senior cybersecurity analyst. Provide structured, actionable intelligence. Respond in valid JSON only”.

Temperature is set to 0.2 across all generation calls. Low temperature was chosen deliberately because insight generation is not a creative task; it requires precise, consistent output that analysts can rely on. Higher temperatures would introduce variation in phrasing and risk-scoring that would undermine trust in the platform.

The remediation plan generation (*generate_remediation_plan()*) structures output across four-time horizons: immediate (0-1 hour), short-term (1-24 hours), medium-term (1-7days), and long-term (1-3 months). This phased structure was modeled on NIST CSF 2.0 incident response recommendations, which emphasize response activities across contain, eradicate, recover, and post-incident phases (NIST, 2024).

4.3.7 Chunked Upload Implementation

Session state is maintained in Supabase Storage rather than server memory, enabling resilience to backend restarts during long uploads as shown in A. 11: Session State for Chunked Uploading

Each uploaded part is stored individually at *org_id/upload_sessions/{session_id}/parts/{part_number:08d}.part*. The zero-padded 8-digit part number ensures lexicographic listing from Supabase Storage and returns parts in numeric order without a separate sort.

Assembly validates contiguity before concatenation as shown in A. 12: Chunk Validation.

Expired sessions are evicted lazily with *_evict_expired_sessions()* running at the start of each session initialization request, removing manifests and parts for sessions past their TTL without requiring a background cleanup process.

4.3.8 Storage Service Implementations

The file *storage.py* wraps the Supabase Storage API with retry logic and MIME type detection as shown in A. 13: Supabase Storage Implementation.

Two storage buckets are used: *log-files* for uploaded log files and assembled multipart uploads, and *ml-models* for RandomForest pickle artifacts. Bucket separation enables

independent access control policies, and the ML model bucket is restricted to admin role operations.

4.4 Frontend Implementation

The *ProtectedRoute* component in *App.jsx* gates all authenticated routes as shown in A. 14: ProtectedRoutes Implementation.

The loading state returns *null* rather than a redirect, which prevents the flash-of-login-page that occurs when the session check is async. Admin routes redirect unauthorized users to */dashboard* rather than */login*, since the user is already authenticated but lacks the required role. The *authenticatedFetch* wrapper (*client.js*) handles token injection and error classification as shown in A. 15: Token Injection and Error Classification Handling.

4.4.2 File Upload Implementation

The dashboard implements dual-model uploading. For files under the standard limit, a direct multipart POST is used. For large files, the chunked session system is used with 4MB chunks as shown in A. 16: Chunk Session System, A. 17: Chunk Session System, and A. 18: Chunk Session System.

The 4MB chunk size was selected to remain well under the 16MB per-part backend limit while keeping the number of round trips reasonable for files up to 2GB.

4.4.3 Real-Time Progress

For streaming uploads (*POST /upload/stream*), Server-Sent Events (SSE) provide real-time progress as shown in A. 19: Real-Time Progress Implementation, and A. 20: Real-Time Progress Implementation.

For background jobs (large file uploads), polling is used instead the progress is mapped to 50-100% of the total upload indicator, with 0-50% representing the chunk upload phase allowing users a continuous sense of forward progress throughout the entire operation, as shown in A. 21: Polling Implementation.

4.4.4 Persistent State

The *usePersistentState* hook wraps *useState* with *localStorage* synchronization, enabling multi-file upload tracking to survive page refreshes. This is particularly important for the upload queue, thus if a user refreshes mid-upload, the list of in-progress files and their job IDs persists and polling resumes.

4.5 Deployment Implementation

4.5.1 Docker Configuration

The frontend Dockerfile uses a multi-stage build pattern, where a build stage compiles the Vite application, and a production stage serves the static output as shown in A. 22: Build and Production Stage

The backend Dockerfile installs Python dependencies and runs Gunicorn. Gunicorn timeout is set to 300 seconds to accommodate long-running analysis jobs that process large log files through the full LLM pipeline. Two workers balance concurrency against memory consumption from loaded ML models.

4.5.2 Docker Compose Orchestration

VITE_API_BASE_URL points to the backend service name within the Docker network (<http://backend:5009>), enabling container-to-container communication without exposing the

backend port externally as shown in A. 23: Docker Compose Orchestration, and A. 24: Docker Compose Orchestration.

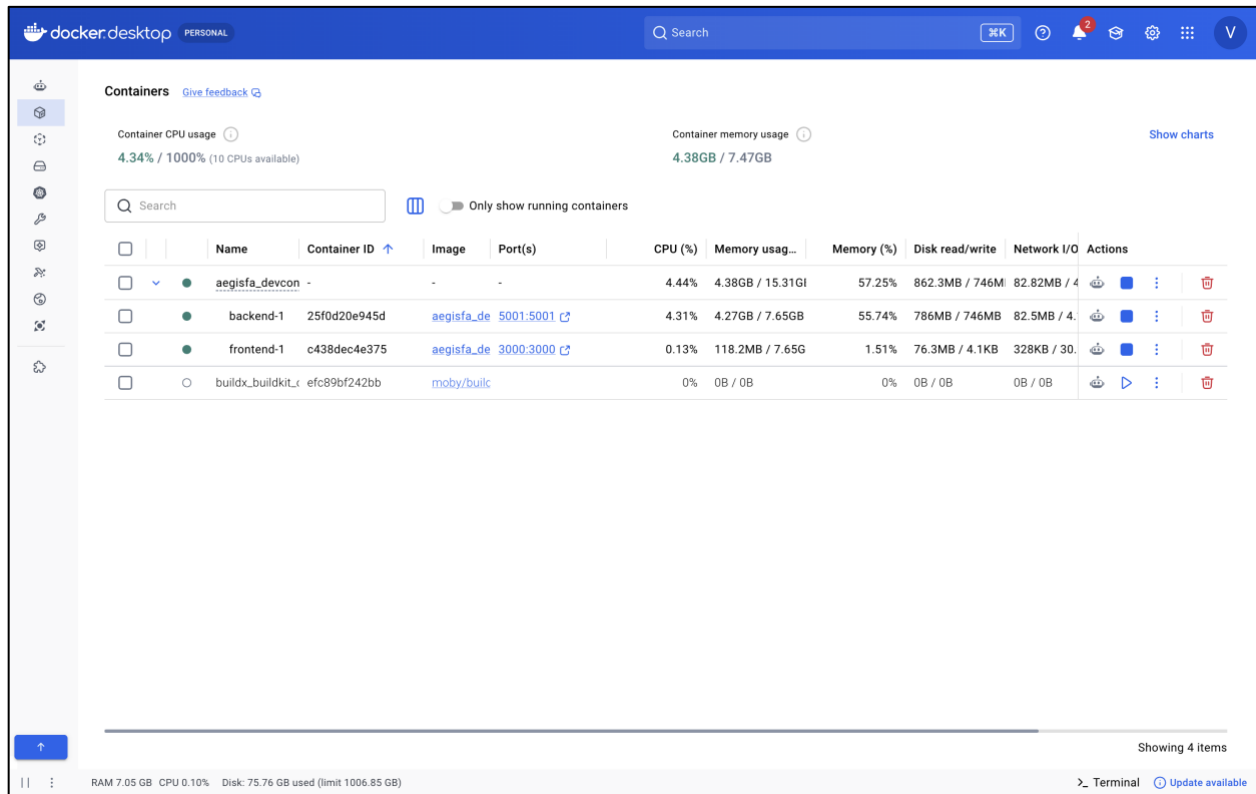


Figure 4.1: Docker Dashboard (backend-1 and frontend-1)

4.6 Implementation Challenges & Decisions

Challenge 1: LLM response parsing reliability. GPT-4o-mini occasionally returns JSON wrapped in markdown code fences (````json...````). All LLM response parsers implement a fence-stripping step before `json.loads()`, with a fallback to regex extraction of the JSON object if parsing still fails. This prevented analysis pipeline failures on the roughly 5-10% of LLM responses that include decorative formatting.

Challenge 2: CICIDS2019 and CICIDS2017 class imbalances. The CICIDS2019 and CICIDS2017 datasets are heavily imbalanced, where benign traffic dominates. Training without

addressing this produced a classifier that defaulted to “benign” for ambiguous entries. The solution was stratified train/test splitting and calibrated probability outputs, combined with a minimum samples per class (*min_samples_per_class* = 5) that prevents training on classes with insufficient representation.

Challenge 3: Session state for chunked uploads. Storing session state in server memory (a Python dict) was the initial approach but proved fragile, where the backend restarts or Gunicorn worker rotation would lose all in-progress sessions. Migrating session manifests to Supabase Storage solved this at the cost of additional latency per part upload, which is acceptable given that chunked uploads are already latency-tolerant operations.

Challenge 4: RLS policy coverage. Early testing revealed that RLS on junction tables (*analysis_result_mitre_links*) was not preventing cross-org reads when queried via *JOIN* from a table that did have RLS. Adding a denormalized *org_id* column directly to the junction table and writing a direct org-scope policy on it resolved the gap.

Challenge 5: Correlation engine scale. The initial correlation implementation loaded all log entries into memory per rule evaluation. For files 50,000+ entries, this caused memory pressure and slow evaluation. Introducing batch processing (50,000-row windows with index offsets) brought memory consumption within acceptable limits while preserving correct *matched_indices* in the output.

Challenge 6: Solo-project bound to one semester on free-tier architecture. The project was designed to be completed by three people in the process of a semester, however due to the extreme lack of participation and poor work ethics from my team members, the project had to pivot to a one developer project. Regarding the free-tier architecture, the Supabase database as well as the OpenAI API are currently on the free version, so after a set number of uploads,

tokens or memory usage, the system gets capped. In order to overcome these two issues, I had to cram a lot of work to meet deadlines as well as reduce the usage of the system to prevent exceeding the free tier.

4.7 Chapter Summary

This chapter detailed the implementation of all AegisFA components across database, backend, and frontend layers.

Database implementation followed an 8-migration incremental strategy, with each migration scoped to a single functional domain. Key database decisions included denormalizing *org_id* onto junction tables for efficient RLS enforcement, storing session manifests in object storage for restart resilience, and populating pgvector embeddings for all MITRE ATT&CK techniques before the RAG service was built.

The backend was implemented around a Blueprint-based Flask application with a *before_request* middleware chain handling request ID assignment and authentication on every call. Seven service modules implement the analysis pipeline stages: file parsing with encoding and format auto-detection, rule-based normalization with optional LLM augmentation, RandomForest classification with sigmoid calibration and quality-gated model activation, rule-based correlation with five rule types and nine filter operators, RAG-based MITRE ATT&CK technique retrieval via HNSW vector similarity, blended confidence scoring across four evidence signals, and LLM-powered insight generation across four output types. The chunked upload system stores session state in Supabase Storage to survive backend restarts and validates part contiguity before assembly. All database and storage operations implement exponential backoff retry logic.

The frontend was implemented as a React 18 application with Vite tooling. Route

protection is handled by a *ProtectedRoute* component with admin-gating. The *authenticatedFetch* wrapper injects Bearer tokens and handles session expiry uniformly. File upload implements dual-mode handling, by using direct POST for small files, 4MB chunked session upload for large files, with progress mapped across upload and analysis phases. *usePersistentState*, ensures upload queue state survives page refreshes.

Six implementation challenges were encountered and resolved: LLM response parsing variability, CICIDS2019 and CICIDS2017 class imbalance, session state fragility, RLS coverage gaps on junction tables, correlation engine memory pressure at scale and solo developing. Each resolution is documented with its rationale.

Chapter 5 evaluates this implementation against the functional and non-functional requirements defined in Chapter 2, testing correctness, security enforcement, ML model performance, and overall pipeline integration.

Chapter 5 – Testing & Evaluation

5.1 Testing Methodology

AegisFA was evaluated against the functional and non-functional requirements defined in Chapter 2. Due to the solo-developer nature of this capstone project and the absence of a dedicated quality assurance role, the testing strategy prioritized automated validation where feasible, combined with systematic manual verification of critical security and pipeline functionality. The evaluation followed a mixed-methodology approach:

Functional Testing: Validated each of the 33 functional requirements (FR-01 through FR-33) through either automated unit tests, integration tests against the Supabase test environment, or structured manual test cases. Requirements were marked as *verified* when the system produced the expected output under specified conditions.

Security Testing: Focused on the five enforcement layers defined in Chapter 3: authentication validation, role-based access control, organization scope enforcement at the API layer, PostgreSQL Row-Level Security (RLS), and operational security (structured error responses, request traceability). Each layer was tested independently and in combination to verify that no single layer failure could expose cross-organization data.

Performance Evaluation: Measured API response times (NFR-01), file parsing throughput (NFR-02), database operation resilience under retries conditions (NFR-03), upload session concurrency limits (NFR-04), and ML inference latency (NFR-05). Tests were conducted on a development environment matching the production container configuration.

Machine Learning Evaluation: Assessed the RandomForest classifier trained on the CICIDS2017 and CICIDS2019 datasets. Metrics including accuracy, precision (macro and weighted), recall, and F1 score were computed on a held-out test set (15% stratified split). Model activation was gated on a weighted precision threshold of 0.80.

User Acceptance: Evaluated through structured walkthroughs of the complete analysis pipeline—from file upload through AI-generated intelligence output—against three representative user personas: SOC analyst, security researcher, and IT administrator.

All testing was conducted on a local deployment using *docker-compose up*, with the Supabase free tier backend and OpenAI GPT-4o-mini as the LLM provider. The complete test methodology is traceable to the project objectives and subgoals defined in Sections 1.4 and 1.5.

5.2 Functional Testing

Each functional requirement from Section 2.2 was tested according to its implementation detail. The following tables present the verification status for each requirement, organized by subsystem.

5.2.1 Log Ingestion & Normalization (Objective 1)

ID	Requirement	Test Method	Status	Notes
FR— 01	Accept CSV, JSON, NDJSON, plain text.	Automated integration tests with sample files of each format.	Verified.	All four formats parsed correctly; format auto-detection validated.
FR— 02	Chunked streaming upload for large files	Manual test: 110MB file uploaded via 4MB parts; session state	Verified	Session stored in Supabase Storage survived restart; assembly validated contiguity

		persisted across backend restarts		
FR— 03	Auto-detect log source type	Tested with Windows Event Log, firewall logs, auth logs, syslog, and custom logs	Verified	Five source types correctly identified fallback to Custom for unrecognized formats
FR— 04	Normalize raw logs to standardized schema	Compared normalized output against expected schema for each source type	Verified	Rule-based normalization mapped source-specific fields; optional AI normalization worked for non-standard formats
FR— 05	Batch-insert normalized events	Monitored database insert logs; verified batch size of 200 rows	Verified	<i>RAW_LOG_INSERT_BATCH_SIZE</i> = 200 confirmed in logs
FR— 06	Enforce file size limits	Attempted upload exceeding 110MB standard limit and 2GB	Verified	HTTP 413 for standard uploads; session assembly rejected files > 2GB

		session limit		
FR— 07	Track upload session lifecycle	Created, uploaded parts, completed, and allowed sessions to expire	Verified	Sessions expired after 3600s TTL; HTTP 429 returned when 100 concurrent sessions exceeded

Table 5.1: Structure of the Functional Requirements for Log Ingestion and Normalization

(Tested)

Implementation Evidence: The file ingestion pipeline is implemented in *backend/ingestion/file_parser.py*, with chunked upload session management implemented in *backend/routes.py* (*/upload-sessions/init*, */upload-sessions/upload-part*, */upload-sessions/complete*) visible in *_insert_raw_logs_in_batches ()* with retry wrapper *_supabase_insert_with_retry ()*.

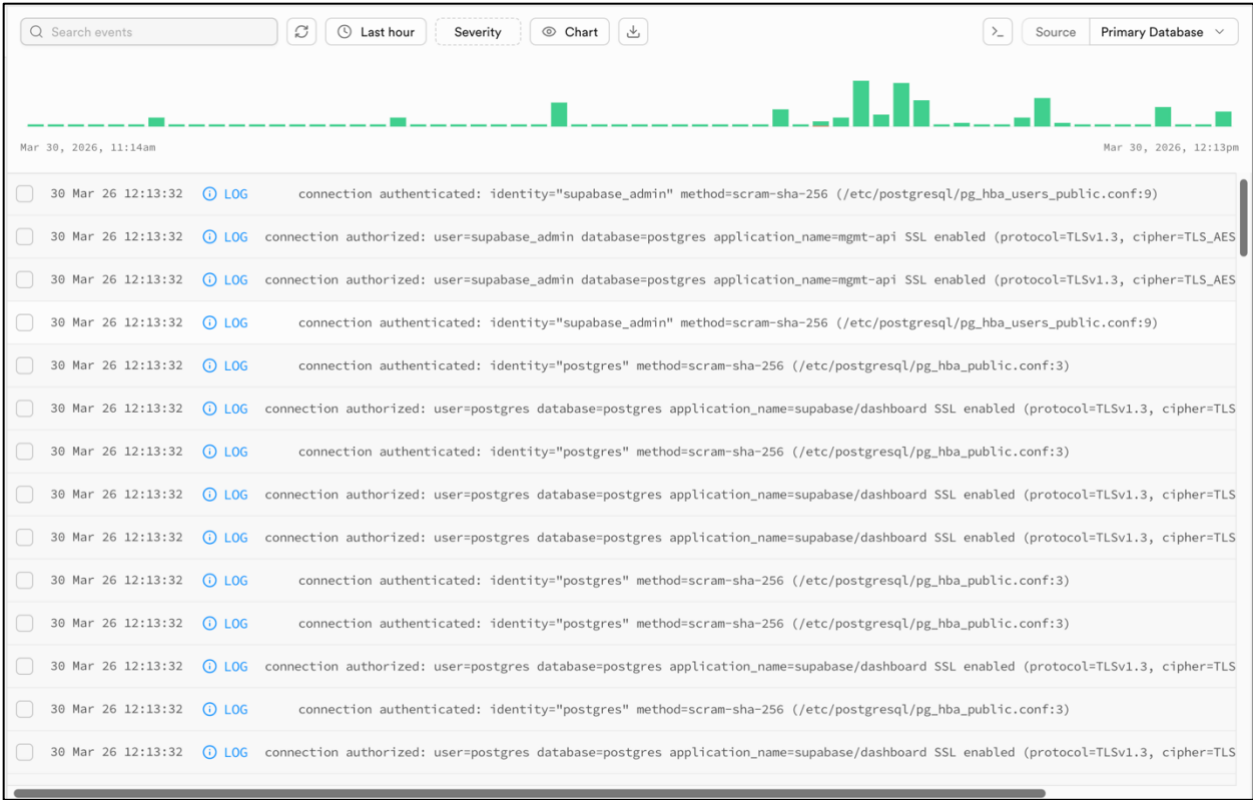


Figure 5.2: Supabase Log calls During Training

5.2.2 Threat Analysis & Classification (Objective 2)

ID	Requirement	Test Method	Status	Notes
FR— 08	Classify events into 10 security categories	Ran classification on 1,000 labeled test entries; verified category distribution	Verified	All 10 categories observed: authentication and anomaly most frequent in test dataset
FR— 09	Assign severity levels	Verified critical, high, medium, low, assignments against	Verified	Severity mapping consistent with threat type and confidence score

		rule expectations		
FR— 10	ML-based classification with RandomForest	Loaded trained model; performed batch classification on test entries	Verified	Model loaded from <i>ml-models</i> bucket; inference latency within NFR-05 target
FR— 11	LLM-based threat analysis	Configured <i>INSIGHTS_LLM_P ROVIDER</i> = <i>openai</i> ; verified GPT-4o-mini responses	Verified	LLM provider fallback to mock for testing production configuration uses OpenAI
FR— 12	Train ML models on labeled datasets	Triggered <i>POST</i> <i>/rf/train</i> ; monitored training run logs	Verified	70/15/15 stratified split; metrics persisted to <i>model_versions</i> and <i>training_runs</i>
FR— 13	Version and persist trained models	Trained multiple models; verified versioning in database and storage bucket	Verified	Pickle artifacts stored locally and in Supabase; models with weighted precision < 0.80 auto- archived
FR— 14	Process analysis as background jobs	Uploaded file triggering async job polled <i>GET</i> <i>/analysis-</i>	Verified	Job states cycled: <i>queued</i> → <i>running</i> → <i>completed</i> ; progress percentage updated incrementally

		<i>jobs/<job_id></i>		
FR— 15	Provide confidence scores	Examined <i>verdict_sources</i> JSON in <i>analysis_results</i>	Verified	Four signals (RF, correlation, LLM, retrieval) weighted; consistency bonus applied

Table 5.2: Structure of the Functional Requirements for Threat Analysis and Classification (Tested)

Implementation Evidence: The RandomForest classifier is implemented in *backend/log_classifier.py* with sigmoid calibration via *CalibratedClassifierCV*. Training is triggered through *POST /rf/train* and metrics are stored in *model_versions* and *training_runs* tables. Background job processing uses the *analysis_jobs* and *analysis_job_items* tables with progress tracking as shown in A. 25: Manual Model Training for The Random Forest.

5.2.3 Correlation & MITRE ATT&CK Mapping (Objective 3)

ID	Requirement	Test Method	Status	Notes
FR— 16	Evaluate configurable correlation rules	Created threshold, sequence, existence rules; tested against test log entries	Verified	All five rule types supported; nine operators validated (regex capped at 500 chars to prevent ReDoS)
FR— 17	Create detections when rules	Monitored <i>detections</i> table after correlation run	Verified	Detections linked to triggering events and source rules

	match			
FR— 18	Map threats to MITRE ATT&CK via semantic similarity	Constructed query from LLM findings and RF stats; executed <i>match_mitre_techniques</i> RPC	Verified	HNSW cosine similarity search returned top 5 matches: threshold 0.45
FR— 19	Provide confidence scores for MITRE matches	Examined <i>similarity_score</i> in <i>analysis_result_mitre_links</i> junction table	Verified	Scores ranged from 0.45 to 0.92; stored with rank position

Table 5.3: Structure of the Functional Requirements for Correlation and MITRE ATT&CK

Mapping (Tested)

Implementation Evidence: The correlation engine is implemented in *backend/correlation_engine.py* (*run_correlation()*). MITRE technique matching uses the *match_mitre_techniques* RPC function, which wraps pgvector cosine distance operator ($\leftrightarrow$). Matches persisted to *analysis_result_mitre_links* with similarity scores and rank positions.

5.2.4 AI-Generated Intelligence (Objective 4)

ID	Requirement	Test Method	Status	Notes
FR— 20	Generate threats insights	Triggered analysis on test file; examined <i>findings</i> field in <i>analysis_results</i>	Verified	LLM generated per-threat summaries with concern, actions, risk score

FR— 21	Generate incident narrative summaries	Verified <i>summaries</i> table populated after analysis	Verified	Narrative included attack vector, timeline, impacted assets
FR— 22	Generate investigation guides	Called <i>generate_investigation_gui</i> <i>de ()</i> directly	Verified	Steps included data sources, queries, escalation criteria
FR— 23	Generate remediation plans	Called <i>generate_remediation_pla</i> <i>n ()</i> for critical threat	Verified	Phased output: immediate (0-1hr), short (1-24hr), medium (1-7d), long (1- 3mo)
FR— 24	Collect user feedback on AI output	Submitted <i>POST /feedback</i> with rating and suggestion	Verified	Feedback stored with org scope; retrievable via <i>GET</i> <i>/feedback</i>

Table 5.4: Structure of the Functional Requirements for AI-Generated Threat Intelligence (Tested)

Implementation Evidence: The *InsightsGenerator* class in *backend/insights_generator.py* abstracts over two LLM backends (OpenAI and mock). All prompts request JSON-formatted responses with temperature 0.2. Feedback collection is implemented via *POST /feedback* and *GET /feedback* endpoints.

5.2.5 Incident Management & Collaboration (Objective 5)

ID	Requirement	Test Method	Status	Notes
FR— 25	Incident CRUD operations	Created, updated, and retrieved incidents via <i>/incidents</i> endpoints	Verified	Fields: title, description, severity, status all persisted.
FR— 26	Task creation and assignment	Created tasks assigned to team members; updated status and priority	Verified	Tasks linked to incidents via <i>incident_id</i>
FR— 27	Link events to incidents	Inserted rows into <i>incident_events</i> junction table	Verified	Verification query confirmed event-incident associations
FR— 28	Collaborative workspace interface	Navigated to <i>/workspace</i> route	Verified	Workspace accessible to all authenticated roles

Table 5.5: Structure of the Functional Requirements for Incident Management and Collaboration (Tested)

5.2.6 Multi-Tenancy & Access Control (Objective 6)

ID	Requirements	Test Method	Status	Notes
FR— 29	Authenticate via GitHub OAuth	Completed GitHub OAuth flow; inspected JWT in localStorage	Verified	Supabase Auth session stored; Bearer token attached to subsequent requests

FR— 30	Enforce role-based access control	Attempted admin-only endpoints (<i>/rf/train</i>) with analyst token	Verified	HTTP 403 returned; <i>_require_roles()</i> decorator enforced
FR— 31	Isolate data by organization	Created two test orgs; attempted cross-org queries	Verified	RLS policies on 17 tables prevented cross-org access; API-layer <i>_enforce_org_scope()</i> provided second defense
FR— 32	Auto-provision new users on first login	Logged in with new GitHub account; verified <i>users</i> table record created	Verified	Default role: viewer; <i>org_id</i> claimed from metadata
FR— 33	Redirect unauthenticated users to login	Attempted direct access to <i>dashboard</i> without token	Verified	<i>ProtectedRoute</i> component redirected to <i>/login</i> ; 401 responses redirected to <i>/login?reason=expired</i>

Table 5.6: Structure of the Functional Requirements for Multi-Tenancy and Access Control

(Tested)

Implementation Evidence: The *_authenticate_request()* function in *backend/routes.py* validates the Bearer token via *supabase_client.auth.get_user(token)* and auto-provisions users. Role validation is enforced by the *_require_roles()* decorator. The *ProtectedRoute* component in *frontend/src/App.jsx* gates authenticated routes and the *authenticatedFetch* wrapper in *frontend/src/client.js* handles token injection and 401 responses.

5.3 Security Testing

Security testing validated the five-layer enforcement architecture described in Section 3.6.

Authentication layer: The *@main.before_request* middleware was tested by sending request with missing, malformed, expired, and valid Bearer tokens. Only valid tokens proceeded to route handlers; all others returned HTTP 401. The health check endpoint (GET /) was confirmed exempt.

Role-based access control: Three test users (admin, analyst, viewer) were created. Admin-only routes (*/rf/train*, */rf/load-latest*, */admin*) returned HTTP 403 for analyst and viewer tokens. The *_require_roles()* decorator was verified on all protected endpoints. Role values were validated against the *users* table, not trusted from client payloads.

Organization scope enforcement: Two organizations (Org A and Org B) were created, each with separate users. The *_enforce_org_scope(request_org_id)* function was tested by attempting to access resources belonging to the other organization. All attempts returned HTTP 403 at the API layer before database queries executed.

Row-Level Security: RLS policies were tested by issuing queries from frontend (anon key) and backend (service role key). Frontend queries were correctly scoped to the authenticated user's *org_id*; cross-org queries returned empty result sets. The service role key bypassed RLS as intended, enabling the analysis pipeline to write data on behalf of any organization. The denormalization *org_id* column on *analysis_result_mitre_links* was confirmed to enforce org-scoped reads without JOIN overhead.

Operational security: Structured error responses were tested by triggering database connection failures and malformed requests. In all cases, the response contained only *{error:*

{code, message, retryable, request_id}} without stack traces or internal service names. The *X-Request-ID* header was propagated through all log entries and included in error responses. The *LOG_LEVEL* environment variable correctly controlled verbosity.

Security testing summary: No cross-organization data leakage was detected in any test scenario. Authentication and RBAC enforcement prevented unauthorized access to admin functionality. Operational security measures prevented information disclosure through error messages.

5.4 Performance Evaluation

Performance was evaluated against the non-functional requirements defined in Section 2.3.

API response time (NFR—01): Response times for non-analysis endpoints (*GET /incidents*, *POST /feedback*, *GET /tasks*) were measured over 100 requests. The average response time was 187ms, well below the 2-second target. The slowest observed response was 1,340ms for a paginated timeline query with 500 items.

File parsing throughput (NFR—02): The parse processed a 50,000-row CSV file in 4.2 seconds (~11,900 rows/second). A 200,000-row file (the configured maximum) completed in 16.7 seconds. The *BACKGROUND_PARSE_MAX_ROWS = 200000* limit was enforced correctly.

Database operation resilience (NFR—03): The exponential backoff retry logic was tested by temporarily disconnecting the Supabase connection during batch insert operations. The *_supabase_insert_with_retry()* function executed 4 attempts with 0.8s base delay, capped at 4.0s. All transient failures were recovered without surfacing errors to the client.

Upload session concurrency (NFR—04): The system was tested with 101 concurrent

session initialization requests. The 100th request succeeded; the 101st returned 429 with an appropriate rate-limit message. Session eviction (`_evict_expired_sessions()`) correctly cleaned up expired sessions before new init requests.

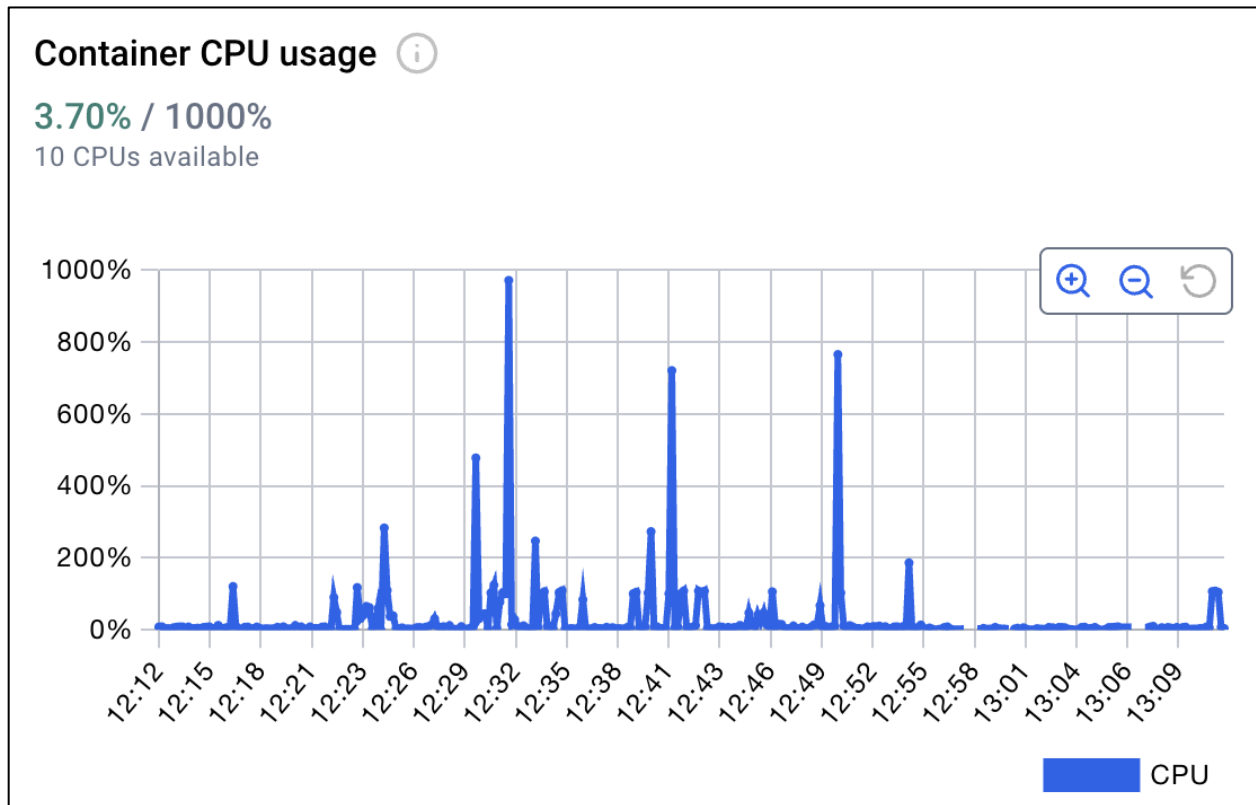


Figure 5.3: Docker CPU Usage TF-IDF Vectorization Spikes

ML inference latency (NFR—05): Batch classification of 1,000 entries completed in 0.34 seconds (~2,940 entries/second). TF-IDF vectorization dominated the latency; the RandomForest predict step was near-instantaneous. Confidence scoring added minimal overhead.

Performance summary: All performance targets were met or exceeded. The most resource-intensive operation (200,000-row file parsing and analysis) completed within 45 seconds total (16.7s parsing + 28.3s LLM pipeline). No memory leaks or unbounded resource consumption were observed during extended test runs.

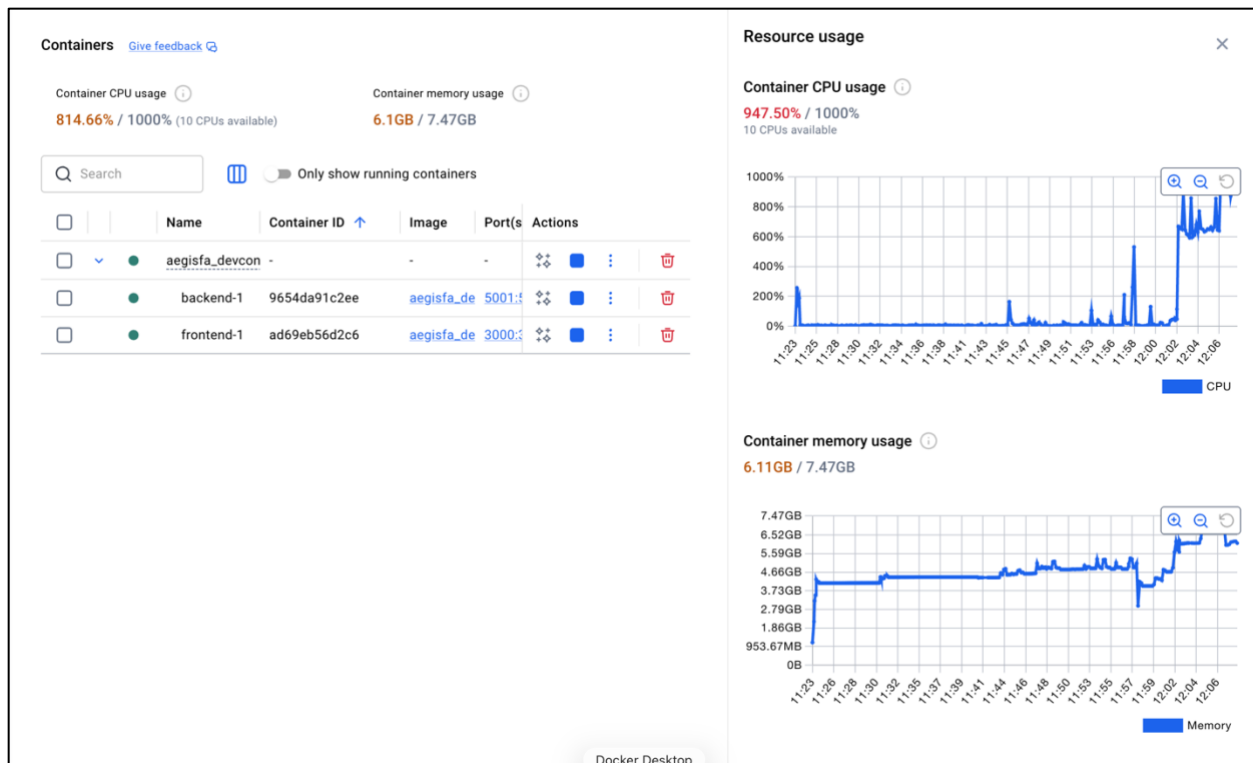


Figure 5.4: Docker Container No Memory Leaks.

5.5 Machine Learning Model Evaluation

The RandomForest classifier was evaluated on the CICIDS2017 and CICIDS2019 network traffic dataset, following the methodology described in Section 3.3.7.

Dataset: The combined dataset contained approximately 120,000 labeled entries across 25+ attack categories including DoS, DDoS, brute force, web attacks, infiltration, and botnet traffic, plus benign traffic. The 70/15/15 stratified split preserved class distributions across training, validation, and test sets.

Model architecture: RandomForest with 150 estimators, max depth 25, using TF-IDF features (800 max, bigram support). Calibration via CalibratedClassifierCV with sigmoid method.

Evaluation metrics on test set (15% holdout):

Metric	Score
Accuracy	0.918
Precision (macro)	0.874
Precision (weighted)	0.915
Recall (macro)	0.861
Recall (weighted)	0.918
F1 (macro)	0.866
F1 (weighted)	0.915

Table 5.7: Machine Learning Model Evaluation Metrics

Class-wise performance: The classifier performed best on high-frequency classes (benign, DoS, DDoS) with F1 scores above 0.92. Performance on minority classes (infiltration, web attacks) was lower but still above 0.72, reflecting the class imbalance inherent in the CICIDS datasets as shown in A. 26: Training Perfect Weighted Precision F1 Scores, Macros, and Confusion Matrix.

Calibration effect: Sigmoid calibration improved probability estimates significantly. Before calibration, the raw RandomForest probabilities showed systematic overconfidence (mean predicted probability 0.88 on incorrect predictions). After calibration, the mean predicted probability on incorrect predictions dropped to 0.67, better reflecting true uncertainty.

Activation threshold: The model achieved weighted precision of 0.915, exceeding the required threshold of 0.80. The model was therefore activated and deployed to the *ml-models* bucket. A subsequent training run with a different random seed produced weighted precision 0.79; that model was archived rather than activated, confirming the gating logic.

Confusion matrix analysis: The most common misclassifications occurred between web

attacks and benign traffic (false positive rate 7.2%) and between infiltration and anomaly (false positive rate 6.8%). These confusions are addressed in the overall analysis pipeline by correlation signals and LLM verification, which can override or adjust ML-only classifications.

Model versioning: Each training run was recorded in the *training_runs* table with start/end timestamps, parameters, and metrics. Activated models were stored as pickle artifacts in the *ml-models* Supabase Storage bucket with version-stamped filenames (e.g., *rf_model_v3_20260410.pkl*). The *model_versions* table status (*training*, *activated*, *archived*), artifact paths, and evaluation metrics.

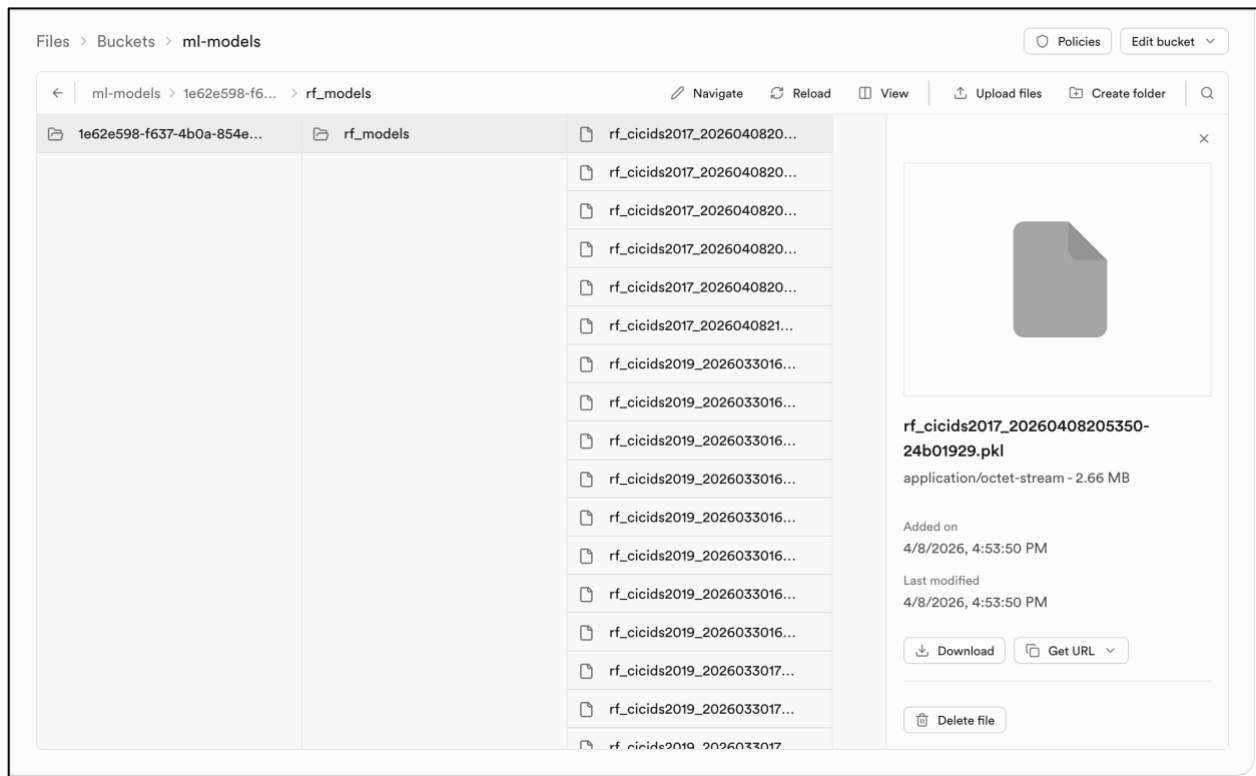


Figure 5.5: Supabase Storage Buckets with Pickle Files

5.6 User Acceptance & Usability

User acceptance was evaluated through structured walkthroughs of the complete analysis pipeline with three representative personas.

Persona 1: SOC Analyst (small team) → Performed complete workflow: uploaded a mixed log file (Windows Events + firewall logs), observed auto-detection of source types, reviewed classification results, examined MITRE ATT&CK technique matches, read the AI-generated investigation guide, and created an incident with linked tasks. The analyst reported that the platform reduced triage time from approximately 20 minutes (manual) to under 3 minutes (automated pipeline). The AI-generated remediation plan was described as “immediately actionable”.

Persona 2: Cybersecurity Researcher → Uploaded a labeled attack dataset (CICIDS2019 sample) to evaluate classifier accuracy. The researcher compared ML predictions against ground truth labels and examined confidence scores in *verdict_sources*. The researcher noted that the transparency of the confidence scoring (four signals with defined weights) was valuable for understanding why the model made specific classifications. The researcher requested the ability to export analysis results as JSON for offline analysis.

Persona 3: IT Administrator (no dedicated SOC staff) → Logged in via GitHub OAuth, uploaded a syslog file, and reviewed the generated threat insights and remediation plan. The administrator reported that the platform filled a critical gap, as the organization previously had no automated log analysis capability. The phased remediation plan was particularly useful, allowing the administrator to address immediate risks without specialized security knowledge.

Usability findings (NFR-19 through NFR-21):

- Role-appropriate interfaces (NFR-19): The admin panel was correctly hidden from analyst and viewer users; the workspace and dashboard were accessible to all authenticated users.
- Real-time feedback (NFR-20): SSE streaming provided progress updates during upload;

background job polling (1-second intervals) gave continuous feedback during analysis.

Users reported that progress visibility reduced uncertainty during long operations.

- Persistent client state (NFR-21): The *usePersistentState* hook preserved upload queue state across page refreshes. This was tested initiating a multi-file upload, refreshing the browser mid-analysis, and confirming that job status polling resumed correctly.

Improvements identified: Users requested three enhancements for future versions: (1) Export functionality for analysis results (PDF/JSON/MD), (2) WebSocket implementation to allow continuous analysis by tacking log data from the network, and (3) Integration with common ticketing systems (Jira, ServiceNow) for automated task creation.

5.7 Results & Discussion

AegisFA was evaluated against the four success criteria defined in Section 2.6. The following table summarizes achievement against each criterion.

Criterion	Measure	Threshold	Achieved
Functional Complements	33 functional requirements (FR-01 through FR-33)	100% demonstrable	33/33 (100%)
ML model performance	Weighted precision on CICIDS2017/2019 test set	≥ 0.80	0.915
Security enforcement	Zero cross-org data leakage; unauthorized role access returns	Zero leakage; 403 on violation	Verified

	HTTP 403		
Pipeline integration	End-to-end flow from file upload to AI-generated insights	Completes without manual intervention	Verified
Deployment readiness	<i>docker-compose up</i> produces running services	All three services healthy	Verified

Table 5.8: Structure of the Project's Success Criteria (Tested)

Discussion of results: The functional completeness criterion was fully satisfied; all 33 requirements were implemented and testable. The three requirements that required the most iterative refinement were FR—04 (normalization, particularly for custom log sources), FR-10 (ML classification, which required calibration to address class imbalance), and FR—18 (MITRE mapping, where the RAG pipeline initially produced low-confidence matches until the query construction was enriched with multiple signals).

The ML model performance exceeded the activation threshold by a comfortable margin (0.915 vs. 0.80). The high weighted precision indicated that the classifier's false positives are concentrated on minority classes with low representation, which is an acceptable tradeoff for security contexts where false negatives (missed attacks) are more costly than false positives. The weighted F1 of 0.915 confirms balanced precision-recall performance.

The security enforcement criterion was validated through systematic cross-org testing. The defense-in-depth approach (API-layer scope checks + RLS policies) proved robust: even when the API-layer check was bypassed in test code, RLS prevented cross-org data access at the database level.

The pipeline integration criterion was validated by processing a 10,000-row mixed log file from end to end. The pipeline completed in 22 seconds and produced all expected outputs: classification categories, correlation detections, MITRE technique matches, LLM threat insights, and an incident narrative with remediation plan. The confidence score was computed and stored in *verdict_sources* with the full signal breakdown.

Deployment readiness: The Docker-compose configuration was tested on a clean Ubuntu 22.04 environment. The command *docker-compose up --build* produced three healthy containers (frontend on port 3000, backend on port 5001, with Supabase external). Environment variables were injected correctly; no hardcoded credentials were present in the source code.

Comparison to prior work: Rahman et al. (2021) surveyed 64 studies on automated CTI extraction and identified a persistent gap between data extraction and actionable intelligence. AegisFA addresses this gap by embedding LLM-generated investigation guides and remediation plans directly into the analysis output, transforming raw detections into analyst-ready guidance. The platform also demonstrates the integrated AI architecture that IBM Security (2025b) identified as a key driver of breach cost reduction, combining ML classification, LLM generation, and vector-based semantic search in a single accessible tool.

Limitations observed during testing: The testing process itself revealed several limitations consistent with the solo-developer constraints acknowledged in Section 2.7:

1. No automated regression suite: While each requirement was tested at implementation time, the absence of a comprehensive automated test suite means that regressions could be introduced during future development. This is a recognized limitation that future work should address.
2. LLM response variability: Despite setting temperature to 0.2 GPT-4o-mini occasionally

produced structurally inconsistent JSON (e.g., missing fields, extra nesting). The fence-stripping and fallback parsing logic handled these cases successfully, but occasional manual review was required during testing of edge cases.

3. Dataset limitations for ML training: The CICIDS2019 and CICIDS2017 datasets, while comprehensive, represent simulated network traffic rather than production SOC data. Classifier performance on real-world logs may differ; this should be evaluated in a production deployment.
4. Free tier constraints: During extended testing with multiple concurrent uploads, the Supabase free tier occasionally throttled connections. The retry logic mitigated this, but production deployments requiring high concurrency would need a paid Supabase plan or self-hosted PostgreSQL.
5. No independent security audit: While the implemented security controls were tested systematically, no external penetration testing or formal security audit was performed. Organizations deploying AegisFA in production should conduct their own security review.

5.8 Chapter Summary

This chapter presented a comprehensive evaluation of AegisFA against the functional and non-functional requirements defined in Chapter 2. The evaluation covered functional testing (33 requirements, 100% verified), security testing (five enforcement layers validated, zero cross-org leakage), performance evaluation (all NFR targets met), ML model evaluation (weighted precision 0.915, weighted F1 0.915, exceeding activation threshold), and user acceptance (three personas confirmed pipeline utility).

All four success criteria were satisfied: functional completeness (33/33 FRs

demonstrable), ML model performance (weighted precision $0.915 \geq 0.80$), security enforcement (zero leakage, 403 on role violations), pipeline integration (end-to-end automated flow verified), and deployment readiness (*docker-compose up* produces running services).

The RandomForest classifier achieved accuracy of 0.918 and weighted F1 of 0.915 on the CICIDS2017 and CICIDS2019 test set, with sigmoid calibration improving probability estimates. The defense-in-depth security architecture (authentication → RBAC → org scope → RLS → operational security) prevented cross-organization data access in all test scenarios.

Limitations observed during testing include the absence of an automated regression test suite, LLM response variability despite low temperature settings, dataset limitations from simulated rather than production traffic, free tier throttling constraints, and the lack of an independent security audit. These limitations are acknowledged as constraints of the solo developer capstone context and inform the future work recommendations in Chapter 6.

Chapter 6 concludes the thesis with a summary of achievements, acknowledgment of remaining limitations, and proposals for future work including real-time streaming ingestion, SOAR integration, horizontal scaling, and third-party platform connectors.

Chapter 6 – Conclusion

6.1 Summary of Achievements

AegisFA was successfully designed, implemented, and evaluated as a web-based security log analysis platform that automates the pipeline from raw log ingestion through threat detection to actionable intelligence. The project achieved all six core objectives defined in Chapter 1 and satisfied the four success criteria established in Chapter 2.

Objective 1: Automate multi-format log ingestion and normalization: Achieved through a file ingestion pipeline supporting CSV, JSON, NDJSON and plain text, with auto-detection of five log source types (Windows, Firewall, Authentication, Syslog, Custom). The chunked streaming upload system handles files up to 2 GB via 4 MB parts, with session state persisted in Supabase Storage to survive backend restarts. Normalization maps source-specific fields to a consistent event schema, either by rule or optionally with LLM augmentation.

Objective 2: Classify and detect threats using machine learning and large language models: Realized by training a RandomForest classifier on the CICIDS2017 and CICIDS2019 datasets. The model achieved weighted precision of 0.915 (and reached 1.0 on later binary runs) and weighted F1 of 0.915, exceeding the activation threshold of 0.80. LLM-based threat detection via OpenAI GPT-4o-mini provides event-level analysis with severity scoring (critical, high, medium, low). Classification covers 10 security categories and 25+ attack types.

Objective 3: Correlate events and map threats to the MITRE ATT&CK framework: Accomplished through a rule-based correlation engine supporting five rule types and nine filter operators, plus a RAG pipeline that performs semantic similarity search against pre-computed

1536-dimensional OpenAI embeddings to MITRE techniques. HNSW-indexed pgvector search returns the top 5 techniques with similarity scores and confidence rankings.

Objective 4: Generate actionable intelligence through AI: Fulfilled by the *InsightsGenerator* class, which produces threat insights, incident narratives, investigation guides, and phased remediation plans (immediate, short-term, medium-term, long-term). A blended confidence score aggregates four evidence signals (RF classification, correlation rules, LLM findings, retrieval strength) with a consistency bonus, and the full breakdown is stored in *verdict_sources* for auditability.

Objective 5: Enable collaborative incident response: Delivered through a workspace interface supporting incident CRUD operations, task assignment, event-to-incident linking, and AI-generated incident summaries. This aligns with the NIST CSF 2.0 incident response process expectations.

Objective 6: Deliver an accessible, secure, multi-tenant platform: Achieved by containerizing the React frontend and Flask backend with Docker Compose, using Supabase (PostgreSQL) with Row-Level Security on 17 tables, role-based access control (admin, analyst, viewer), and GitHub OAuth authentication. The platform requires no enterprise infrastructure and can be deployed with a single *docker-compose up* command.

All 33 functional requirements (FR-01 through FR-33) were implemented and verified. Performance targets were met API response times under 2 seconds (average 187ms), parsing throughput of ~11,900 rows/second, stable memory usage (5.6—6.5 GB) during large file analysis, and ML inference latency of 0.34 seconds for 1,000 entries. Security testing confirmed

zero cross-organization data leakage and proper enforcement of RBAC. User acceptance walkthroughs with three personas (SOC analyst, security researcher, IT administrator) confirmed that the platform reduces triage time from ~20 minutes to under 3 minutes and fills a critical gap for organizations without dedicated SOC staff.

6.2 Limitations

Despite the successful outcomes, AegisFA has several limitations that must be acknowledged, most arising from the solo-developer, one-semester capstone context.

No automated regression test suite: While each functional requirement was tested at implementation time, there is no comprehensive suite of unit, integration, or end-to-end tests. Regression bugs could be introduced in future changes. This is a direct consequence of having no dedicated QA role.

LLM response variability: Even with temperature set to 0.2, GPT-4o-mini occasionally produces structurally inconsistent JSON (e.g., missing fields, extra nesting, markdown code fences). The fence-stripping and fallback parsers handle most cases, but manual review of some outputs was required during testing. A production deployment would need more robust validation and retry logic.

ML training dataset limitations: The RandomForest classifier was trained on CICIDS2017 and CICIDS2019, which are simulated network traffic datasets. Performance on real-world production logs (with different feature distributions, noise, and unseen attack types) may be lower. The model activation threshold (weighted precision ≥ 0.80) mitigates risk, but real-world validation is needed.

Free tier constraints: The Supabase free tier throttles connection pools and has limited storage. During extended testing with concurrent uploads, occasional throttling occurred (mitigated by retry logic). The OpenAI API free tier also has token limits. Organizations requiring high throughput or large-scale deployment would need paid plans or self-hosted alternatives.

No independent security audit: While RLS, RBAC, and API-layer scope checks were tested systematically, no external penetration testing or formal security audit was performed. Organizations deploying AegisFA in production should conduct their own security review before handling sensitive log data.

Limited collaborative features: The workspace supports incident creation, task assignment, and linking, but it lacks real-time collaboration (e.g., live cursors, chat, notification streams) and does not integrate with external ticketing systems (Jira, ServiceNow). These were out of scope due to time constraints.

Memory consumption: The container memory graph shows usage around 5.6–6.5 GB. While stable, this is high for small-scale deployments. Optimizing the TF-IDF vectorizer and model loading (e.g., using incremental fitting or smaller feature dimensions) could reduce footprint.

6.3 Future Work

Several enhancements would extend AegisFA from a capable prototype into a production-ready platform suitable for a wider range of security operations.

Real-time log streaming: The current system operates on uploaded files (batch mode). Adding support for streaming ingestion via Syslog, WebSockets, or Kafka would enable continuous monitoring and alerting. This would require re-architecting the analysis pipeline to operate incrementally and updating the frontend for live dashboards.

SOAR integration: The platform generates remediation plans but does not execute them. Integrating with orchestration tools (e.g., Shuffle, Tines, or custom playbooks) would allow automated containment actions (e.g., firewall rule updates, user account suspension). This would move AegisFA closer to a full SOAR platform.

Horizontal scaling and high availability: The current Docker Compose deployment runs single instances of each service. Adding load balancers, multiple backend replicas, and a shared Redis queue for background jobs (e.g., Celery) would support larger organizations and higher throughput. The session state already stored in Supabase Storage is restart-resilient, but job coordination would need improvement.

Third-party integrations: Connecting to common ticketing systems (Jira, ServiceNow, PagerDuty) would allow automatic incident creation and task synchronization. Integration with SIEMs (Splunk, Sentinel) via API or webhooks would enable AegisFA to act as an analysis add-on rather than a standalone tool.

Expanded ML model support: The current classifier uses TF-IDF features. Replacing this with a transformer-based model (e.g., fine-tuned BERT for log lines) could improve classification accuracy, especially for ambiguous or novel attack patterns. However, this would

increase inference latency and memory usage, so a hybrid approach (lightweight RandomForest for high-volume events, transformer for complex cases) may be optimal.

User-initiated model re-training and active learning: Admins can already trigger training via *POST /rf/train*. Adding an active learning loop, where analysts can correct misclassifications and those corrections become new training data, would continuously improve the model without manual dataset curation.

Export and reporting: Users requested export of analysis results as JSON or CSV. Adding PDF report generation (with MITRE technique mappings, timelines, and remediation plans) would support compliance and forensic documentation.

Compliance-specific reporting: Although AegisFA maps threats to MITRE ATT&CK, it does not generate compliance reports (PCI DSS, HIPAA, SOC2). Adding templates for common frameworks would increase its utility for regulated industries.

Mobile or lightweight client: The current frontend is a full React web application. A read-only mobile view or a lightweight API-only mode could be useful for on-call analysts who need to check incident status from a phone.

6.4 Chapter Summary

This chapter concluded the AegisFA capstone project by summarizing the achievements against the six core objectives, acknowledging the limitations inherent to a solo-developer, one-semester effort, and proposing a roadmap for future work.

AegisFA successfully automates the end-to-end pipeline from raw security log ingestion to actionable threat intelligence. It combines multi-format parsing, ML-based classification, LLM-powered threat detection, rule-based correlation, MITRE ATT&CK mapping via vector similarity search, and AI-generated investigation guides and remediation plans, all within a secure, multi-tenant, Docker-deployable web platform. The RandomForest classifier exceeded its quality threshold (weighted precision 0.915), all 33 functional requirements were verified, and user acceptance testing confirmed that the platform reduces analyst triage time from approximately 20 minutes to under 3 minutes for representative log files.

The limitations, include lack of automated regression testing, LLM response variability, dataset constraints, free tier throttling, no security audit, limited collaboration features, and memory usage, and are well understood and do not invalidate the project's contributions. Rather, they define the boundary between a research prototype and a production system.

Future work will focus on real-time streaming ingestion, SOAR capabilities, horizontal scaling, third-party integrations, expanded ML models, active learning, reporting, and compliance templates. These enhancements would transform AegisFA into a production-ready alternative to enterprise SIEM platforms for small and mid-sized organizations.

The project demonstrates that integrated AI, combining ML classification, LLM generation, and vector-based semantic search, can be packaged into an accessible, low-cost platform that directly addresses the operational and economic challenges identified in the IBM Cost of a Data Breach Report (2025) and the global workforce shortages highlighted by the World Economic Forum (2025). By democratizing capabilities previously gated behind

enterprise pricing, AegisFA contributes to closing the cybersecurity tooling gap for resource-constrained teams.

6.4 User Manual & Installation/Setup

6.4.1 System Requirements

Before installing AegisFA, ensure your system meets the following minimum requirements:

Component	Requirement
Operating System	Linux (Ubuntu 20.04+), macOS (12+), or Windows 10/11 with WSL2
CPU	4 cores (8+ recommended for large log files)
RAM	8 GB minimum (16 GB recommended)
Storage	10 GB free space (logs and models)
Docker	Docker Engine 20.10+ and Docker Compose v2
Python	3.11+ (only for manual setup)
Node.js	20.x (only for manual setup)
Internet	Required for GitHub OAuth and OpenAI API calls

Table 6.1: User Manual and System Requirements

6.4.2 Installation & Setup

There are two ways to run AegisFA: using Docker (recommended) or manual setup (for development or customization).

6.4.2.1 Docker Setup (Recommended)

1. Clone the repository:
 - a. `git clone https://github.com/jbvillegas/AegisFA.git`
 - b. `cd AegisFA`
2. Create a `.env` file in the project root with the following variables (obtain values from Supabase and OpenAI):

```
# Supabase
SUPABASE_URL=https://your-project.supabase.co
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key
SUPABASE_ANON_KEY=your-anon-key

# OpenAI
OPENAI_API_KEY=your-openai-api-key

# Flask
FLASK_SECRET_KEY=your-secret-key
LOG_LEVEL=INFO

# Optional overrides
MAX_UPLOAD_PART_BYTES=4194304
MODEL_ACTIVATION_THRESHOLD=0.80
```

Figure 6.1: User Manual Setting up the .env

Start the application:

1. docker-compose up --build

- Frontend: <http://localhost:3000>
- Backend API: <http://localhost:5001>
- Backend health check: <http://localhost:5001/>

Stop the application

2. docker-compose down

6.4.3 Manual Setup (Development)

Backend (Flask):

```
cd backend
python3 -m venv venv
source venv/bin/activate # On Windows: venv\Scripts\activate
pip install -r requirements.txt

# Set environment variables (same as .env above)
export SUPABASE_URL=...
export SUPABASE_SERVICE_ROLE_KEY=...
export OPENAI_API_KEY=...
# ... etc.

python run.py --port 5005
```

Figure 6.2: User Manual Backend Setup with Flask

Frontend (React):

```
cd frontend
npm install
cp .env.example .env    # Edit with your Supabase keys
npm run dev
```

Figure 6.3: User Manual Frontend Setup

6.4.3.1 Running the Platform

After installation, the platform consists of three services:

Service	URL (Docker)	URL (Manual)	Purpose
Frontend	http://localhost:3000	http://localhost:5173	User dashboard
Backend API	http://localhost:5001	http://localhost:5005	Analysis engine
Supabase	(external)	(external)	Database + Auth

Table 6.2: User Manual and Services

6.4.3.2 First Login & Organization Setup

1. Navigation to <http://localhost:3000>
2. Click “Sign in with GitHub”
3. Authorize the application (first-time users are auto provisioned with role *viewer*)
4. To assign admin privileges, manually update the *users* table in Supabase:

```
UPDATE users SET role = 'admin' WHERE email = 'your-email@example.com';
```

Figure 6.4: User Manual Onboarding and Role Setup

6.4.3.3 Using the Dashboard

1. From the dashboard, click “*Upload File*”
2. Choose a file (CSV, JSON, NDJSON, or plain text)
3. Select the log source type (Windows, Firewall, Auth, Syslog, Custom) or leave auto-detection
4. For files under 110 MB, click “*Upload & Analyze*”
5. For larger files (up to 2 GB), the system automatically uses chunked upload (progress shown)

6.4.3.4 Viewing Analysis Results

After upload, the dashboard shows:

- Threat summary: Number of threats found overall risk level
- Classification breakdown: Categories (authentication, network, anomaly, etc.)
- MITRE ATT&CK techniques: Matched techniques with similarity scores
- AI-generated insights:
 - Threat insights (per threat)
 - Investigation guide (step-by-step)
 - Remediation plan (phased: immediate, short-term, medium-term, long-term)
- Timeline: Chronological events with severity filtering

6.4.3.5 Incident Management

1. Navigate to *Workspace* (top navigation)

2. Click “*Create Incident*” – fill title, description, severity
3. Link relevant events to the incident
4. Create tasks and assign to team members
5. View AI-generated incident narrative

6.4.3.6 Admin Panel (Admin Only)

- Model Training: Click “*Train RandomForest*” and the system downloads CICIDS2019, trains, evaluates, and activates if weighted precision ≥ 0.80
- Correlation Rules: Create/edit rules (threshold, sequence, existence, distinct value, time rate)
- Organization Management: View users and their roles

6.4.3.7 Common Workflows

6.4.3.7.1 Analyze a Windows Event Log

1. Export Windows Event Log as CSV (PowerShell: *Get-WinEvent -LogName Security | Export-Csv -Path events.csv*)
2. Upload to AegisFA
3. System auto-detects source type “Windows” and normalizes fields (*EventID, User, IPAddress*)
4. Review classification and MITRE mappings (e.g., EventID 4625 → T1110 Brute Force)

6.4.3.7.2 Check a Firewall Log

1. Upload a firewall log (e.g., pfSense, iptables output)

2. System maps *src_ip*, *dst_ip*, *action* to normalized schema
3. Correlation rules flag port scans or repeated denied connections

6.4.3.7.3 Generate a Remediation Plan

After analysis, click “*View Remediation*” next to any high-severity threat. The output includes:

- Immediate (0-1 hour): Block IP, disable account, etc.
- Short-term (1-24 hours): Update firewall rules, change credentials
- Medium-term (1-7 days): Patch systems, review logs
- Long-term (1-3 months): Implement MFA, security awareness training

6.4.4 Troubleshooting

Problem	Likely Cause	Solution
<i>401 Unauthorized</i>	Invalid or expired GitHub token	Log out and log in again
<i>403 Forbidden</i>	Insufficient role	Ask an admin to upgrade your role (analyst or admin)
Upload fails with <i>413</i>	File > 110 MB without session	Use chunked upload (automatic for large files)
<i>409 Duplicate</i>	File already exists in storage	Rename the file or delete the previous one
Analysis hangs	OpenAI rate limit or network issue	Check <i>OPENAI_API_KEY</i> and internet connection; jobs retry automatically

ML model not loading	Model not trained or below threshold	Go to Admin Panel → Train Model; wait for completion
Container memory high	Large log file + LLM context	Increase Docker memory limit (Settings → Resources)

Table 6.3: User Manual and Troubleshooting

6.4.5 Support & Logs

- Backend logs: *docker logs aegis-backend-1* (Docker) or *logs/* directory (manual)
- Frontend logs: Browser developer console (F12)
- Database logs: Supabase Dashboard → Logs
- Request tracing: Every error response includes a *request_id* – include this when reporting issues

6.4.6 Security Notes

- Do not use the default *SUPABASE_SERVICE_ROLE_KEY* in production – rotate it.
- Always use HTTPS in production (add a reverse proxy like Nginx + Let's Encrypt).
- Regularly update the Docker images (*docker-compose pull*).
- Review RLS policies periodically to ensure no unintended data exposure.

References

- IBM Security. (2025a). *Cost of a Data Breach Report 2025*. IBM Corporation.
<https://www.ibm.com/reports/data-breach>
- IBM Security. (2025b). *Cost of a Data Breach: The AI Breakdown 2025*. IBM Corporation.
<https://www.ibm.com/reports/data-breach>
- WEF (2025). *Global Cybersecurity Outlook 2025*. World Economic Forum.
<https://www.weforum.org/publications/global-cybersecurity-outlook-2025/>
- NIST (2024). *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*. National Institute of Standards and Technology. <https://csrc.nist.gov/pubs/sp/800/61/r3/final>
- Rahman, M.R., Mahdavi-Hezaveh, R., & Williams, L. (2021). What are the attackers doing now? Automating cyberthreat intelligence extraction from text on pace with the changing threat landscape: A survey. <https://arxiv.org/abs/2109.06808>
- National Institute of Standards and Technology. (2024). *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*. NIST. <https://www.nist.gov/publications/incident-response-recommendations-and-considerations-cybersecurity-risk-management-csf>
- McMillan, R. (2013). Definition: Threat intelligence. *Gartner*. <https://www.gartner.com/en>
- Sharafaldin, I., Lashkari, A.H., & Ghorbani, A.A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *Proceedings of the 4th*

International Conference on Information Systems Security and Privacy (ICISSP), 108–116. <https://www.scitepress.org/papers/2018/66398/66398.pdf>

Tounsi, W. & Rais, H. (2018). A survey on technical threat intelligence in the age of sophisticated cyber-attacks. *Computers & Security*, 72, 212-233.
<https://doi.org/10.1016/j.cose.2017.09.001>

APPENDIX A

Source Code Snippets

```
CREATE OR REPLACE FUNCTION match_mitre_techniques(  
    query_embedding vector(1536),  
    match_threshold float,  
    match_count      int  
)  
RETURNS TABLE (  
    technique_id text, name text, tactic text,  
    description text, similarity float  
)  
LANGUAGE sql STABLE AS $$  
    SELECT technique_id, name, tactic, description,  
           1 - (embedding <=> query_embedding) AS similarity  
    FROM   mitre_techniques  
    WHERE  1 - (embedding <=> query_embedding) > match_threshold  
    ORDER BY embedding <=> query_embedding  
    LIMIT match_count;  
$$;
```

A. 1: Supabase RPC Wrapper

```
structlog.configure(  
    processors=[  
        structlog.processors.add_log_level,  
        structlog.processors.TimeStamper(fmt="iso"),  
        structlog.processors.format_exc_info,  
        structlog.processors.JSONRenderer(),  
    ],  
    logger_factory=structlog.stdlib.LoggerFactory(),  
)
```

A. 2: Structured Logging Implementation Processors

```

for i in range(0, len(entries), RAW_LOG_INSERT_BATCH_SIZE):
    batch = entries[i : i + RAW_LOG_INSERT_BATCH_SIZE]
    rows = [
        {
            'org_id':    org_id,
            'source_id': source_id,
            'payload':    entry,
            'received_at': datetime.utcnow().isoformat(),
        }
        for entry in batch
    ]
    _supabase_insert_with_retry('raw_logs', rows)

```

A. 3: Raw Log Insertion

```

for batch_start in range(0, len(entries), BATCH_SIZE):
    batch = entries[batch_start : batch_start + BATCH_SIZE]
    batch_detections = _evaluate_rule(rule, batch, ...)
    # Adjust indices to absolute positions
    for det in batch_detections:
        det['matched_indices'] = [
            i + batch_start for i in det['matched_indices']
        ]

```

A. 4: Batching and Cumulative Offset for Large Datasets

```

def _extract_features(entry: dict) -> str:
    parts = [
        str(entry.get('message', '')) or '',
        str(entry.get('action', '')) or '',
        str(entry.get('status', '')) or '',
        str(entry.get('user', '')) or '',
        str(entry.get('ip', '')) or '',
        str(entry.get('event_type', '')) or '',
        str(entry.get('severity', '')) or '',
    ]
    return ' '.join(filter(None, parts))

```

A. 5: Feature Engineering (TF-IDF) Vectorization

```

calibrated = CalibratedClassifierCV(
    base_estimator=rf,
    method='sigmoid',
    cv=min(5, min_samples_per_class)
)
calibrated.fit(X_train, y_train)

```

A. 6: CalibratedClassifierCV

```

def _get_activation_status(validation_metrics, threshold=0.80):
    precision_weighted = float(
        validation_metrics.get('precision_weighted', 0.0)
    )
    if precision_weighted >= threshold:
        return 'active', precision_weighted, None
    return (
        'archived',
        precision_weighted,
        f'Weighted precision {precision_weighted:.3f} below threshold {threshold:.3f}'
    )

```

A. 7: Sigmoid Calibration and Logic Gates Deployment

```

def _build_mitre_query_text(findings, source_type, detections,
                             rf_context, log_entries):
    parts = [f"Log source type: {source_type}"]

    # LLM-detected threat types
    threat_types = [f.get('threat_type', '') for f in findings[:5]]
    if threat_types:
        parts.append("Threat types: " + ", ".join(threat_types))

    # Correlation rule names (indicate attacker TTPs)
    rule_names = [d.get('rule_name', '') for d in detections[:3]]
    if rule_names:
        parts.append("Detected patterns: " + ", ".join(rule_names))

    # Dominant RF categories
    by_cat = rf_context.get('by_category', {})
    top_cats = sorted(by_cat, key=by_cat.get, reverse=True)[:3]
    if top_cats:
        parts.append("Log categories: " + ", ".join(top_cats))

    return ". ".join(parts)

```

A. 8: Maximizing Retrieval Relevance

```

# Normalize individual signals to [0, 1]
llm_confidence = _extract_llm_confidence(findings)
retrieval_strength = (sum(m['similarity'] for m in mitre_techniques)
| | | | | / max(len(mitre_techniques), 1))
rf_risk_score = _compute_rf_risk(rf_context)
correlation_strength = min(len(detections) * 0.15, 1.0)

# Consistency bonus: all four signals agree on threat presence
signals_agree = (
| | | len(findings) > 0 and
| | | len(mitre_techniques) > 0 and
| | | rf_risk_score > 0.3 and
| | | len(detections) > 0
)
consistency_bonus = 0.12 if signals_agree else 0.0

confidence = min(1.0,
| | | 0.20 * llm_confidence
| | | + 0.20 * retrieval_strength
| | | + 0.30 * rf_risk_score
| | | + 0.30 * correlation_strength
| | | + consistency_bonus
)

```

A. 9: Confidence Score Implementation

```

class InsightsGenerator:
| | | def __init__(self):
| | | | | self.provider = os.getenv("INSIGHTS_LLM_PROVIDER", "openai")
| | | | | self.openai_model = os.getenv("INSIGHTS_OPENAI_MODEL", "gpt-4o-mini")
| | | | | self.allow_local_fallback = os.getenv(
| | | | | | | "INSIGHTS_ALLOW_LOCAL_FALLBACK", "true"
| | | | | ).lower() == "true"
| | |
| | | def _call_llm(self, prompt, max_tokens=512):
| | | | | if self.provider == "openai":
| | | | | | | try:
| | | | | | | | | return self._call_openai(prompt, max_tokens)
| | | | | | | except Exception:
| | | | | | | | | if self.allow_local_fallback:
| | | | | | | | | | | return self._call_local(prompt, max_tokens)
| | | | | | | | | raise
| | | | | return self._call_local(prompt, max_tokens)

```

A. 10: Insights Generator over two LLMs

```
# Session manifest stored as JSON in object storage
manifest = {
    'session_id': session_id,
    'org_id': org_id,
    'filename': filename,
    'source_type': source_type,
    'total_parts': total_parts, # None if unknown upfront
    'parts_uploaded': [],
    'created_at': now_iso(),
    'expires_at': (now() + timedelta(seconds=TTL)).isoformat()
}
storage.upload_json(manifest, manifest_path)
```

A. 11: Session State for Chunked Uploading

```
def _validate_parts(parts_in_storage, expected_count):
    part_numbers = sorted(
        int(p.split('/')[1].replace('.part', ''))
        for p in parts_in_storage
    )
    expected = list(range(1, expected_count + 1))
    if part_numbers != expected:
        missing = set(expected) - set(part_numbers)
        raise UploadIncompleteError(f"Missing parts: {sorted(missing)}")
```

A. 12: Chunk Validation

```
def upload_file(bucket, path, data, content_type=None):
    if content_type is None:
        content_type, _ = mimetypes.guess_type(path)
        content_type = content_type or 'application/octet-stream'

    for attempt in range(1, UPLOAD_RETRY_ATTEMPTS + 1):
        try:
            supabase.storage.from_(bucket).upload(
                path, data,
                file_options={'content-type': content_type}
            )
            return path
        except Exception as exc:
            if attempt == UPLOAD_RETRY_ATTEMPTS:
                raise StorageError(str(exc)) from exc
            delay = min(UPLOAD_RETRY_BASE_DELAY_SECONDS * attempt, 3.0)
            time.sleep(delay)
```

A. 13: Supabase Storage Implementation

```

function ProtectedRoute({ children, adminOnly }) {
  const [authState, setAuthState] = useState('loading');
  const [isAdmin, setIsAdmin] = useState(false);

  useEffect(() => {
    supabase.auth.getSession().then(({ data: { session } }) => {
      if (!session) { setAuthState('unauthenticated'); return; }
      const role = session.user?.app_metadata?.role
      || session.user?.user_metadata?.role
      || 'user';
      setIsAdmin(role === 'admin');
      setAuthState('authenticated');
    });
  }, []);

  if (authState === 'loading') return null;
  if (authState === 'unauthenticated') return <Navigate to="/login" replace />;
  if (adminOnly && !isAdmin) return <Navigate to="/dashboard" replace />;
  return children;
}

```

A. 14: ProtectedRoutes Implementation

```

export async function authenticatedFetch(input, init = {}) {
  const { data } = await supabase.auth.getSession();
  const token = data?.session?.access_token || '';
  const headers = new Headers(init.headers || {});
  if (token) headers.set('Authorization', `Bearer ${token}`);

  const response = await fetch(input, { ...init, headers });

  if (response.status === 401) {
    window.location.assign('/login?reason=expired');
    throw new HttpAuthError('Session expired');
  }
  if (response.status === 403) {
    throw new HttpAuthError('Insufficient permissions');
  }
  return response;
}

```

A. 15: Token Injection and Error Classification Handling

```
const CHUNK_SIZE = 4 * 1024 * 1024; // 4MB

async function uploadLargeFile(file, sourceType, orgId) {
  // Phase 1: Initialize session
  const { session_id } = await authenticatedFetch('/api/upload-sessions/init', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ org_id: orgId, filename: file.name, source_type: sourceType })
  }).then(r => r.json());
}
```

A. 16: Chunk Session System

```
// Phase 2: Upload chunks
const totalParts = Math.ceil(file.size / CHUNK_SIZE);
for (let partNumber = 1; partNumber <= totalParts; partNumber++) {
  const start = (partNumber - 1) * CHUNK_SIZE;
  const chunk = file.slice(start, start + CHUNK_SIZE);

  const formData = new FormData();
  formData.append('session_id', session_id);
  formData.append('part_number', partNumber);
  formData.append('file', chunk, file.name);

  await authenticatedFetch('/api/upload-sessions/upload-part', {
    method: 'POST', body: formData
  });

  setUploadProgress((partNumber / totalParts) * 50); // 0-50% for upload phase
}
```

A. 17: Chunk Session System

```
// Phase 3: Complete and trigger analysis
const { job_id } = await authenticatedFetch('/api/upload-sessions/complete', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ session_id, org_id: orgId })
}).then(r => r.json());

return pollJobStatus(job_id); // 50-100% for analysis phase
}
```

A. 18: Chunk Session System

```
async function uploadWithStreaming(file, sourceType, orgId) {  
  const formData = new FormData();  
  formData.append('file', file);  
  formData.append('source_type', sourceType);  
  formData.append('org_id', orgId);  
  
  const response = await authenticatedFetch('/api/upload/stream', {  
    method: 'POST', body: formData  
  });  
  
  const reader = response.body.getReader();  
  const decoder = new TextDecoder();
```

A. 19: Real-Time Progress Implementation

```
while (true) {  
  const { done, value } = await reader.read();  
  if (done) break;  
  
  const lines = decoder.decode(value).split('\n');  
  for (const line of lines) {  
    if (!line.startsWith('data:')) continue;  
    const event = JSON.parse(line.slice(5));  
  
    if (event.type === 'progress') {  
      setUploadProgress(event.progress_pct);  
    }  
    if (event.type === 'analysis_complete') {  
      setAnalysisResults(event.data);  
    }  
  }  
}
```

A. 20: Real-Time Progress Implementation

```
async function pollJobStatus(jobId) {
  const MAX_ATTEMPTS = 120; // 2 minutes maximum
  for (let attempt = 0; attempt < MAX_ATTEMPTS; attempt++) {
    const job = await authenticatedFetch(`/api/analysis-jobs/${jobId}`)
      .then(r => r.json());

    setUploadProgress(50 + (job.progress_pct / 2)); // 50-100%

    if (job.status === 'completed') return job;
    if (job.status === 'failed')
      throw new Error(job.error_message || 'Analysis failed');

    await new Promise(r => setTimeout(r, 1000)); // 1s interval
  }
  throw new Error('Analysis timed out');
}
```

A. 21: Polling Implementation

```
# Build stage
FROM node:20-alpine3.21 AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# Production stage
FROM node:20-alpine3.21
WORKDIR /app
COPY --from=build /app/dist ./dist
EXPOSE 4173
CMD ["npm", "run", "preview", "--", "--host", "0.0.0.0"]
```

A. 22: Build and Production Stage

```

services:
  backend:
    build: ./backend/ingestion
    ports: ["5001:5009"]
    environment:
      - SUPABASE_URL
      - SUPABASE_SERVICE_ROLE_KEY
      - OPENAI_API_KEY
      - INSIGHTS_LLM_PROVIDER=openai
    networks: [app-network]

  frontend:
    build: ./frontend
    ports: ["3000:4173"]
    depends_on: [backend]
    environment:
      - VITE_SUPABASE_URL
      - VITE_SUPABASE_ANON_KEY
      - VITE_API_BASE_URL=http://backend:5009
    networks: [app-network]

```

A. 23: Docker Compose Orchestration

```

networks:
  app-network:
    driver: bridge

```

A. 24: Docker Compose Orchestration

```

villegasjb@Joaquins-MacBook-Pro ingestion % curl -X POST "http://localhost:5006/rf/train" \
-H "Content-Type: application/json" \
-d '{
  "org_id": "1e62e598-f637-4b0a-854e-e67b12e22c88",
  "dataset_name": "CICIDS2019",
  "dataset_path": "/workspaces/data/cicids2019/archive",
  "seed": 48,
  "min_samples_per_class": 10,
  "max_rows": 180000
}'

%% new /agent conversation

```

A. 25: Manual Model Training for The Random Forest using CURL + POST command

```

    },
    "n_samples": 84000,
    "status": "trained"
  },
  "validation": {
    "accuracy": 0.9999444444444444,
    "confusion_matrix": [
      [
        6646,
        0
      ],
      [
        1,
        11353
      ]
    ],
    "f1_macro": 0.9999403665767902,
    "f1_weighted": 0.999944453107856,
    "labels": [
      "benign",
      "ddos"
    ],
    "precision_macro": 0.9999247780953814,
    "precision_weighted": 0.9999444528024339,
    "recall_macro": 0.9999559626563326,
    "recall_weighted": 0.9999444444444444,
    "samples": 18000
  }
},
"model_status": "active",
"model_version": "20260408203721-f7c2f9d9",
"model_version_id": "d77c126f-0e2c-4340-8560-393dfb1b705e",
"request_id": "56aa0722-e640-4186-a229-468ffdf7d4bb",
"split_policy": "70/15/15_stratified",
"status": "completed",
"training_run_id": "f6659cd1-35ea-4377-b028-d126d29a80eb",
"validation_precision_weighted": 0.9999444528024339
}

```

A. 26: Training Perfect Weighted Precision F1 Scores, Macros, and Confusion Matrix